Skyla Wakefield

Candidate No. 3675

King's School Grantham

Centre No. 26220

7517 Computing Project

Contents

1 Analysis

1.1 Background

TimBox is a software development company based in Grantham, England. The company was started in 2019 by John Smith and has grown to a size of around twenty employees. TimBox currently creates custom software for other businesses. To celebrate the company's fifth anniversary, John has decided to start a new branch of TimBox dedicated to developing video games.

John Smith is a family friend who studied graphic design at Boston college before starting TimBox. From this connection, I asked John if I could produce a game demo for the start-up of the new branch – an offer which he happily accepted.

This has led John to request that I make a 2d action-platformer. He wants the game to have a target audience of older children (13-15), so the game should be moderate in difficulty with a high skill ceiling to allow for an older, more experienced secondary audience (16+) to also enjoy the game. As well as standard platformer controls, John would like special movement options to provide multiple ways of beating a level – where the skill ceiling will come in. He would also like the game to feature combat, using a classic movement-oriented style, as opposed to a turn-based system. The game itself will take place in a magical, monster-infested tower, with the player's goal being to climb up to the top and end its curse. John has stressed his lack of desire for me to make the game to look good, as he will provide assets once the game is complete so I can focus on the game itself.

1.2 Investigation

1.2.1 Interview

To understand what type of game John wanted, I conducted an interview on Thursday 25th of January 2024. The interview went as follows:

"You originally suggested I make a platformer, but what type of platformer would you like?"
John - *"I would like the game to be a 2D side-scrolling platformer, broken into levels each representing a floor of the tower. I want the game to feel fast-paced, so it would be beneficial to have each level on a timer which – in conjunction with enemies blocking their path – will force the player to have a reactive playstyle."*

"You also mentioned that the game would include combat, could you elaborate on how you would like that to work?"
John – "I think the combat should be simple, which pairs with the platforming to produce fast-paced levels, whilst also allowing players to easily understand the controls, removing the need for long tutorials to introduce new players."

"Overall, how difficult would you like the game to be?"
John – *"I would like the game to start easy, with little to no monsters so the player can get used to the controls, but the game should progressively get harder in both platforming and monster encounters. The changes in difficulty should be offset by the player gaining new abilities, but I would also like the game to still be possible using only basic controls."*
"Some games have difficulty options, do you think that suits this game?"
John – *"I would rather the game gives time for the player to get better, rather than present difficulty options at the start."*

"Finally, do you have any ideas for how enemies will work in the game?"
John – *"I want the enemies to act as a blockade to the player's progress in a level, which cannot be ignored in most cases. I think it would be good to have a variety of enemy types which function differently from one another. For example, one enemy could be flying at the top of the level and swoop down to attack, whereas another sticks to a platformer and slides around its perimeter. It may also be good to include some random factors to each enemy, to make each playthrough feel different, such as altering speed, health, or damage."*

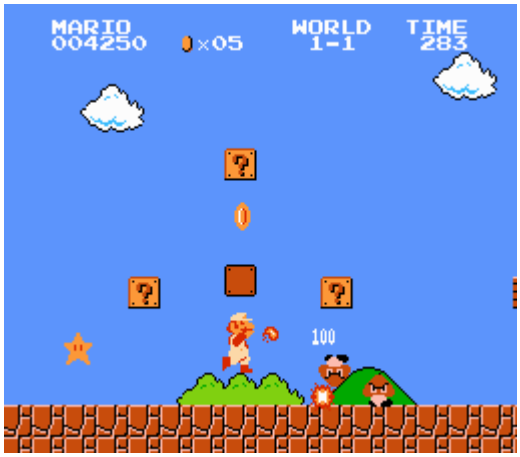
From this first interview, I have identified a few key requirements for my program:

- The game screen should scroll to keep the player on screen.
- Levels are timed.
- Player – Enemy combat should be simple (Click a button to attack)
- Standard Up, Down, Left, Right controls.
- The game should contain a variety of enemies which each function differently.
- Enemies are generated with slight mutations to vary speed, health, and/or damage.

This has led me to research other games like what has been requested, so I can determine how existing systems have fulfilled the given criteria, as well as look into game mechanics which were not brought up in this interview.

1.2.2 System Research

Mario:



The original Super Mario Bros for the NES contains many of the features that John has requested. In this title, Mario must run to the right of the screen to reach the flag at the end of each level, facing light platforming challenges, and dealing with enemies along his path. Each level consists of a grid of tiles, which get drawn over a background image (in this level it is a sky which acts as the background).

The first feature which is shared is side scrolling. When Mario approaches the final tiles of the screen, the game stops moving Mario to the right, and

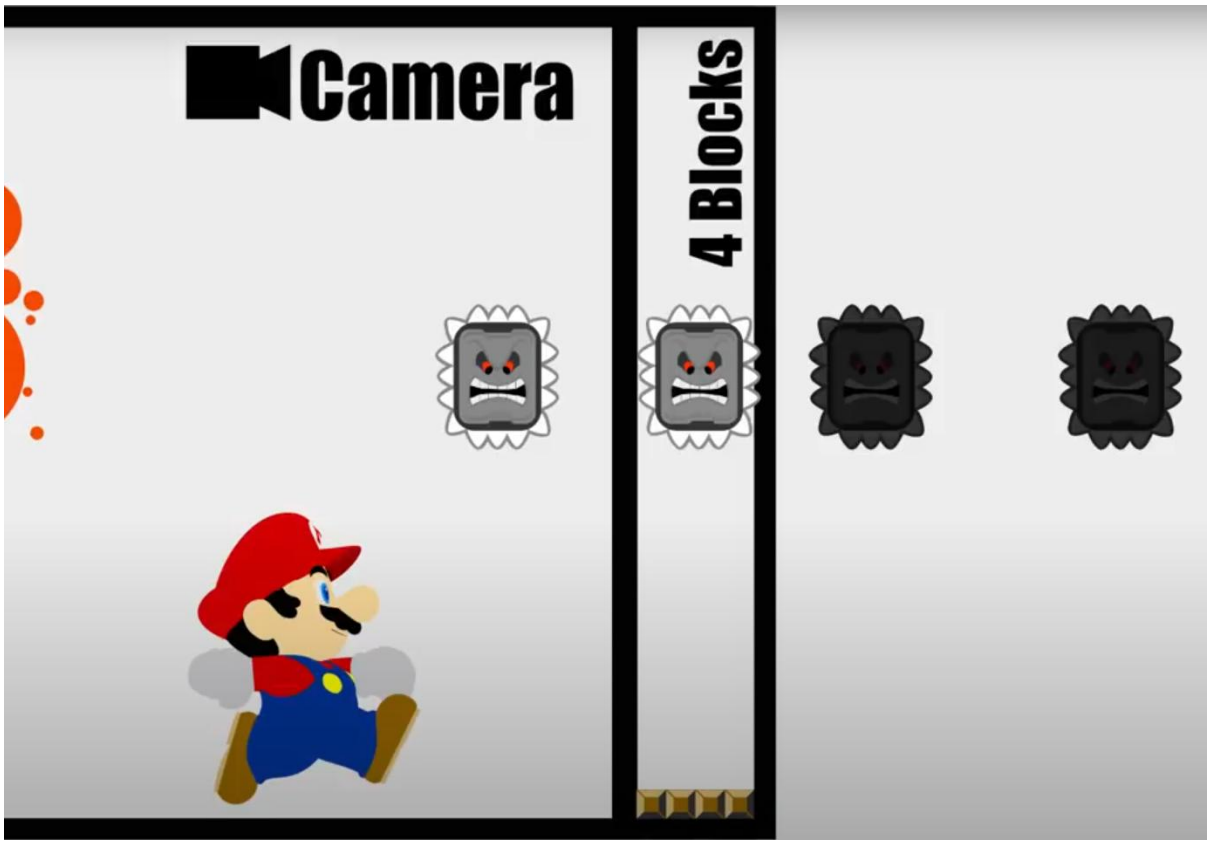
instead moves the background to the left, preventing Mario from running off the screen. This scrolling is not perfect, however; as the screen will only scroll when Mario runs right, with an invisible wall blocking the left side of the screen. This is because the game loads in two screens worth of tiles whilst you play and replaces the tiles no-longer on screen with tiles from a future screen. While this is more memory efficient than loading in the whole level at once, it drastically reduces the movement options available to the player, something I would like to avoid.

As Mario runs through each level, he can collect one of four power-ups, which act both as a health bar, and a change in move-set. The player starts as small Mario, with only one point of health. By grabbing a Super Mushroom as small Mario, the player becomes Super Mario, becoming taller and gaining an extra health point. By grabbing a Fire Flower, the player becomes the Mario on screen, gaining an extra health point and the ability to throw fire balls, which bounce on the floor until they hit a wall or an enemy. Mario can grab a Star, which (for 10 seconds) makes Mario invincible, faster, and instantly defeat any enemy that makes contact. Finally, the player can grab a 1-Up Mushroom which increases the number of times the player can lose a level before having to reset. Each time Mario is hit, he either becomes small Mario, or dies, forcing the player to retry the level.

This game also has the player on a timer (top right corner), which forces the player to beat the level within three hundred seconds or fail and be sent to the start and must do it again. While three hundred seconds is generous for most Mario levels, every one second left on the timer when the level is beaten earns the player fifty points for their score (top left corner). This incentivises the player to beat each level as quickly as possible, to earn the maximum points possible (even though the score does not affect anything in the game and is purely cosmetic).

Super Mario Bros uses only the four directions as inputs, plus an A and a B button. The direction inputs are used to move left and right, with up and down used to interact with some objects in the environment. The A button is used to jump, and the B button is mapped both to run and to fire projectiles as Fire Mario. This game includes some refinements to how the player moves, to provide a smoother gameplay experience. The game uses a drag factor when moving horizontally, gradually accelerating the player to a top speed when a direction is held or slowly changing direction when the opposite direction is pressed. This effect allows for small adjustments in the player's position, whilst preventing the player from coming to a halt after running at top speed. There are also improvements to jumping. The first of these is altering jump height based on how long the player holds the A button. The second of these is known as "Coyote time," where the player has additional frames to perform a jump after falling off an edge. Both Coyote time and drag is minimal – only lasting a few frames – but these slight adjustments provide more realistic and smooth movement.

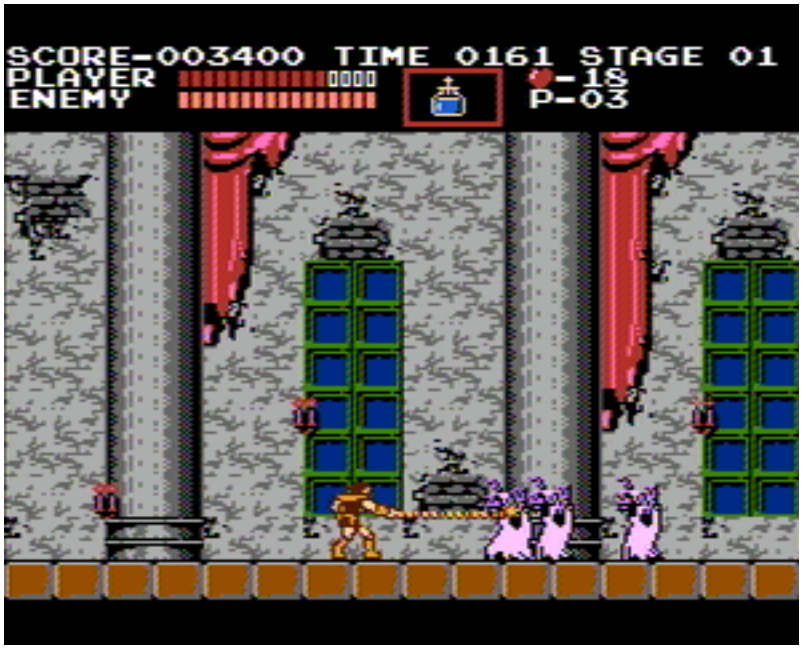
Mario Maker:



(Graphic from *"Don't hit the Run Button" in Super Mario Maker!* by Ceave Gaming on YouTube - https://www.youtube.com/watch?v=-FbssndzWjg&t=485s)

Mario Maker is a game engine which lets players create custom Mario levels using a predefined tile set. Their level editor works by selecting the tile you want from a tile palette, then placing it onto a grid which is converted into a level upon pressing play. While the gameplay of Mario levels in Mario Maker is similar to the above, only tiles which are onscreen ±4 tiles area active. This logic will be useful to limit the objects which will be considered during collision detection, as well as ignoring enemies which are off-screen from using processing time to move without being seen.

Castlevania:



In the original Castlevania, you play as Simon Belmont, who is trying to find and kill Count Dracula in his tower. As the player does so, they must face Dracula's minions as well as platforming challenges which block their path.

In terms of platforming, Castlevania is a lot simpler than Super Mario Bros, and anything done by Castlevania has already been covered in the Mario section.

The primary reason I have investigated Castlevania is due to its simplistic combat. The player's main attack is a whip, which will damage enemies which are up to three tiles in front of the player. To prevent the player from rapidly pressing the attack button, the whip (known as the "Vampire Slayer") has a brief charging period, as well as a short attack animation, which are required to carry out the attack. These animation pauses force the player to space themselves away from monsters to actually carry out the attack. The player also has access to alternative attacks, such as throwing holy water, which burns enemies a few tiles away, or a hand-axe which can be thrown like a boomerang across a longer range than the whip. These alternatives can be collected as you play through the game and provide more flexibility in playstyle.

The main targets of your attacks are monsters which will spawn in small groups, which need to be slain before they can hit you back. While most enemies only need one hit of the whip to be taken out, Others can take up to twelve. The main distinction between each enemy is the actions they can use. Some enemies – such as the zombie - have a basic AI which can only move towards the player (like a slightly more advanced Goomba). Other enemies like the vampire bat fly at the top of the stage before swooping down to attack, requiring you to react quickly, or prioritise dodging the attack. Other enemies move too quickly for the whip to be effectively, and others attack from above (like the vampire bat), requiring the use of the arching hand-axe to be taken down.

Castlevania's combat system is simple in execution, with the game just checking if an enemy collides with a weapon, and dealing damage accordingly, but the variety of enemies, the number which are on screen at any given time, and the different ways in which they need to be dealt with allows this simple system to be more than enough.

1.2.3 Interview

After looking into these games, I conducted another interview to ask about things I have learned, as well as any points that were not raised in the first interview. I conducted the interview on February 1st, 2024.

In terms of mechanics, John agrees that Mario's platforming adjustments would be beneficial. The following are questions which were not asked in the first interview:

"Many retro games do not offer the ability to save the game. Is that a feature which you would like to be included?"
John – *"Very much so. I want people to be able to pick up the game and play a few levels every day, rather than having to reset every time they play. I would like the game to save after every level, so players can gradually chip away at the game across a few weeks. I would also like the game to have three save files, allowing multiple people to play through the game at their own pace, rather than getting frustrated at the harder levels which they would not have reached yet."*

"In the existing games I researched, the enemies would appear quite frequently. How frequently would you like enemies to appear?"
John – *"I would like enemies to appear in small but frequent hordes throughout the level. Whilst some groups can be avoided with good platforming, others should be necessary encounters for the player to progress."*

"Different games handle health in a variety of ways, so how do you want the game to handle health?"
John – *"I want the game to use a simple health system: You get hit, you lose some health, and if your health drops below 0, you die. The player should start the game with ten health and five lives. When the player dies, they will get sent back to the start of the level and lose a life. If the player loses all their lives, they should be sent back to where they last saved. The health system should also apply to enemies, who will be removed from the game when their health drops below 0."*
"And how can the player get back health?"
John – *"I would like the game to contain healing items which can be used to restore a small amount of health but they should appear infrequently (maybe once or twice) in each level."*

"Some games have different types of platforms other than floor tiles; do you have any which you want in your game?"
John – *"Alongside the standard floor tile, I would like the game to have one-way platforms which the player can jump through then land on. I would also like the game to have honey tiles, which stop the player from being able to jump – this can be placed onto floors to make them sticky or appear as walls which slow the player down. Finally, I would like a spike tile which will damage anything that walks into it."*

"Finally, I will be making a level editor to make creating levels a lot easier for me. Should this feature be available to the player?"
John – *"That would be a particularly good addition to increase the playtime of the game. This level editor should allow players to create and save their own custom levels, as well as edit any of previously saved levels."*

1.2.4 Pygame Research

I intend to make this project using python, due to its module system and general utility. The pygame library provides a set of functions and classes specifically to make games in python.

The primary function of pygame in this project will be for input and output. I need to accept keyboard presses and mouse clicks as input. This can be done with the pygame.key module and

the pygame event system. On that note, pygame provides an event system, which will be used to monitor mouse inputs, but also to exit the application.

The program needs to draw the game to the monitor, done using the pygame.display class. This is a specific implementation of the pygame.surface class, which includes a "flip" function to draw the image to the screen.

The pygame.surface class is just a class to represent an image. It offers auxiliary functions to get the image dimensions, but more importantly, it offers a function to fill the image with a single colour – useful to set a background or to clean the old image – as well as a function to draw a different image onto the surface (called blitting). As one would expect, I will use this class to store the visual elements of the program.

Another important class is the pygame.rect. This just stores the position of the top-left corner, the width, and the height – the dimensions of a rectangle. This is useful, as it provides a system for hitbox, needed to make sure the player does not fall through the floor, and in combat to see if a weapon intersects with the player. To support this, the rect has a set of functions to quickly check for a collision, being the collidepoint(), colliderect(), and collidelist() functions. These check if a point or rectangle would collide with the rect or return the first occurrence of a colliding rectangle in the collidelist() case.

The final aspect of pygame I will be using is the Clock() class. This will be used to ensure that the game runs smoothly at 60 frames per second – stalling the logic until a 60th of a second has passed since the start of the frame. Importantly, this will not call the logic every 60th of a second, but rather, act as a barrier to stall execution, ensuring smooth visual updates.

Demonstrating the research

```
########################################          SECTION 1              ####################################
import pygame

def reveal_mouse(btn: int):
    print(list(pygame.mouse.get_pos()))                                 #Gets the mouse's position
    match btn:
        case 1: print("1: Left")
        case 3: print("3: Right")
        case 2: print("2: Middle")

def tick_colour(colour, amount, a) -> list:
    colour[a] = min(max(colour[a]+amount,0),255)
    return colour


pygame.init()                                                           #"Turns on" pygame
screen = pygame.display.set_mode((256,256))                             #The screen is a surface object
clock = pygame.time.Clock()                                             #Counts the time elapsed

example_img = pygame.image.load("Example_Img.png").convert()            #Loads an image without transparency
transparent_img = pygame.image.load("Transparent_Img.png").convert_alpha()  #Loads an image with transparency
transparent_img = pygame.transform.scale(transparent_img, (64,64))      #Resizes an image

running = True
hiding = False
bg_col = [0,0,0]
ex_pos = [0,0]
while running:
    pygame.display.flip()                                               #Draws the game to the screen

    screen.fill(bg_col)                                                 #Fills a surface with a single colour

    keys = pygame.key.get_pressed()                                     #Checks which keys are currently down
    if(keys[pygame.K_p]):                                               #Check if "p" is down
        hiding = not hiding
    if(keys[pygame.K_DOWN]):                                            #Check if the down arrow is down
        bg_col = tick_colour(bg_col, 1, 0)
        ex_pos[1] += 1
    if(keys[pygame.K_UP]):                                              #etc
        bg_col = tick_colour(bg_col, -1, 0)
        ex_pos[1] -= 1
    if(keys[pygame.K_LEFT]):
        bg_col = tick_colour(bg_col, -1, 1)
        ex_pos[0] -= 1
    if(keys[pygame.K_RIGHT]):
        bg_col = tick_colour(bg_col, 1, 1)
        ex_pos[0] += 1
    screen.blit(example_img, ex_pos)                                    #Draw the example player
```

```
    screen.blit(transparent_img, (32,160))                              #Draw an example goal
    screen.blit(transparent_img, (160,32))                              #Draw a second example goal

    ex_col = pygame.Rect(ex_pos, (32,32))                               #Create a rect for the above objects
    tA_col = pygame.Rect((32,160), (64,64))
    tB_col = pygame.Rect((160,32), (64,64))
    exampleList = [tA_col, tB_col]
    if(ex_col.collidelist(exampleList) != -1):                          #Test collisions over a list
        tick_colour(bg_col, 1, 2)
        print("Entered a goal", end = '')
        if(pygame.Rect.colliderect(tA_col, ex_col)):                    #Test a collision between two rects
            print(" - A")
        elif(pygame.Rect.colliderect(tB_col, ex_col)):
            print(" - B")
    else:
        tick_colour(bg_col, -1, 2)

    for event in pygame.event.get():
        if(hiding):
            continue
        if(event.type == pygame.MOUSEBUTTONDOWN):                       #See if a mouse is down
            reveal_mouse(event.button)
        if(event.type == pygame.MOUSEBUTTONUP):                         #See if a mouse is up
            reveal_mouse(event.button)
        if(event.type == pygame.KEYDOWN):                               #See if a key is down
            print(pygame.key.name(event.key))
        if(event.type == pygame.QUIT or keys[pygame.K_ESCAPE]):         #See if the player has quit
            pygame.display.quit()
            running = False
    clock.tick(60)                                                      #Caps the program at 60 fps

########################################          SECTION 2             ####################################
import time
def clock_display(runs: int):
    start = time.time()
    for x in range(runs):
        lstA = [int(str(y)) for y in range(1000)]
        lstB = list(map(lambda z: z + 1, lstA))
        lstC = sum(lstB)
        lstD = [ele + lstC for ele in lstA]
    mid = time.time()
    clock.tick(5)                                                       #Caps the program at 5 fps
    end = time.time()
    print("\nTime spent = ", end-start)
    print("Time used = ", mid-start)
    print("Time stalled = ", end-mid)

clock_display(250)
clock_display(1000)
```

The above code is for a simple program to demonstrate everything I have mentioned beforehand. Comments have been used only to identify which part of the pygame package is being demonstrated. Section 1 is a simple "game" to demonstrate the functions I will use, and Section 2 demonstrates that the clock class is only a barrier.

Test Video: https://www.youtube.com/watch?v=Z1Wc4TBE1hE

1.3 Program Requirements

From the above research, I have created a list of key requirements which my program must fulfil:

1) The player can move left and right using A/D respectively, or using the relative arrow key, as well as being able to jump by pressing W or the Up Arrow Key.
2) Include Mario's movement improvements (see 1.2.2)
3) The game screen should scroll to keep the player on screen.
4) The game must contain the following tile types:
    - Floor/Wall tile
    - One way tile
    - Honey tiles (slow the player and prevent jumping)
    - Spike tile (damage the player)
5) Levels are loaded from a file, and are accurately assembled to be played
6) Beating a level causes the next level to begin
7) Levels must be beaten in one hundred seconds
8) The player must be able to save
9) The player must be able to create a save (if space allows)

10) The player must be able to delete an existing save (to make space)
11) The game must correctly load the relevant form – covered later in 2.3
12) Contain the above health system
13) The player can attack enemies to lower their health (and the inverse), with one attack animation doing one attack's worth of damage
14) Levels contain enemies which:
    - Attack and move in diverse ways
    - Have mutations which occur randomly between playthroughs, altering enemy stats.
15) The player can restore health by using healing items
16) The player can update their movement options using consumable items

The game should contain menus which:

1) Respond to mouse input – for example;
    - A button does a function when clicked
    - An image does not do a function when clicked
2) Connect to one another to separate information
3) Can be dynamically updated
    - A list of save files updates after a death/level clear
    - A list of custom levels can grow as levels are added

The game should also contain a level editor which:

1) Allows the player to create custom levels
2) Level should be created by "painting" tiles onto a grid
3) The player can save the levels they create
4) The player can edit the levels they have saved
5) The player can play the levels they have saved
6) Elements can be placed anywhere
7) The game should check to make sure the level is playable – and not running a "bad" level

1.4 Data Volumes

1.4.1 Whilst Playing A Level (During Runtime)

Assuming a screen will be made up of a grid of twenty-four by sixteen tiles, with ten screens per level, and only one object per tile, there will be a maximum of 3840 objects being stored in RAM. Since most of the screen will be empty to allow the player to move, it is more likely for only 768 objects (1/5 of the max) to be in RAM at any point in time.

I intend on my game running at 60 frames per second and will only monitor at most ten keys from the keyboard, so the maximum keyboard inputs to be interpreted per second is equal to six hundred per second. Although, this number is unlikely to be reached, as keyboards tend to max out at about three thousand key inputs per minute (achieved easily by holding down a key), giving a true maximum keyboard inputs of fifty inputs per second (much more manageable).

The game will also have variables for the player's health (and similarly for enemies), but these are simply a few integers, so will take up negligible space.

1.4.2 General Storage

Below is a table containing the different objects which are being stored even whilst the program is not running:

Data being saved	How many will be stored?	How many will be added daily?
Levels	100	5
Enemy Types	3	0 – All preset
Tile Types	7 (4 tile types, with one-ways having four unique directions)	0 – All preset
Healing Item Types	3	0 – All preset
Upgrade Types	5	0 – All preset
Player Saves (stored in a text file)		
How many saves per game?	3	0 – but the three will be rewritten every time the game is played
Player Username	One per save	0 – the exact value will change, but no more will be added
Game Metadata (Current Level, Current Lives and Current Health)	One of each per save	0 – the exact numbers will change, but no more will be added

Note: enemy, tile and healing items are stored in the code whereas the levels and save data will be files (discussed in 1.5).

1.5 Analysis Data Dictionary

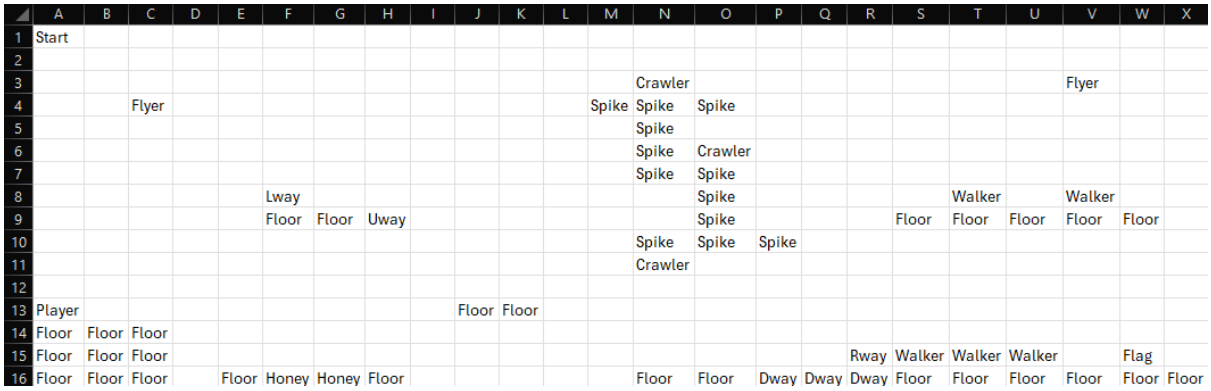
The game will store three sets of save data, allowing multiple people to save their progress in their respective playthroughs. All saves will be stored in a shared .txt file, with each save being on their own line, and each data value being separated by a comma. The values I will store for each save are:

- The player's username (used to distinguish between files)
- The player's current level
- The player's current health (both lives and damage)

This will be stored a string for the username, and three integer values for the metadata per save

The levels to be played also need to be stored. As this is a tile-based game, I will store levels in a .csv file, with a grid of strings each representing the name of a tile – "" is an empty tile. Each level will get its own file to make them easy to maintain.

All the tiles will be stored as classes with their functionality preset which can be instantiated as required.



	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
1	Start																							
2																								
3														Crawler								Flyer		
4			Flyer										Spike	Spike	Spike									
5														Spike										
6														Spike	Crawler									
7														Spike	Spike									
8						Lway									Spike					Walker		Walker		
9						Floor	Floor	Uway							Spike				Floor	Floor	Floor	Floor	Floor	
10														Spike	Spike	Spike								
11														Crawler										
12																								
13	Player									Floor	Floor													
14	Floor	Floor	Floor																					
15	Floor	Floor	Floor															Rway	Walker	Walker	Walker		Flag	
16	Floor	Floor	Floor		Floor	Honey	Honey	Floor						Floor	Floor	Dway	Dway	Dway	Floor	Floor	Floor	Floor	Floor	Floor

This is an example of a level in the .csv format I will be using (loaded in Microsoft Excel). It should be noted that the cell A1 contains "Start". This is due to a quirk of compression csv uses. If "Start" was not present, then rows 1 and 2 would be wholly empty, which the csv format views as unneeded, and thus it deletes them. This only happens up to the first row containing a value, so the "Start" stops the first rows from being deleted.

To allow for every tile to be used, the "Start" only needs to be present if A1 would otherwise be empty, as .csv only checks IF something is present, not WHAT is present.

1.6 Gameplay Loops and Interactions

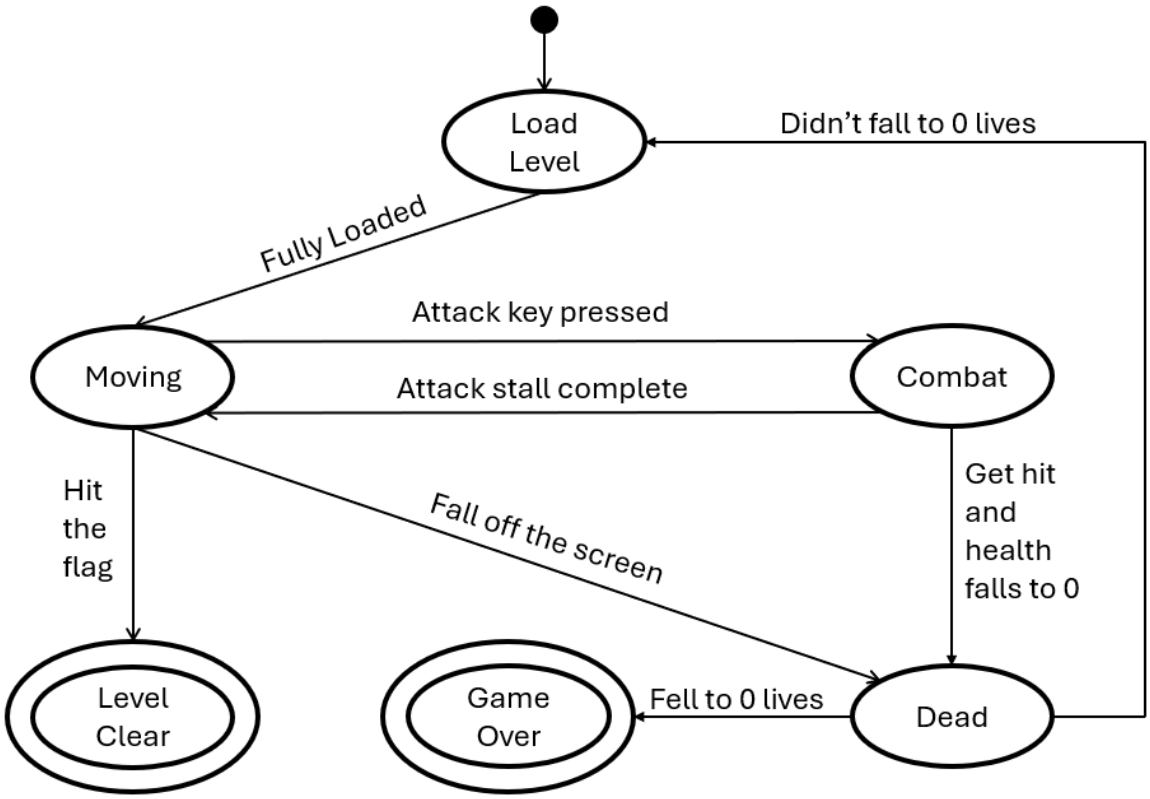
The main game will consist of a primary loop of platforming to the flag, then loading the next level, then repeat. There will be a secondary loop of attack enemies, then avoiding their attacks to break up the platforming gameplay. There will also be a secondary loop of exploring alternate paths to get movement updates. There will be a final gameplay loop to beat the level as fast as possible – enforced and measurable with the timer.

The following is a table of interactions for the game:

Type	Cause	Effect
Player to System	Click	The game updates the screen
Player to System	Arrow keys pressed	The player character moves left, right, and up respectively
System to System	Player stands off of a platform	Apply gravity to accelerate and fall downwards
System to System	Player touches a platform	The player stops falling and has a special effect applied based on the platform – Honey slows the player and prevents jumping, Spikes deal damage
Player to System	Player presses the attack key	The player performs an attack
System to System	An object touches an attack hitbox	The object takes damage if the attacker was on the opposite team
System to System	Player object touches the flag	The game closes the current level, and loads the next one

1.7 Finite State Machines and Flowcharts

1.7.1 Gameplay Finite State Machine



The game starts in the "Load Level" state. Here the program reads the relevant level file and instantiates objects for each tile and entity. The program will then automatically transition to the "Moving" state.

The user has input on the gameplay during the "Moving" and "Combat" states. All other states are entirely controlled by the program.

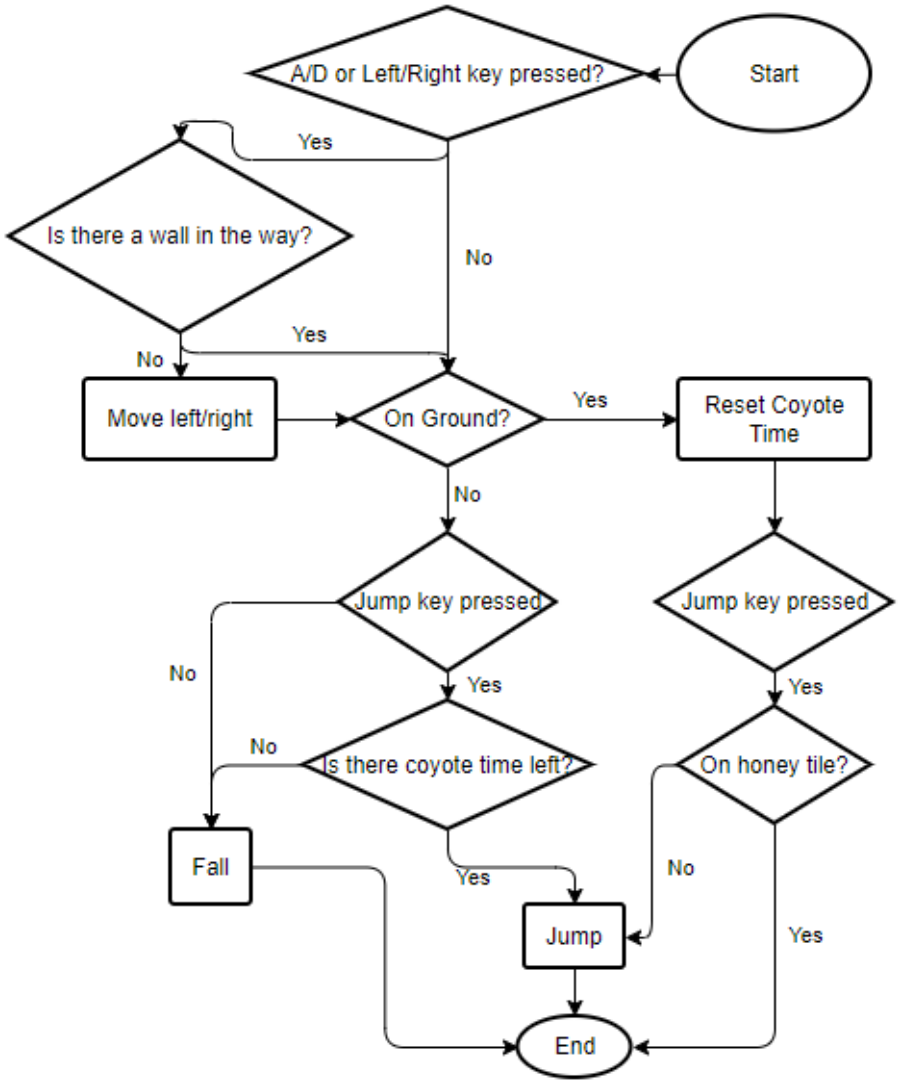
The "Moving" state refers to when the player is not in combat but has control over the character. The character might not necessarily be moving, but the character is not attacking nor being attacked. In this state, the program performs gravity and collision logic.

In the "Combat" state, the player is either performing an attack or being attacked by an enemy. In this state, the program performs logic to see if an attack connects with a target, and then any related logic such as dealing damage.

There are two death conditions: Fall off the screen or fall to/below 0 health. In both cases, the game enters the "Dead" state, where it stops taking user input and determines whether to restart the level, or perform a game over. During this state, the game will perform a short animation to indicate that the player is dead, then change states according to how many lives the player has remaining. If the player falls to zero lives the game enters the "Game Over" state, where the game terminates and returns to the file select screen. In this state, the program needs to save the game to reflect the game over.

If the player touches the flag, the game enters the "Level Clear" state. In this state, the game is terminated, with the next level subsequently being loaded, and the save file being updated.

1.7.2 Movement Flowchart



Movement is broken into two sections: horizontal, and vertical.

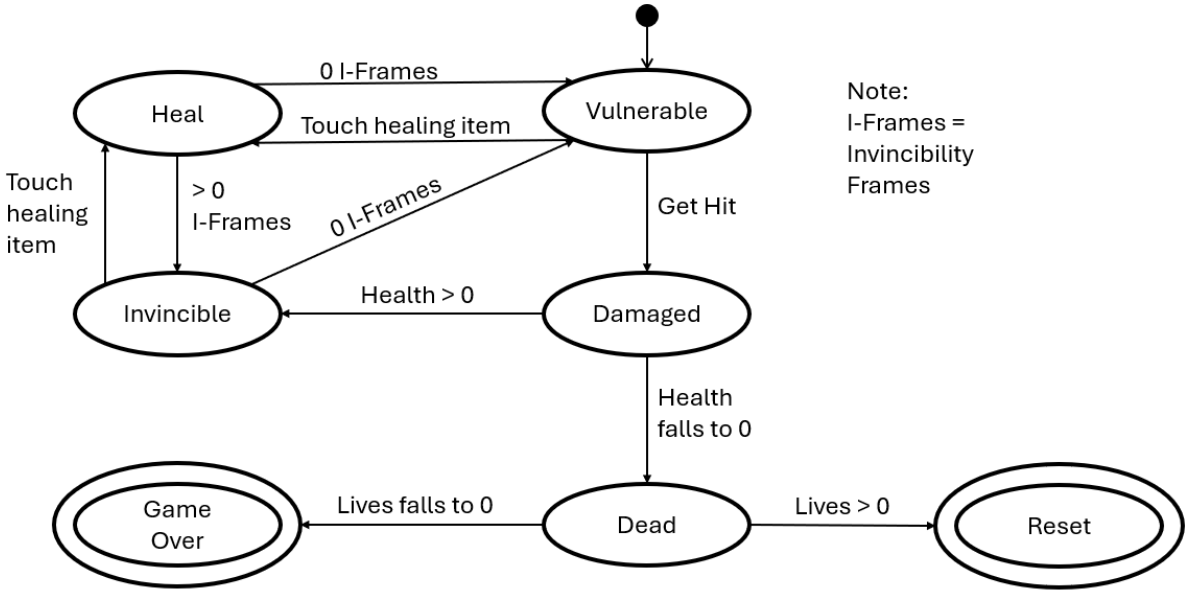
In the horizontal section, the program checks if the player is pressing one of the A, D, left arrow, or right arrow keys. If so, the program needs to move the player, if not then no horizontal movement is required this frame. Before moving horizontally, the program checks if the player is going to move into a wall – a movement we want to prevent. If so, the program cancels any horizontal movement, otherwise we perform logic to move left or right according to which keys were pressed.

In the vertical section, we first ask if the player is grounded. If so, then the player can jump freely, and they gain coyote time for when they are next airborne. If the player is grounded, and they press the jump button, we check if they are touching a honey tile. These tiles prevent vertical movement, so if we are touching a honey tile, we end the vertical movement (which reaches the end state), else we perform a jump , which then ends the vertical logic.

If the player was not grounded, they may still be able to perform a jump shortly after leaving the ground (coyote time). We first check if the player even wants to jump. If not, then they simply fall, and the vertical logic is done. If they do, however, then we check if the player still has coyote

time – a timer which decrements whilst the player is falling. If they do not, then the player falls. If they do, then we instead perform a jump as though the player were grounded.

1.7.3 Damage Finite State Machine



The player starts in the "Vulnerable" state. As the name implies, in this state the player is vulnerable to sources of damage, such as enemy attacks or spike tiles.

If the player takes damage, they transition to the "Damaged" state. This is a transition state, used by the program to determine whether the player is dead or not. If not, they transition to the "Invincible" state. In this state the player is immune to all sources of damage and will automatically transition after a timer (I-Frames, or invincibility frames) reaches 0.

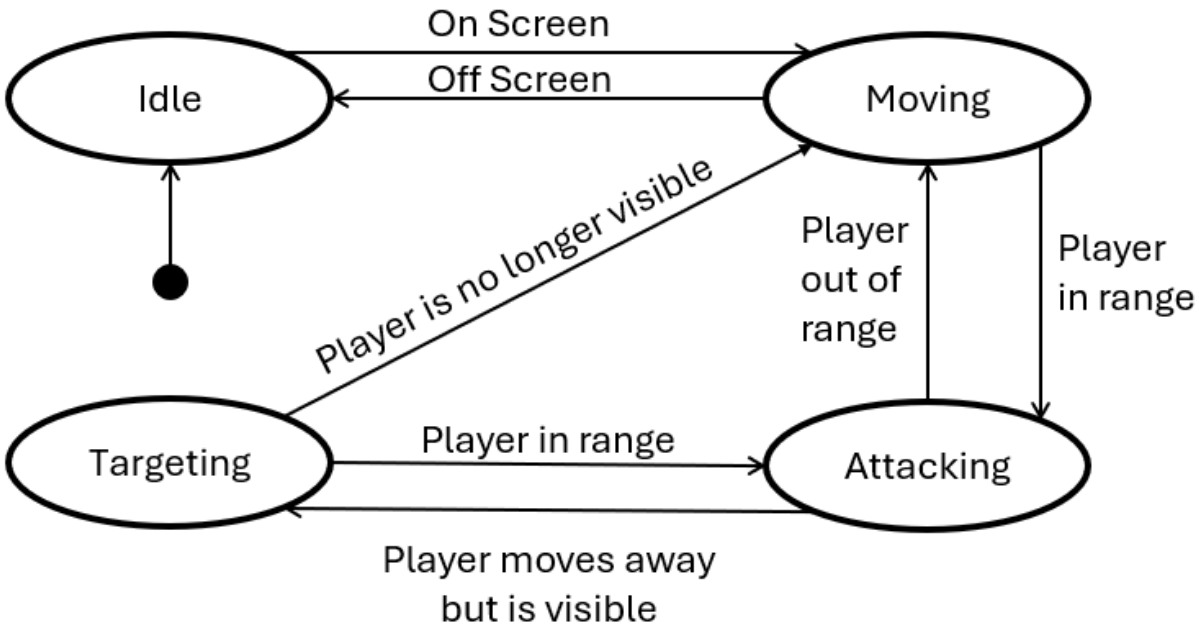
The "Invincible" state can also be reached by touching a healing item. If this occurs, the player transitions to the "Heal" state, where the player is healed, and its I-Frames timer is increased. After touching a healing item, the player will almost always transition immediately to the "Invincible" state.

Once the player's I-Frames timer reaches zero, they will transition to the "Vulnerable" state. These four states comprise the main loop which the player will stay in during the runtime of the game.

If the player enters the "Damaged" state and their health falls to zero, they will instead leave the main loop and transition to the "Dead" state. This is another transition state and is used to determine what to do if the player dies. If the player still has lives left, they transition to the "Reset" state, which terminates the game and reloads the level. If the player has no lives left, however, they will transition to the "Game Over" state. In this state, the game is fully terminated, and the file select screen is loaded.

Both the "Reset" state and the "Game Over" state will be used to save the game to reflect the change to the player's life count.

1.7.4 Walker Control Finite State Machine



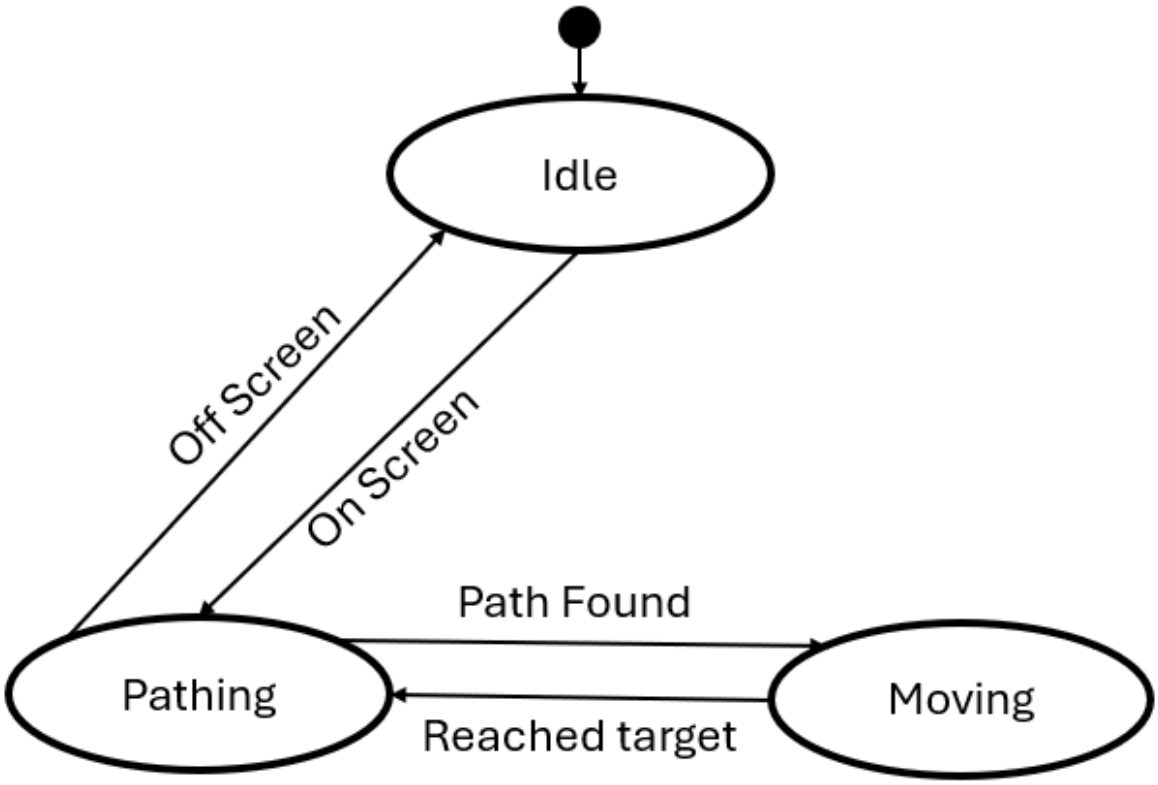
A walker enemy starts in the "Idle" state. In this state the walker is idle and will not move or attack. This is so the program does not waste time with logic for an entity which has no impact on what's happening on screen.

Should the walker become visible, it enters the "Moving" state. In this state it performs the logic to walk back and forth on a platform.

If the player gets in range of a moving walker, the walker transitions to the "Attacking" state. In this state, the walker is stationary and tries to perform an attack each frame (attack logic handled by the main program).

Should the player get out of range of the walker, but the player is still visible to the walker, then the walker will transition to the "Targeting" state. In this state, the walker performs a modified version of the "Moving" state, but the walker will not turn around if it reaches an edge – it will just stay still. If the player is no longer visible, we return to the "Moving" state, which results in the program dropping the modified movement logic.

1.7.5 Flyer Control Finite State Machine

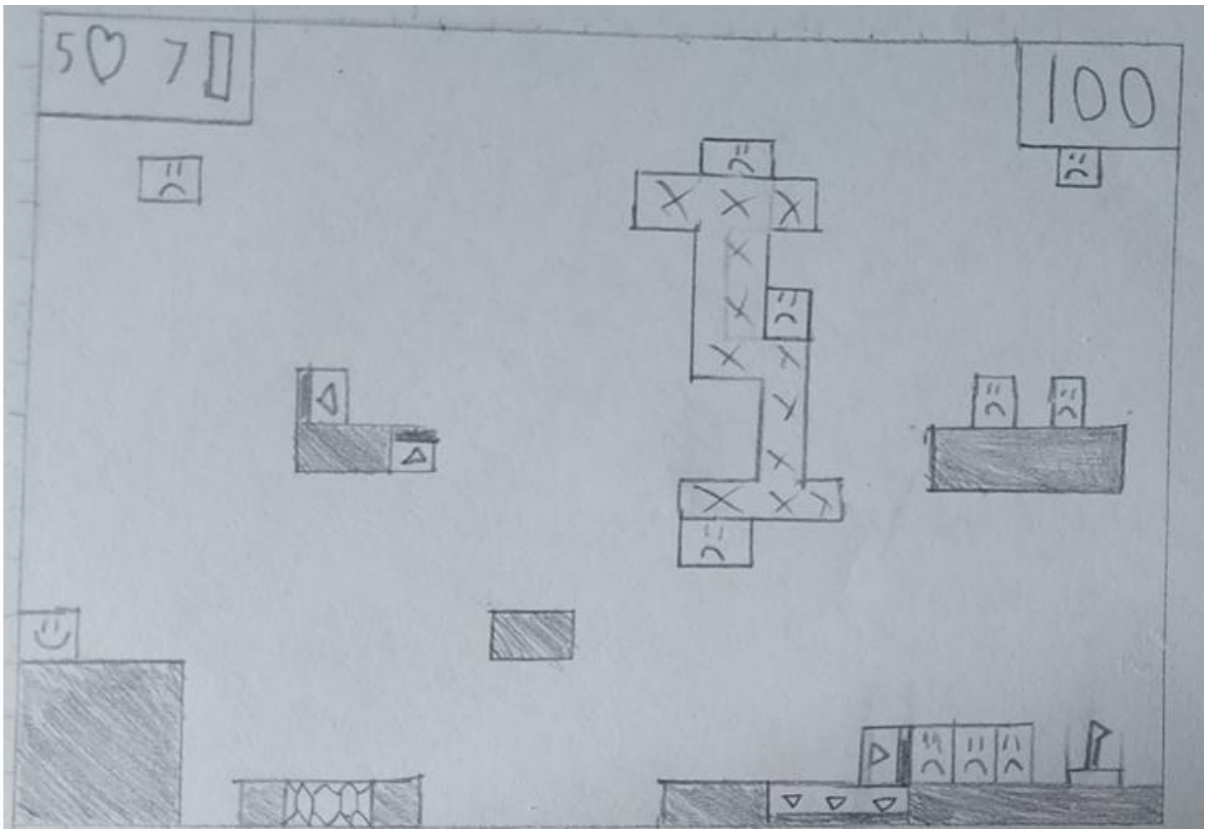


A flyer starts in the "Idle" state. Like walkers, in this state they do nothing, to save the program time.

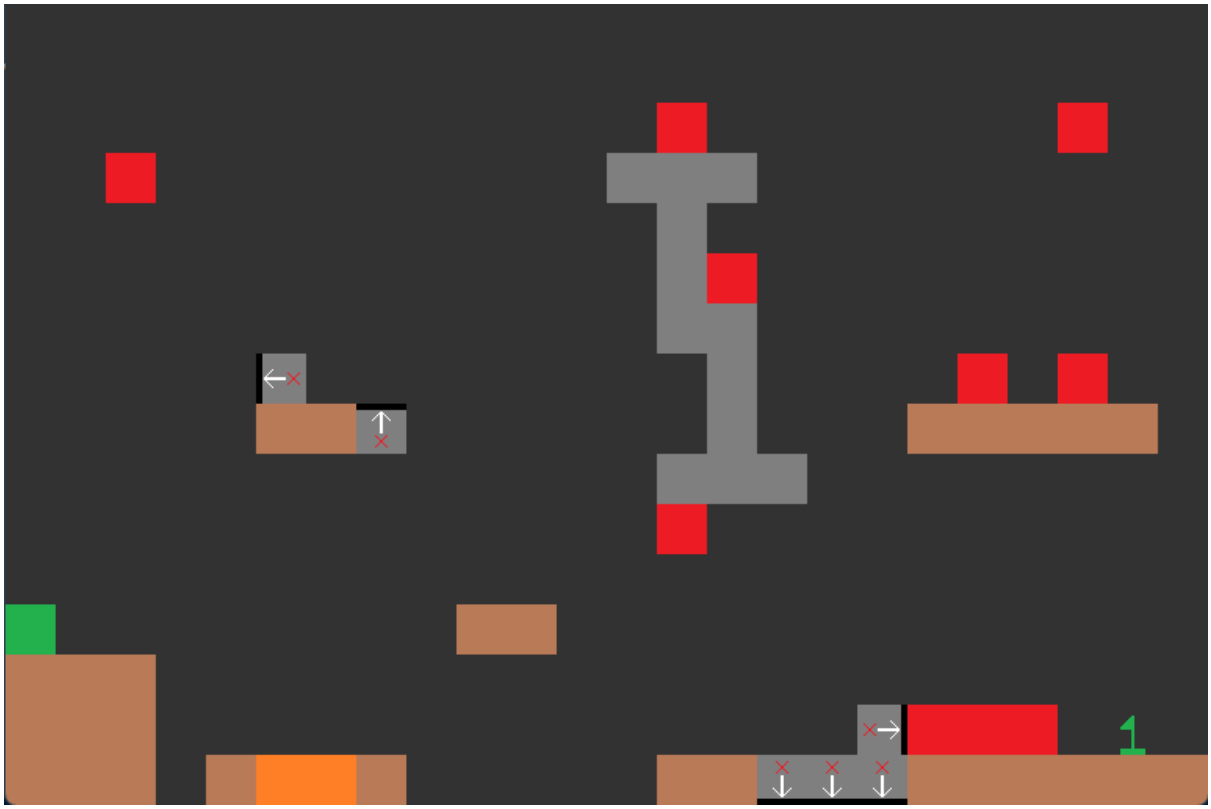
When a flyer enters the screen, they enter the "Pathing" state. In this state, the flyer performs pathfinding logic to find the shortest path from itself to the player.

As pathfinding takes a decent amount of time – especially with multiple flyer enemies – once the flyer finds a path, it transitions to the "Moving" state. In this state, the flyer moves to the next tile along its path, before transitioning back to the "Pathing" state. This allows the flyer to appear to be constantly targeting the player but only performing the time expensive pathfinding algorithm every couple frames.

1.8 Prototype Design



An example of what a level would look like.
Shaded tiles are floor tiles, hexagon filled tiles are honey, arrow tiles are one-way, X tiles are spikes, angry faces are enemies, smiley face represents the player's spawn, and the flag represents the end.



An implementation of the above level in a prototype engine

1.8.1 Client Feedback

I presented this design to John on April 12th, 2024. They had the following response:

"This is strong but has some major faults in my eyes. Firstly, I like the use of distinct colours, with the entities making themselves distinct from the duller floor and background colours. I dislike, however, that all the enemies are the same colour, as from a gameplay perspective it would be difficult from a glance to determine which enemy is which. Maybe use different shades of red for each type of enemy or update the sprite to indicate the unique types. A second flaw of the sprite's is in the one-way platforms. While it makes sense to use the same sprite but rotated, the base sprite is confusing. Using the three downwards one-ways as an example: I would initially believe that these would allow the player to walk over them, preventing movement downwards. I recognise that the border line at the bottom indicates that there is a blocking edge, but the sprite as a whole leaves a lot to be desired – I would suggest a simpler design with two thick arrowheads, pointing in the allowed direction. A final flaw with this prototype is in the layout of the level itself. The platforms appear randomly added, making the linear path appear busy with unnecessary challenges. I would like level's to effectively use the space available, with multiple paths to the end that make a run feel unique. Similarly, use the enemies to produce small moments in the level, rather than clumping them at the end flag."

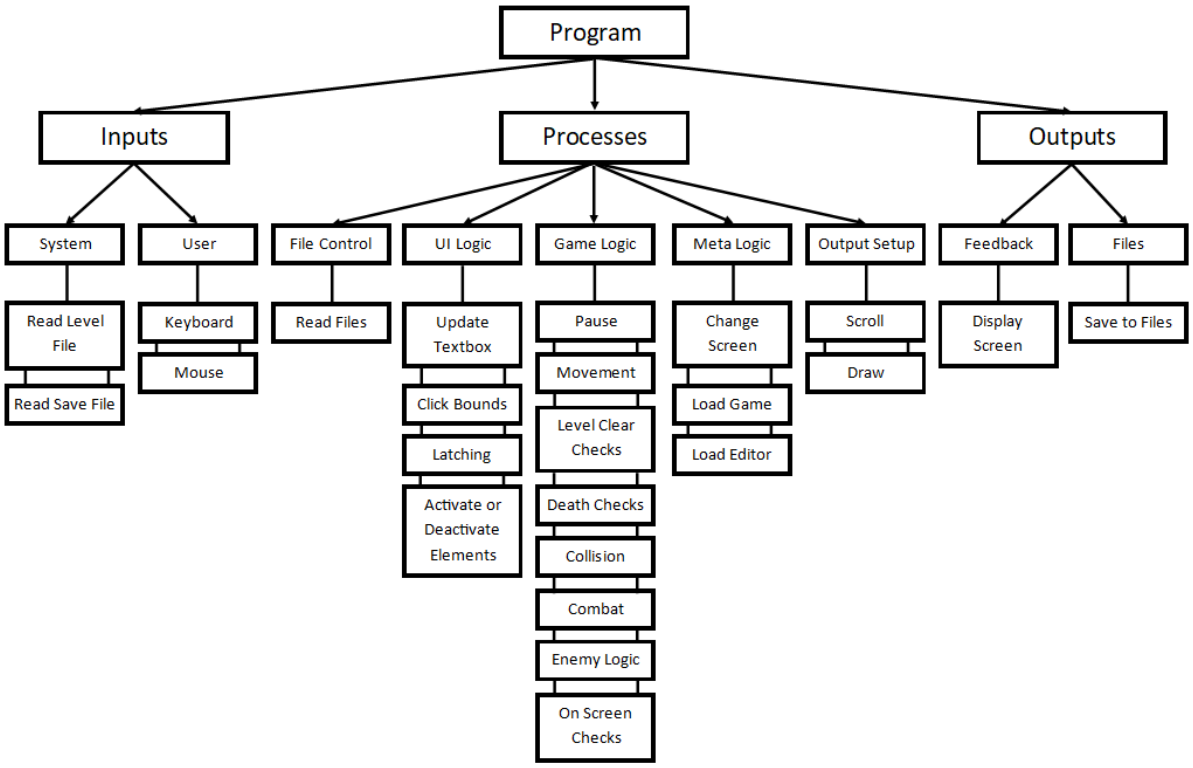
1.9 Languages I Will Use

I have decided to use Python to develop my game. Python is a multi-paradigm language (Imperative, Functional, and Object-Oriented). The object-oriented nature will make it easy to manage the multitude of objects in my game (such as the player, enemies, and tiles). By being a functional language, I can more easily organise my code into simpler functions, limiting the use of repeated code. Python also has a module system, allowing me to install and use the Pygame module to manage Input/Output elements of the game, such as monitoring keyboard strokes and displaying updated graphics. This module system also allows me to modularise my program, keeping the game logic separate from the menu logic, and the enemy logic separate from the tile logic. Through this, I can improve the readability of my code, allowing a more efficient development process.

2 Design

2.1 Design Overview

A top-down design approach will be used to modularise the program to create a successful result. By separating classes and segments of the main game loop into different Python scripts, I can more easily identify errors whilst programming and safely make changes without affecting the rest of the program. By separating into multiple scripts, I can more easily structure my code, as only the necessities for that specific script to run need to be present. Another benefit is through the ability to reuse code, saving space and allowing for cleaner code – instead of rewriting bubble sort each time I can just call example.bubble_sort() .
By using a main menu, I can further separate the level editor and main game to prevent unnecessary dependencies. I intend to use files to store levels – which will be loaded once when the level is played – as well as a file to store player data – just a list of values.



2.1.1 Inputs

Read Level File	File	When a level starts, the level data will be loaded from the relevant file
Read Save File	File	After loading the level, the relevant save data needs to be loaded
Keyboard	Key presses	Each frame, the program will check if the player is pressing any keys
Mouse	Left mouse clicks	Same as above, but for left mouse clicks

2.1.2 Processes

Read Files	Opens a file and reads the contents without editing them (side-effect free)
Update Textbox	The process of converting keyboard presses into a string for a Textbox element
Click Bounds	The process of checking if a click (mouse button down OR mouse button up) is fully in a button
Latching	A process to ensure the click starts and ends within the element (See 2.2.4)
Activate or Deactivate Elements	The process of including/excluding UI elements from being considered during a click event
Pause	The process of halting/continuing the game logic when the player pauses the game
Movement	Converting the user's keyboard inputs into player movements. Also manages the gravity logic
Level Clear Checks	Check whether the player has reached the level flag
Death Checks	The logic to check if the player is dead (no health left) or off screen – and thus perform the death logic
Collision	The logic to prevent the player from falling through a platform (See 2.2.2)
Combat	The logic to collect and resolve the attacks of the player and enemies (See 2.2.8)
Enemy Logic	The logic to update and use the state of each enemy's finite state machine for pathfinding and combat (See 2.2.7 and 2.2.9 to 2.2.14)
On Screen Checks	The process of minimizing the number of elements the program needs to draw to only those on screen (Mario Maker style) (See 2.2.5)
Change Screen	A process to change between game states, and display their respective UI elements
Load Game	The process of "activating" the game elements and logic processes
Load Editor	The process of "activating" the game elements and logic processes
Scroll	The process of moving the game instead of the player, to allow for more than one game screen
Draw	The process of drawing the relevant game elements to the screen object

2.1.3 Outputs

Display Screen	Display the image drawn by the game so the player can see what is happening
Save to Files	The program needs to save edited levels from the editor, as well as to update the save data file after the player beats a level or dies

2.2 Key Algorithms

2.2.1 glob.glob()

Glob is a premade library of functions for python. I will specifically be using glob.glob(), which takes a glob pattern and returns a list of all files which match that pattern.

This is effectively a regex for file exploration and will be used to find a list of custom levels, as the names are number of files present will vary through use of the level editor – and thus cannot be found statically.

2.2.2 Collision Detection

I intend to use pygame rects (rectangles) to represent the objects of the project. More specifically each object will have an associated rect for use in collision detection.

This system comes with the built in rect.colliderect() and rect.collidelist() functions which will check if a given rect intersects/collides with another rect.

Luckily, pygame is open source, with all the code available on GitHub, so the logic can be explored:

```
pg_rect_colliderect(pgRectObject *self, PyObject *const *args,
                    Py_ssize_t nargs)
{
    /* This function got changed to use the FASTCALL calling convention in
     * Python 3.7. This lets us exploit the fact that we don't have an
     * intermediate Tuple args object to extract all the arguments from, saving
     * us performance in the process. This decoupling forces us to deal with
     * all the different cases (dictated by the number of parameters) by
     * building specific code paths.
     * Given that this function accepts any Rect-like object, there are 3 main
     * cases to deal with:
     * - 1 parameter: a Rect-like object
     * - 2 parameters: two sequences that represent the position and dimensions
     * of the Rect
     * - 4 parameters: four numbers that represent the position and dimensions
     */

    SDL_Rect srect = self->r;
    SDL_Rect temp;

    if (nargs == 1) {
        /* One argument was passed in, so we assume it's a rectstyle object.
         * This could mean several of the following (all dealt by
         * pgRect_FromObject):
         * - (x, y, w, h)
         * - ((x, y), (w, h))
         * - Rect
         * - Object with "rect" attribute
         */
        SDL_Rect *tmp;
        if (!(tmp = pgRect_FromObject(args[0], &temp))) {
            if (PyErr_Occurred()) {
                return NULL;
            }
            else {
                return RAISE(PyExc_TypeError,
                             "Invalid rect, all 4 fields must be numeric");
            }
        }
        return PyBool_FromLong(_pg_do_rects_intersect(&srect, tmp));
    }
    else if (nargs == 2) {
        /* Two separate sequences were passed in:
         * - (x, y), (w, h)
         */
        if (!pg_TwoIntsFromObj(args[0], &temp.x, &temp.y) ||
            !pg_TwoIntsFromObj(args[1], &temp.w, &temp.h)) {
            if (PyErr_Occurred())
                return NULL;
            else
                return RAISE(PyExc_TypeError,
                             "Invalid rect, all 4 fields must be numeric");
        }
    }
    else if (nargs == 4) {
        /* Four separate arguments were passed in:
         * - x, y, w, h
         */
        if (!(pg_IntFromObj(args[0], &temp.x))) {
            return RAISE(PyExc_TypeError,
                         "Invalid x value for rect, must be numeric");
        }

        if (!(pg_IntFromObj(args[1], &temp.y))) {
            return RAISE(PyExc_TypeError,
                         "Invalid y value for rect, must be numeric");
        }

        if (!(pg_IntFromObj(args[2], &temp.w))) {
            return RAISE(PyExc_TypeError,
                         "Invalid w value for rect, must be numeric");
        }

        if (!(pg_IntFromObj(args[3], &temp.h))) {
            return RAISE(PyExc_TypeError,
                         "Invalid h value for rect, must be numeric");
        }
    }
    else {
        return RAISE(PyExc_ValueError,
                     "Incorrect arguments number, must be either 1, 2 or 4");
    }

    return PyBool_FromLong(_pg_do_rects_intersect(&srect, &temp));
}
```

This is the code for the colliderect() function, which is just validating if the given rects are valid. If they are, _pg_do_rects_intersect() is called with the two rects.

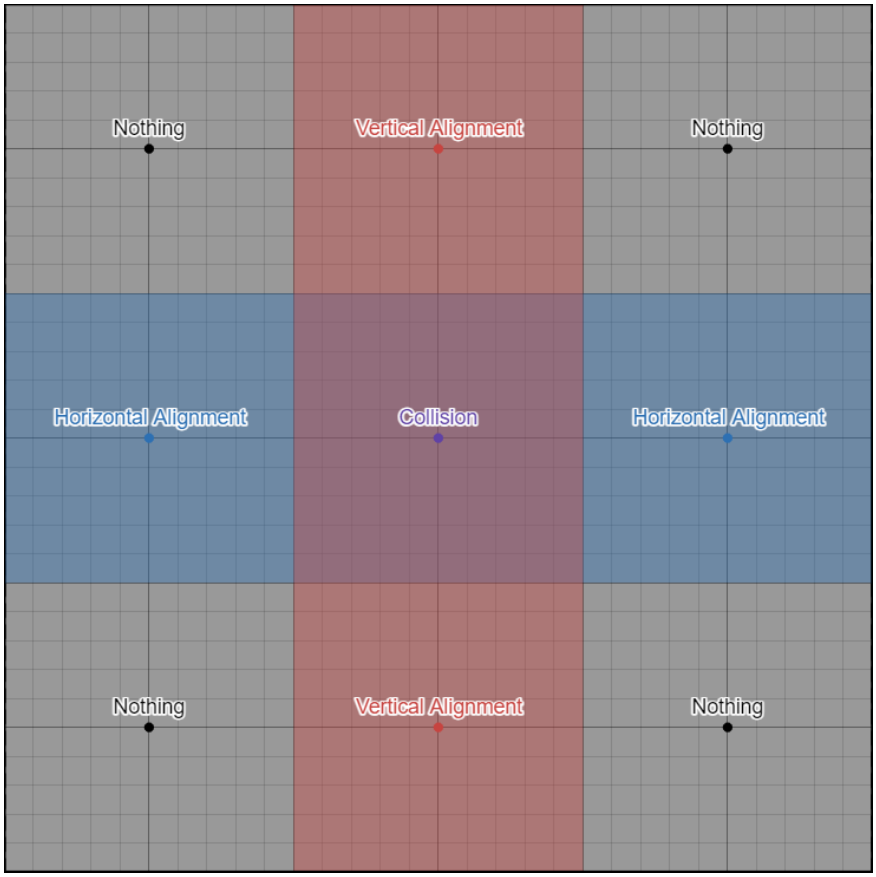
```
_pg_do_rects_intersect(SDL_Rect *A, SDL_Rect *B)
{
    if (A->w == 0 || A->h == 0 || B->w == 0 || B->h == 0) {
        // zero sized rects should not collide with anything #1197
        return 0;
    }

    // A.left   < B.right  &&
    // A.top    < B.bottom &&
    // A.right  > B.left   &&
    // A.bottom > B.top
    return (MIN(A->x, A->x + A->w) < MAX(B->x, B->x + B->w) &&
            MIN(A->y, A->y + A->h) < MAX(B->y, B->y + B->h) &&
            MAX(A->x, A->x + A->w) > MIN(B->x, B->x + B->w) &&
            MAX(A->y, A->y + A->h) > MIN(B->y, B->y + B->h));
}
```

This is the actual collision detection algorithm and uses the following logic.

First a final validation is carried out to check if any of the dimensions are zero, and if they are then the function fails (return 0)

Looking at the return clause, four comparisons are made and logically AND ed together.
The 4 clauses can be read as follows (from the comment preceding the return clause):
The left of A < the right of B
The top of A < the bottom of B
The right of A > the left of B
The bottom of A > the top of B



To confirm to us that this will work, the first and third conditions check if the two rects are vertically aligned, and the second and fourth conditions check if the two rects are horizontally aligned.

This is true as if the left of A is further right than the right of B, then A is strictly to the right of B (in this case condition 1 returns False), with the same statements holding for the left of B and the right of A, and the vertical conditions.

Looking at the diagram above, we see that a collision occurs if and only if the rects are both horizontally and vertically aligned, which is equivalent to the four conditions all being true, which is equivalent to the result of an AND between all four conditions. Thus, the professionally made algorithm does in fact work.

As a final note, the implementation of conditions 2 and 4 appear to read bottom of A < top of B, but pygame has the (0, 0) co-ordinate pair as the top left corner of the screen, so the smaller the y co-ordinate, the higher up the point is. As a result, Min(A -> y, A -> y + A -> h) returns the smaller y co-ordinate of A, and Max(B -> y, B -> y + B -> h) returns the larger y co-ordinate of B, so it tests the top of A < the bottom of B, which is what we want.

Code Sources:

https://github.com/pygame/pygame/blob/main/src_c/rect.c

At lines 664 and 356 for colliderect() and do_rects_intersect() respectively.

2.2.3 UI Click Detection

To integrate UI elements into the game, I will need a way to detect whether a click event occurs within a UI element.

To check if the click event is in bounds, I will use the same algorithm as above.

It is important to note that some UI elements exist as components of a container UI element – such as an entry to a list. As a result, these component elements will have positions which are relative to the container. This presents a problem: a click is relative to the screen, but a UI element is relative to a container. This requires a method to convert true co-ordinates to relative co-ordinates.

This is as simple as:

relative position = real position – container position

2.2.4 "Latching"

Even if a click happens in a UI element's bounds, a click event should only be considered if the click starts and ends inside the element. This is the "latching" logic, which will also be used for pausing, as the game should only pause and unpause when the pause key is up.

"Latching" is done through two variables, the latch variable, and a trigger variable. Below is the pseudocode for a "latch" update.

```
Method update_latch(trigger: bool):

        Global latch
        If(not trigger):

                Latch = True

        Else If(latch):

                Perform Action

                Latch = False
```

Value of trigger variable	Click Event	Pause Event
True	Mouse Button Up	Pause Key Up
False	Mouse Button Down	Pause Key Down

2.2.5 On Screen Entities

To limit the number of entities we consider each frame, we can instead start the frame by determining which entities are even on screen – more specifically, on screen + 4 tiles in case the player scrolls onto them.

```
Method On_Screen(object_to_check, screen):

        If object_to_check.left_side < screen.left_side

          or object_to_check.right_side > screen.right_side:

                Object_to_check.on_screen = False
```

```
    Else:
        Object_to_check.on_screen = True
```

Alongside making the game run faster through reducing the number of objects to draw, we no longer must perform pathfinding for enemies that are offscreen, which will stop enemies from reaching the player before the level would want them to be there, hopefully producing a better experience for the user.

2.2.6 Pythagorean Length

The project will use a co-ordinate system to determine where to draw each object. At points, we need to know the distance between two objects, such as for changing states of an entity's finite state machine.

This will be done using the Pythagorean theorem:

$$A^2 = B^2 + C^2$$

In this case, B and C represent the absolute difference between the x and y co-ordinates respectively of the two objects.

In Pseudo code:

```
Method Pythagoras(objectA, objectB):
    X_distance = absolute(objectA.x – objectB.x)
    Y_distance = absolute(objectA.y – objectB.y)
    B_squared = X_distance * X_distance
    C_squared = Y_distance *Y_distance
    Return root(B_squared + C_squared)
```

2.2.7 Enemy States

The project will contain three enemy types: Walkers, Flyers, and Crawlers.

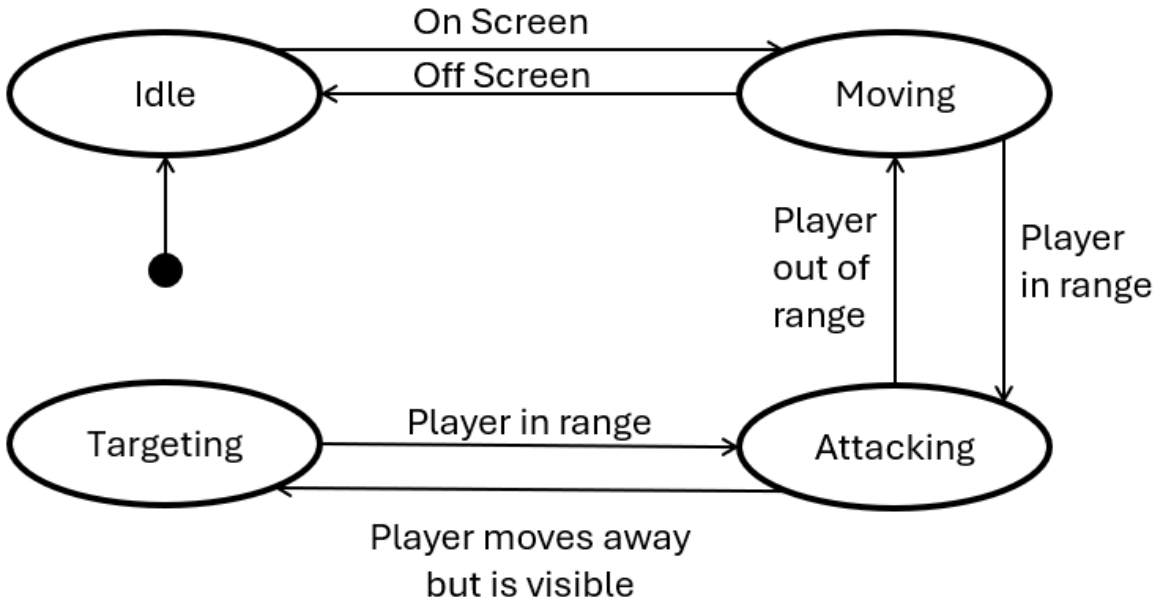
Crawlers will trace the perimeter of a platform, and only have one state, so will not be considered further.

Walkers and Flyers operate as their names imply, and each have three states which affect their behaviour.

These can be modelled through a simple finite state machine – one for each type

Walker FSM

The Walker has three states: Idle, Moving, and Attacking



Method Update_state(State, On_Screen, Distance)

    If (State == "idle" and On_Screen):

        State = "moving"

Else If(State == "moving" and not On_Screen):

        State = "idle"

    Else If(State == "moving" and Distance <= Attack_Range):

        State = "attacking"

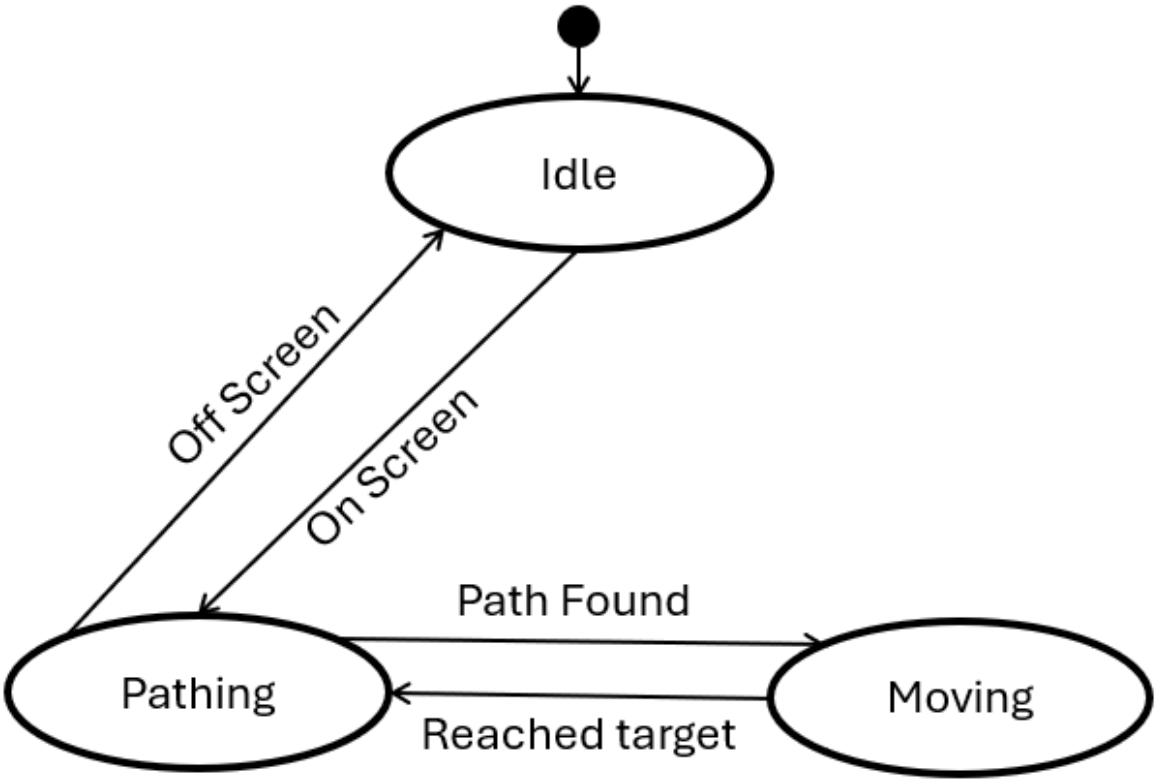
    Else If(State == "attacking" and Distance > Attack_Range):

    State = "moving"

In this algorithm, Distance refers to the Pythagorean distance between the player and the walker.

Flyer FSM

The Flyer has three states: "idle", "active", "moving"



Method Update_state(State, On_Screen, Move_Timer)

```
    If (State == "idle" and On_Screen):
        State = "active"
Else If(State == "active" and not On_Screen):
        State = "idle"
    Else If(State == "active" and Move_Timer > 0):
        State = "moving"
    Else If(State == "moving" and Move_Timer == 0):
    State = "active"
```

In this algorithm, Move_Timer is a variable which counts down the number of frames the flyer has been moving for. This allows a reduced use of an expensive pathfinding algorithm which (while fast) would inevitably slow the performance of the game if called too much.

2.2.8 Attack Logic

Attacks are broken into two stages: Collection and Resolution. In the main game loop, the game allows all entities to attack (if they choose to).

```
Method Attack(attacker, possible_targets, attack_hitbox):
    Connections = []
    For target in possible_targets:
        If target == attacker:
            Next
        If Collision_Detection(attack_hitbox, target):
            Connections.append([attacker, target])
    Return Connections
```

This method will take in the attacking object, all damageable entities that are on-screen, and the hitbox for the attack. It will then return all the targets which the attack collided with (whilst ignoring the attacker themselves). The returned list will then be added to a larger Collected_Attacks list, which is then used in the resolution phase. While Attack could be called by each entity each frame, Resolve will only be called once at the end of each frame before entities need to be drawn.

```
Method Resolve(Collected_Attacks, Damageable_Entities):
    For connection in Collected_Attacks:
        Attacker = connection[0]
        Target = connection[1]
        If Target.Protected:
            Next
        Target.Health -= Attacker.Attack_Damage
        Target.Protected = True
    For Entity in Damageable_Entities:
        Entity.Protected = False
        If Entity.Health <= 0:
            Delete Entity
```

2.2.9 Pathfinding

For ground-based enemies, they will just consider if the player is on their left or right and move accordingly. Other enemies will fly in the air, using top-down movement controls. For this, we can create a graph of the level (marking the enemy node and the player node) and use the A* algorithm to determine where to move.

A* is a shortest pathfinding algorithm for non-negatively weighted graphs. A graph for our level will just be all empty tiles (the nodes), which have edges to all adjacent (including diagonally) empty tile. The weight for each edge is 1 when orthogonally adjacent, or $\sqrt{2}$ when diagonally adjacent (I will use 10 and 14 as weights for orthogonal and diagonal edges respectively, as $\sqrt{2} \approx 1.4$, so multiplying both by 10 gives integer values and prevents obscure bugs which can occur with decimals in python). Each node will be labelled with their tile indices (calculated using top_left // 32). The source node's label will be the enemy position // 32, and the destination node's label will be player position // 32. This graph is constructed as the level is loaded.

```
Method A* _Pathfinding(source, destination, graph):
    Closed_list =   2D array of Booleans
    Nodes = 2D array of Node objects

    Set source node data to 0, and parent = source

    Open_List = MinHeap(Source Node)
    While Open_List is not empty:
        Closest_Node = Open_List.pop()
        Flag  Closest_Node as visited in Closed_list

        If Closest_Node is destination:
            Return path
        For each neighbouring tile:
            Check if it exists and can be reached
            Check if it has already been visited (True in the closed_list)
            Calculate new g and h values
            F = g + h
            If Nodes[neighbour].f > F:
                Update neighbour's f, g, and h
                Set neighbour's parent to closest_node
    If Open_List is empty:
        Return "FAIL"
```

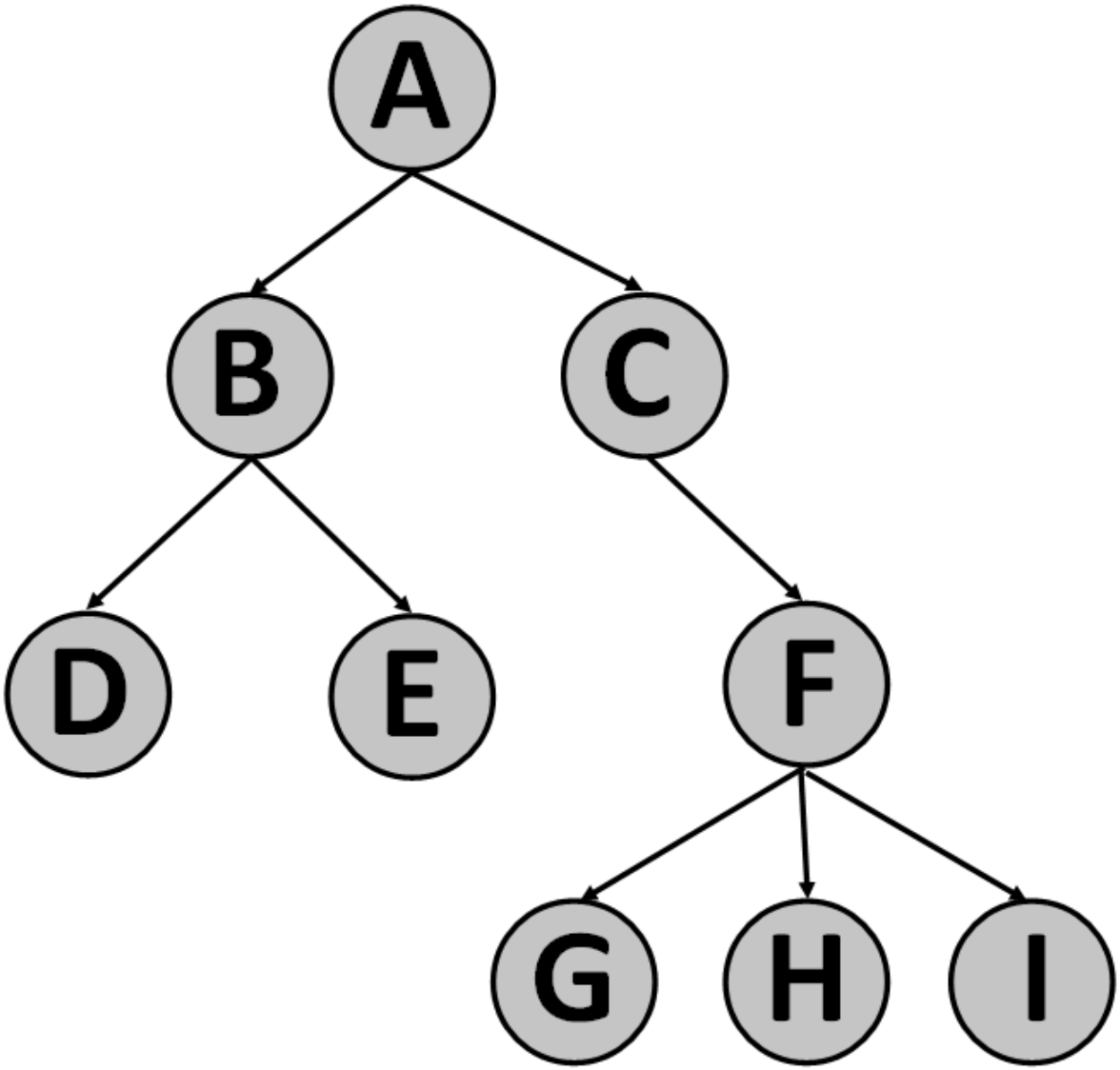
This uses 3 data structures and 2 auxiliary method. This needs the Pythagoras function as described in (2.2.6), as well as a trace_path function, which will be discussed in 2.2.10. The first

data structure is the node. A node is a class which bundles the information of a tile together, including the previous (parent) node and the current f, g, and h values. The second data structure is a tree, which is a special graph where there are no cycles – discussed in the data structures section. The final data structure is a Min-Heap, which (in brief) keeps track of the smallest element – this will also be discussed in the data structures section.

We store the tree within the nodes to allow us to quickly trace the final path. We store the open_list as a min_heap so we can always access the "closest" node quickly.

2.2.10 Trace Path Tree

A tree is a graph in which there are no cycles (you can't get from node A through any of its child nodes). This is used in 2.2.9 to store the shortest path. As a tree is an abstract data structure, we don't need a dedicated section of memory to store the graph, and instead we can just store references to the parent node within the node itself.



This is an example of a tree, where A is the root node. The trace_path algorithm starts from a leaf node (say E) and follows parent pointers up to A, in this case E -> B -> A. By storing the graph

using parent pointers, we can just start from the leaf node (the destination of the search) and use a while loop to reach the root (source) node.

```
Method Trace_path(destination):
    Node = Graph[destination]
    Path = []
    While Node.parent is not Node:
        Path.append(Node)
        Node = Node.parent
    Return Path
```

2.2.11 Min-Heap Invariant Management

A Min-Heap is an array in which A[k] <= A[2k + 1] and A[k] <= A[2k + 2] for all k. This is useful as the smallest element is always at the first entry, so it is suitable for use as the open list in the for the a-star algorithm in 2.2.9. This relationship happens to describe a balanced binary tree, where A[0] is the root, and for A[k], A[2k + 1] and A[2k + 2] are its children. This gives us some terminology to help describe any algorithms.

The challenge comes in to preserve the minimum invariant. We need two key algorithms, one to insert a value into the heap, and one to pop the smallest value – both of which must preserve the minimum invariant.

For the insertion method, we append the value to the end of the heap. We can find that node's parent using parent_pos = (my_pos – 1) // 2. We can then compare this node with the parent node and swap them if the new node is smaller. We repeat, using parent_pos as my_pos until the parent is smaller than the child, or the new node is now the root.

```
Method Bubble_down(position):
    Parent = (position – 1) // 2
    While Heap[Parent] > Heap[position]  AND Parent > 0:
        Swap Heap[Parent] and Heap[position]
        Parent = (Parent – 1) // 2
```

Insertion is just appending followed by bubble_down(length(heap) – 1).

Popping is slightly harder. We always return the first entry (hence popping over deletion), and then delete that first entry. But now the root has been removed and we have two trees. So how do we recombine these trees whilst keeping the heap balanced? Instead of deleting the first entry immediately, we can instead overwrite it with the last entry, then bubble that entry down.

```
Method Bubble_up(position):
    Child = 2 * position + 1
    While Child < len(Heap):
        RightChild = Child + 1
```

```
If(RightChild < len(Heap) NAND Heap[Child] < Heap[RightChild]):
        Child = RightChild
Heap[position] = Heap[Child]
Position = Child
Child = 2 * position + 1
```

This method starts at the root, then loops through its children, swapping with the smaller of the two. To pop, we store the first entry, replace the first entry with the last entry, then perform Bubble_up. We can then safely delete the last entry and return the stored first entry.

Python has a built-in heapq module, which I will be using to implement the heap. Like 2.2.2, the code is open source, and viewable on GitHub.

```
def heappush(heap, item):
    """Push item onto heap, maintaining the heap invariant."""
    heap.append(item)
    _siftdown(heap, 0, len(heap)-1)
```

```
def _siftdown(heap, startpos, pos):
    newitem = heap[pos]
    # Follow the path to the root, m
    # newitem fits.
    while pos > startpos:
        parentpos = (pos - 1) >> 1
        parent = heap[parentpos]
        if newitem < parent:
            heap[pos] = parent
            pos = parentpos
            continue
        break
    heap[pos] = newitem
```

This is their implementation of Insert, with _siftdown() being the implementation of Bubble_down().

```
def heappop(heap):
    """Pop the smallest item off the heap, maintaining the heap invariant."""
    lastelt = heap.pop()    # raises appropriate IndexError if heap is empty
    if heap:
        returnitem = heap[0]
        heap[0] = lastelt
        _siftup(heap, 0)
        return returnitem
    return lastelt
```

```
def _siftup(heap, pos):
    endpos = len(heap)
    startpos = pos
    newitem = heap[pos]
    # Bubble up the smaller child until hitting a leaf.
    childpos = 2*pos + 1    # leftmost child position
    while childpos < endpos:
        # Set childpos to index of smaller child.
        rightpos = childpos + 1
        if rightpos < endpos and not heap[childpos] < heap[rightpos]:
            childpos = rightpos
        # Move the smaller child up.
        heap[pos] = heap[childpos]
        pos = childpos
        childpos = 2*pos + 1
    # The leaf at pos is empty now.  Put newitem there, and bubble it up
    # to its final resting place (by sifting its parents down).
    heap[pos] = newitem
    _siftdown(heap, startpos, pos)
```

This is their implementation of pop, with _siftup() being the implementation of Bubble_up(). The only difference is that they bubble an empty entry and fill it in with the "last" entry at the end, whereas the pseudocode swaps the first and last entries before the bubbling.

Code Sources:

https://github.com/python/cpython/blob/main/Lib/heapq.py

Heappush(), _siftdown(), heappop(), and _siftup() can be found at lines 132, 207, 137, and 260 respectively.

2.2.12 Shudder (Crawler Movement)

A crawler is an enemy which moves anticlockwise around the perimeter of a platform, no matter how weird the shape is.

This is done by moving in a straight line until you hit a wall (rotate clockwise) or reach a corner (rotate anticlockwise). The important part of the algorithm is how we check for these conditions, this being the Shudder algorithm.

Method Shudder(direction):

Move 1 unit in direction

Check for Collision

Move back 1 unit

Return if a collision occurred

We need to also introduce the concept of the sticking point, which is the direction in which the crawler sticks to the platform. This is always 90 degrees anticlockwise from the direction of movement.

To perform a movement, we can now just do the following:

```
Method Movement():
    Vector = Convert_to_vector(direction)
    Move by vector
    Wall_Check = Shudder(direction)
    Stick_Check = Shudder(Sticking_Point)
    If(Wall_Check):
        Rotate clockwise
    Else If(Stick_Check):
        Rotate anticlockwise
```

Convert_to_vector is a method with no logic, just takes in a direction and returns a vector as a list.

2.2.13 Jiggle (Walker Movement)

All the tiles will be stored as a grid. Each tile will have coordinates which are multiples of 32 (as each tile is a square of lengths 32 by 32). The problem arises when an object does not walk a "clean" number of pixels in a frame.

Suppose the player moves 1 to 8 pixels each frame (1 on frame 1, 2 on frame 2, etc). In this case, after frame 7 we'll have moved 28 pixels, but on frame 8 we'll have moved 36 pixels. If there is a wall 32 pixels away from where we start then we'd have a problem, as frame 7 we'd be 4 pixels away, but frame 8 we'll be 4 pixels in the wall, which can't happen. One solution is to run a collision check, then move back 4 tiles (in this case) to get out of the wall. But what happens if we moved 32 pixels in a frame? In this case, we would have passed through the wall, which is behaviour we don't want.

Another solution (which I will use) is to incrementally move (jiggle) the object until it either completes its move or collides, in which case we end the movement early.

This is as simple as a loop, shown in the pseudo code below:

```
Method Jiggle(To_Move):
    Moved = 0
    While Moved < To_Move
        Move 1 pixel
        If collided:
            Move back 1 pixel
```

Return

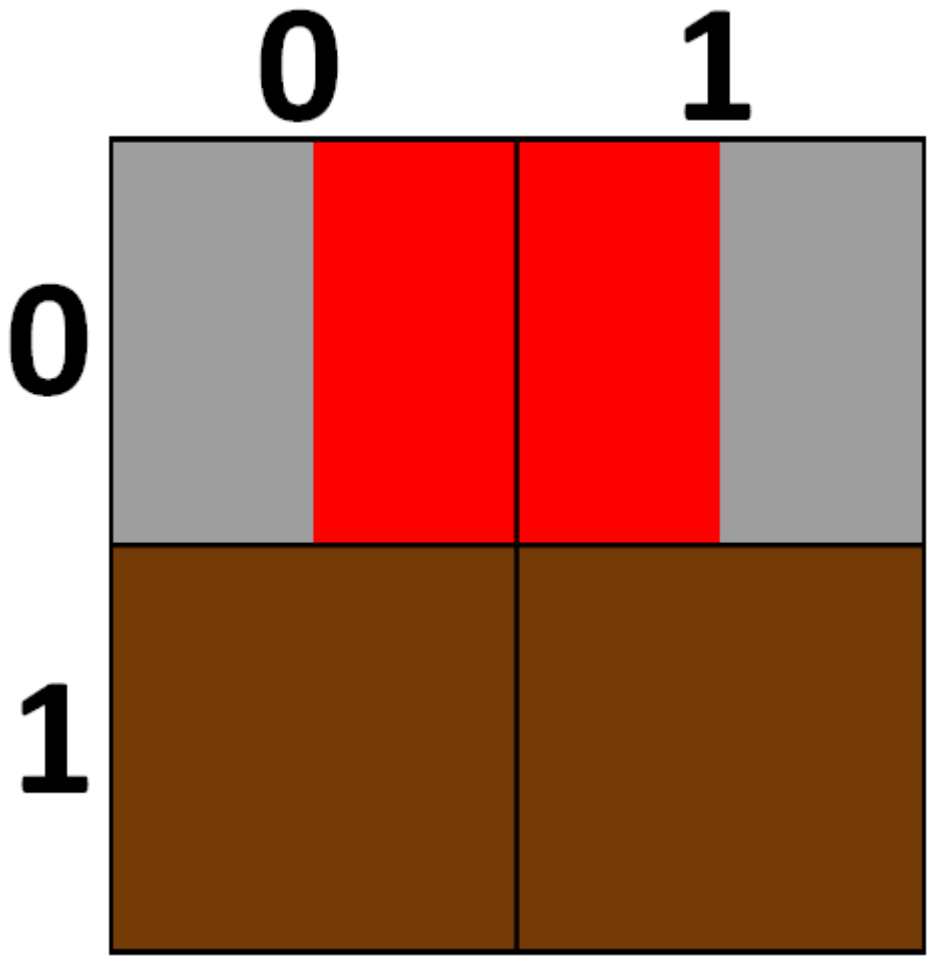
Moved += 1

2.2.14 Walker Vision

As with the crawler's shudder, a walker enemy doesn't want to fall off the edge of a platform, and definitely doesn't want to permanently walk into a wall. As a result, we need an algorithm to determine whether the walker should turn around – almost like vision.

This algorithm will use a 2-dimensional list of values, with each entry representing which tile is in the respective position – effectively the output of the level's csv file.

For both conditions, we need to know where the walker currently is. This can be done by just taking the objects pygame position and dividing by 32 (the number of pixels per tile).



But what about here? Should the walker be in (0,0) or (1,0)?

This matters as if we choose the wrong index, then the walker will turn around a whole tile in advance. Luckily, this can be solved by first checking which direction the walker is moving, and then take the opposite corner. So, if we're moving left, we take the right corner, and if we're moving right, we take the left corner. As the pygame position is the top left corner, we can just add 1 to the x co-ordinate when moving left to get the right corner

```
Method Get_Indices(position):
    X = position[0] // 32
    Y = position[1] // 32
    If (Moving_Left):
        X += 1
    Return X, Y
```

We can now test the two conditions.

For the first condition, look at what tile is below the tile we want to move to, and if it is unwalkable, then we turn around.

```
Method Check_Floor(X, Y):
    If(Moving_Left):
        Tile = Grid[Y + 1][X - 1]
    Else:
        Tile = Grid[Y + 1][X + 1]
    Return Tile is walkable
```

For the second condition, look at the tile we want to move to, and if it is solid, then we turn around.

```
Method Check_Wall(X, Y):
    If(Moving_Left):
        Tile = Grid[Y][X - 1]
    Else:
        Tile = Grid[Y][X + 1]
    Return Tile is not solid
```

We also need to check if the player is visible, as if they are then we don't want to turn around.

This can be done by first checking if the player is even on the same level as the walker (the y indices match), and then looping through the row of tiles to see if the player is present.

```
Method Player_Check(X, Y, Player):
    Player_X, Player_Y = Get_Indices(Player.Position)
    If (Player_Y != Y):
        Return False
    If (Player_X == X):
        Return True
```

```
        Tile = grid[y][x]
        While Tile is not solid AND X > 0 AND X < Edge of screen:
            X += 1
            Tile = grid[y][x]
            If (Player_X == X):
                Return True
        Return False
```

(In the case of moving left, we decrement X instead)

The final algorithm is then just a combination of these rules:

```
Method Vision():
        X, Y = Get_Indices(Position)
        Floor_Validation = Check_Floor(X, Y)
        Wall_Validation = Check_Wall(X, Y)
        Player_Visible = Player_Check(X, Y, Player)

        If (Floor_Validation NAND Wall_Validation):
            If (Player_Visible):
                Do Nothing
            Else:
                Turn around
        Else:
            Move
```

2.2.15 Kinematics (Walker Acceleration)

To make the movement of the player feel smoother, we need to add acceleration. This requires a part of mechanics called kinematics, which boils down to the relationship between time, velocity, and acceleration.

Acceleration will be used when the player falls – accelerating downwards – or when the player runs – accelerating in that direction.

These systems will use the same equation:

$$V = U + A * T$$

Where V is the final velocity, U is the initial velocity, A is the acceleration, and T is the time spent accelerating.

We can simplify this equation, as T will always be 1 (as we move every frame), so:

$$V = U + A$$

2.2.16 Mutation

To make each enemy feel different, I will use a simple algorithm to randomise an enemy's health, damage, attack speed, and move speed upon instantiation.

This algorithm is as simple as generating a random number from a to b for each stat which will be that enemy's value for that stat. More specifically, the random number will be a weight from 0 to m, which will then be used to determine which mutation occurs (higher roll = stronger enemy).

Method Mutation():

    Attack_roll = random number from 0 to 7

    If Attack_roll = 0 or 1:

        Damage = normal, attack_speed = slow

    If Attack_roll = 2 or 3:

        Damage = normal, attack_speed = normal

    If Attack_roll = 4:

        Damage = good, attack_speed = normal

    If Attack_roll = 5:

        Damage = normal, attack_speed = fast

    Else:

        Damage = good, attack_speed = fast

This is how the attack stats will be randomised for a walker, and the same logic is used for all other stats. The specific values and weights of each stat are different for each enemy type, to account for their differing movement and attack styles (for example, the crawler attacks every frame, so it can't attack faster).

2.2.17 Enumeration

This is a built-in function in python, which simply takes in an iterable (such as a list), and returns each element with a position – enumerating the values.

This is not a complicated algorithm, but necessary for the implementation of many algorithms, as well as to find the index of a value in a list which isn't apparent. For example, if you had a list of objects, you wouldn't inherently know the index of a pencil just from the fact it is a pencil. This idea extends to a list of game objects, which can't easily be kept track of.

2.2.18 Garbage Collection

Python automatically allocates and deallocates memory. The bulk of the system is simple, you create an object, python runs an allocation request, so you don't have to in your code, and when you delete an object, python releases that memory, allowing other software to use it. This

system is simply to make Python more accessible, stopping learners from unintentionally using too much RAM and creating massive memory leaks through forgetting to release the memory.

What matters here is a part of the deallocation process called "garbage collection".

In short, python's garbage collector is an algorithm which checks whether an object is being referenced – potentially through other objects – and if it isn't, the object can be removed. After all, a programmer wouldn't have access to the object without a reference. On the most part, this is a useful addition but has some side-effects which affects how this project will need to manage objects.

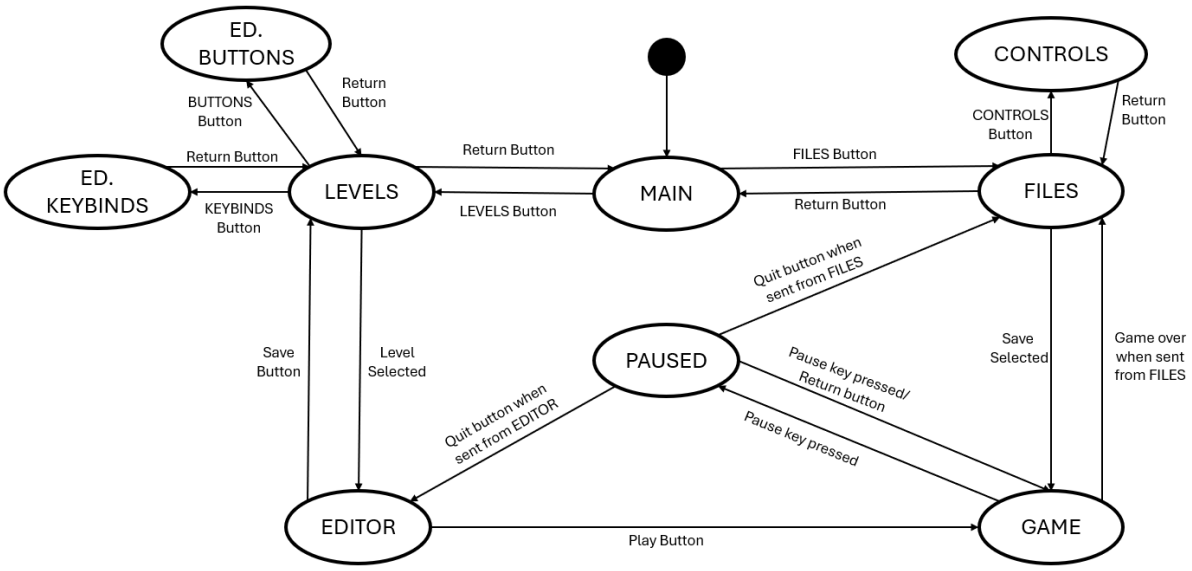
To explain the problem, consider an enemy object. A reference to this object will be stored in a list so the project can have a dynamic set of enemies. If the enemy is defeated, then we just delete the object, and all is good. But because of garbage collection, the reference to the object in the enemy list is preventing the object from being deallocated, and so the enemy still exists within the code, and will continue to exist in the game – an incorrect behaviour.

So, in some cases, just deleting the object is not enough, and instead I will need to run my own garbage collection to gather which objects need destroying, and then remove their references one by one.

2.3 Game States

Screen	Description
MAIN	This is the screen that first loads when the program is run, and provides the option to either play levels, or use the editor (via two buttons, one for each option)
FILES	This screen loads when the player chooses to play a level, and allows the user to select which file they want to play on, as well as an option to return to MAIN
DELETE	This screen loads when the player chooses to delete a save file and allows the user to select which file they want to delete, as well as an option to return to FILES. To prevent files from accidentally being deleted, a captcha will be required to actually delete the file.
GAME	This screen loads when the player selects a file, and is where the game itself is played
PAUSED	This screen loads when the player presses the pause key whilst GAME is loaded. This will halt the logic occurring in the game, and will allow the player to either continue (unloading PAUSED and continuing the game logic) or to exit (which will save the game's information and load MAIN)
LEVEL SELECT	This screen loads when the player chooses to use the editor on MAIN, and is used to list all the levels that the player can edit, an option to create a new level, and an option to return to MAIN
EDITOR	This screen loads when the player chooses a level to edit, and consists of a grid to place tiles on, and a tile palette to select how the player interacts with the grid
TOOL_TIPS	This is actually 3 screens with the same purpose: display the controls of a certain game state. The tool tips in question are: Controls, Editor_Buttons, and Editor_Keybinds, which display the game controls, the editor buttons and the editor key binds respectively. The Controls screen has a button to return to FILES, and the other two have a button to return to LEVEL SELECT

Only one screen will be loaded at a time, with the exception of PAUSED, which may be loaded over GAME.



This is a state transition diagram showing how a user can traverse game states.

2.4 Data Structures

Iterables

2.4.1 to 2.4.3 are iterable variables. These are variables which can be iterated through, through an identifying key or index.

2.4.1 List

This is just (as the name would imply) a list of variables which share the same name but are differentiated by indices. This structure is automatically provided by Python and is defined as my_list = [5,12,4,7]. The first value is selected using my_list[0] as Python uses 0-based indexing (ordering things by 1, 2, 3, etc is 1-based). A list can be used to keep variables "in one place".

Index	0	1	2	3	4	5	6	7
Value	14	7	"u"	89	"e"	-	0.167	"five"

An example of a list, using 0-based indexing - starting at 0. The value at index 5 is blank, as it is yet to be assigned.

This will be used in pretty much every aspect of the program, but one example is the enemy list, as each level can have variable numbers of enemies so defining individual variables is neither plausible nor practical.

List Comprehension

List comprehension is a concept in functional programming in which you can define a (potentially complicated) list through a set of rules, rather than defining each element individually.

One application is to convert a predefined list of integers into their string counterparts, reducing the complexity of the code. Another is to initialise a list of variable length (such as the level grid).

List Concatenation

In python, two lists can be combined into one through the + operator. This allows us to (for example) define multiple "component" lists separately with individually clear names, which can then be combined into one list with a generic name.

2.4.2 Tuple

A tuple is identical to a list, but its values cannot be modified once assigned – this property is also called "immutability". Tuples are useful for when a function is to return multiple values that just need to be read.

List comprehension nor list concatenation are functions of tuples, and so some cases where a tuple should be used, we use a list instead. If neither function is needed, we can use the following rule: use a list when elements are being removed or replaced, else use a tuple to prevent accidentally modifying the data.

2.4.3 Dictionary

A dictionary (or associative array) is in effect a more general list, where instead of an integer index pointing to a value (index-value pair), a dictionary uses an integer or a string as a key which would then similarly point to a value (key-value pair). Like lists, keys are atomic, so one key will always point to exactly one value, and this structure is similarly provided by Python, and can be defined using my_dict = {key: value, key: value, key: value}, or my_dict = dict([key: value, key: value, key: value]). Dictionaries are useful when collating multiple values which can only be distinguished through non-integer values.

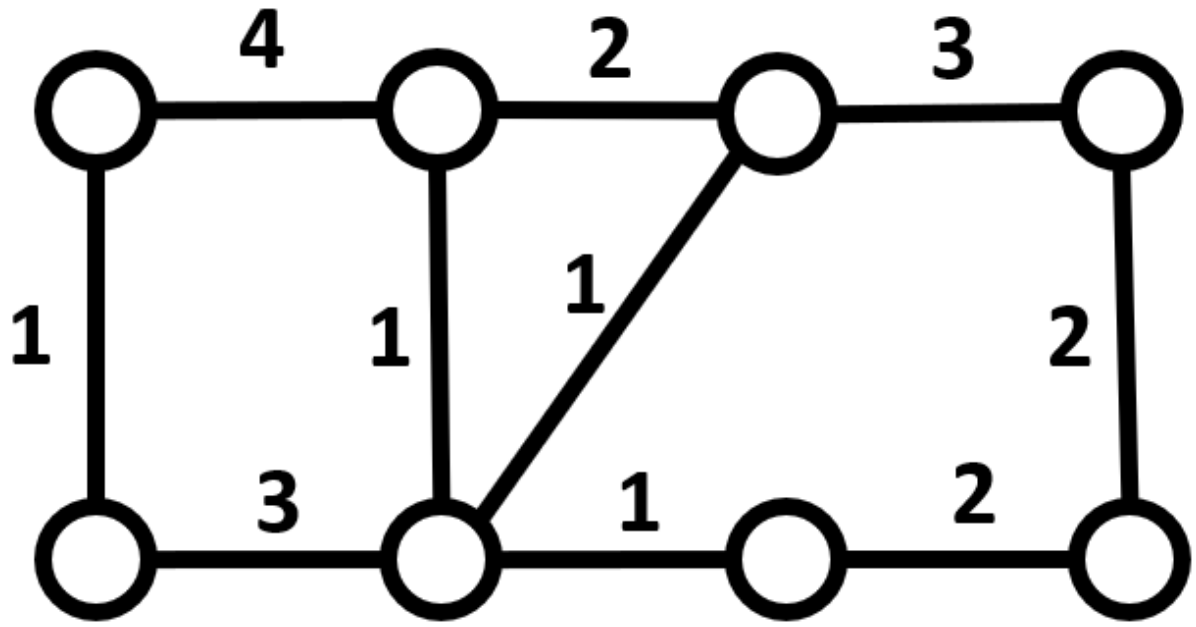
Key	"half"	"one"	"e"	"zero"	"pi"
Value	0.5	1	2.1828	0	3.1415

An example of a dictionary, being used to store mathematical constants.

Keys need to be "simple" variables: strings, integers or tuples, but can't be abstract data types (discussed below).

2.4.4 Graph

A graph is a data structure based on the mathematical object of the same name. A graph consists of nodes – which are objects which store the data – which we "connect" using an edge – a pointer. An edge in a graph represents some kind of relationship between two nodes, which is constant across the whole graph. Edges can be directed, meaning that node A connects to node B, but node B does not connect to node A. Edges can also have weights, which indicate the extent of the connection. For a computer to store a graph, the nodes are stored as any object would be. Edges are much harder to store but can be stored in 2 ways: adjacency lists and adjacency matrices. In an adjacency list, each node stores their outgoing edges as elements of a linked list, and then all the linked lists are stored in one larger list – the adjacency list. Alternatively, you can use an adjacency matrix, which is a 2-dimensional array (or in python a list of lists) which uses the start and end node as the x and y co-ordinate respectively and stores the weight of the edge connecting them (0 if no edge is present, 1 if an edge is present and the graph is unweighted). Lists are used when the graph is sparse (nodes don't have many edges).



An example of a weighted graph

	A	B	C	D	E	F	G	H
A		4	-	-	1	-	-	-
B	4		2	-	-	1	-	-
C	-	2		3	-	1	-	-
D	-	-	3		-	-	-	2
E	1	-	-	-		3	-	-
F	-	1	1	-	3		1	-
G	-	-	-	-	-	1		2
H	-	-	-	2	-	-	2	

An example adjacency matrix for the above graph – stored using a 2D list

Node	A	B	C	D	E	F	G	H
Edges	{B:4, E:1}	{A:4, C:2, F:1}	{B:2, D:3, F:1}	{C:3, H:2}	{A:1, F:3}	{B:1, C:1, E:3, G:1}	{F:1, H:2}	{D:2, G:2}

An example adjacency list for the above graph – stored using a list of dictionaries

In this example, the number of edges (20 as each edge is counted twice due to be undirected) is much less than the maximum number of edges (56) and is this a sparse graph. As a result, it would be better to use an adjacency list to store the graph, rather than an adjacency matrix.

I will be using a graph to store the adjacencies between tiles in the tile_grid, which is necessary to perform the A* algorithm (as pathfinding entities need to know where things are). Each node will have up to 8 outgoing edges, which will result in the final graph being sparse. As such, an adjacency list* will be used to store edges.

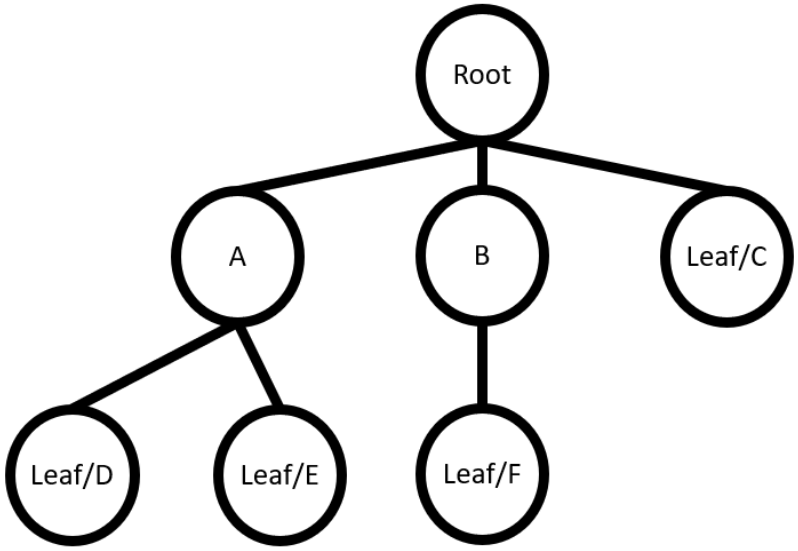
The Node "data structure"

A node is a class which is exclusively used to store data. In this case, it is used to store the position of the tile it's representing, and may also store other variables, such as a pointer to its

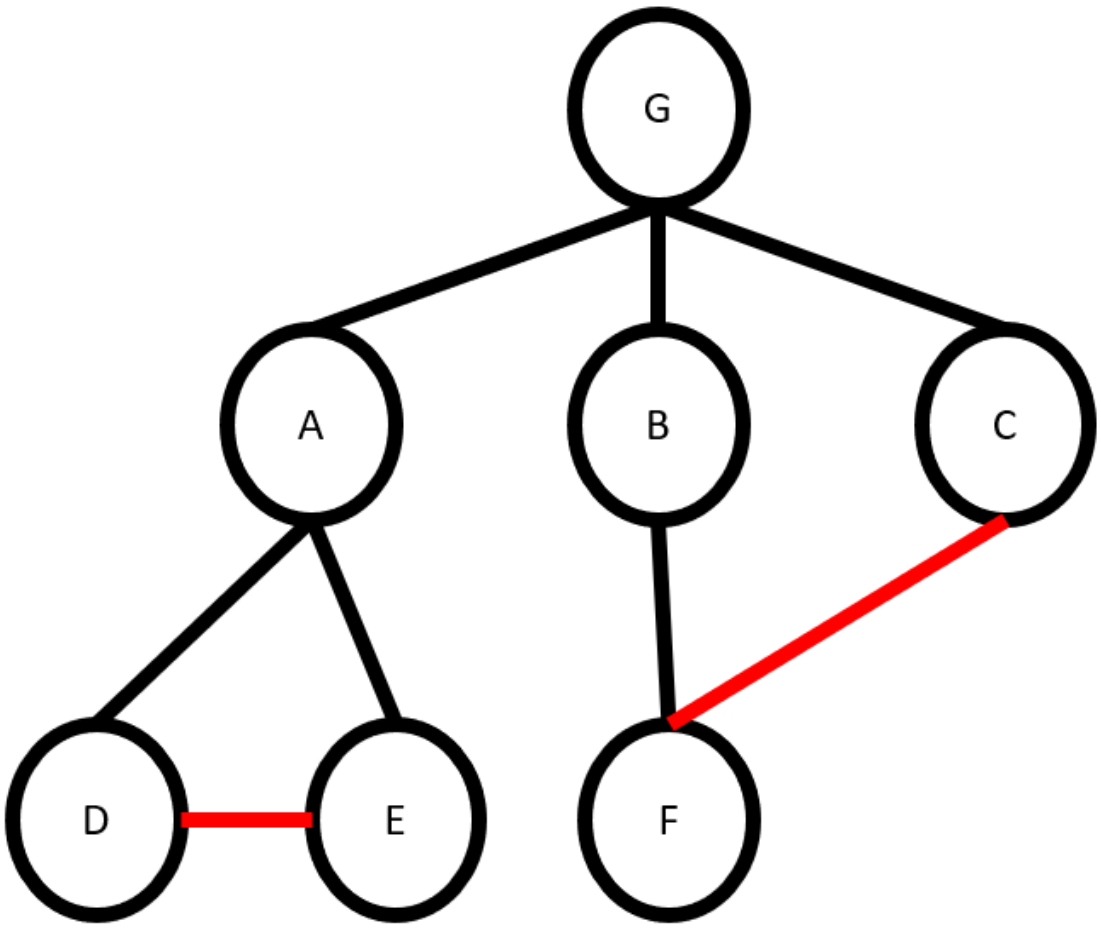
parent, or data needed by a graph function, like the heuristic value of the node for the a-star algorithm.

2.4.5 Tree

A tree is a variant of a graph, in which each node has at most 1 incoming edge (called a parent), and a singular node having no parents (called a root). If a node is not a parent to another node (it has no children) it is called a leaf. Trees are stored identically to graphs. Trees are used to identify a relationship between each node and the root. One application in my program is storing the shortest paths from an enemy's position (the root node) to the player's position.



An example of a tree



An example of a graph which is not a tree due to the cycles. By removing the red lines, a tree would be present

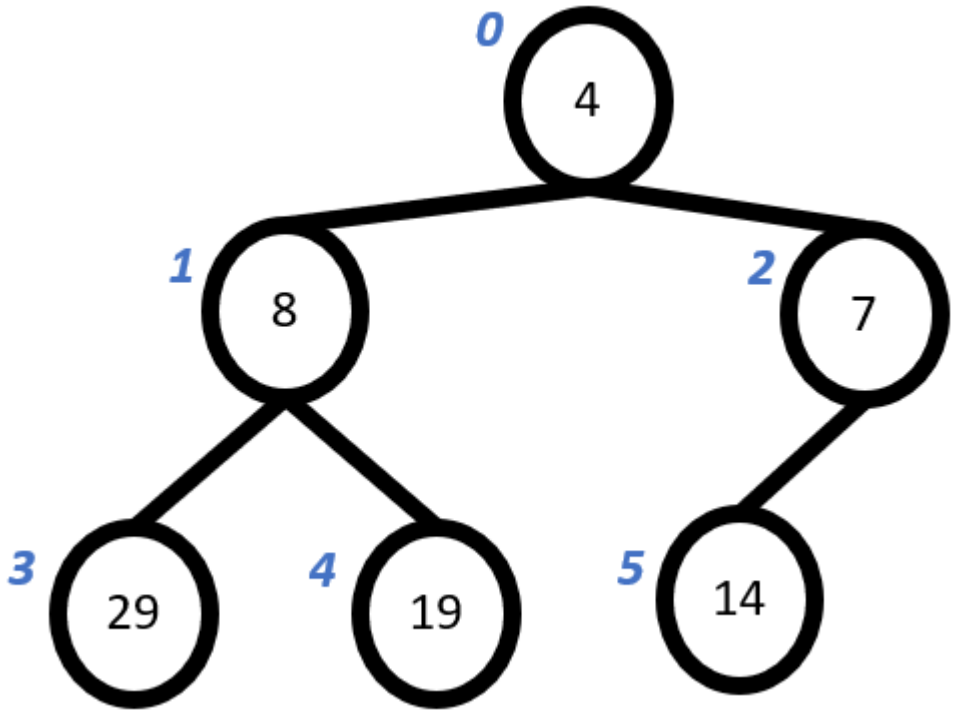
I will be using a tree to store the current path found during the A* algorithm – using the start node as the root. As each step of the path will lead to a brand-new node, no cycles can form. Also, by using a tree, the shortest path can be quickly reconstructed by going from the destination node (which will be a leaf) and jumping up parents until the root node is reached.

This can be stored through pointers in the node class.

2.4.6 Binary Heap (Complete Binary Tree)

Whilst trees use parent-child relationships to establish a connection, in a heap there is no importance to which elements are parents and which are children. Instead, a binary heap uses parent-child relationships to maintain a balanced characteristic. As such, the heap can be implemented as an array with the 0^{th} element as the root, and the following rules for the ith node: the parent of the ith node is at position (i-1)//2, and its two children are at (2*i)+1 and (2*i)+2 respectively. When elements are added to a heap, it is identical to appending to the array. Now, when an element is removed from the heap, we take one of its children (typically the right first) to become the new parent, and we repeat until we reach a node with no children. This is called balancing and is used to ensure consistent lookup times when searching from a parent/child node.

A binary heap comes with two main variants: max and min-heap. These are near identical to binary heaps but impose certain restrictions on each node – called the heap invariant. In a min heap, the parent node MUST be less than or equal to both of its child nodes. If not, then we perform swaps to ensure the invariant holds. The inverse rule is applied for a max heap. Due to the balanced nature of the heap, when we add/remove an element, it only takes $O(\log_2(n))$ time to return the invariant, as opposed to the O(n) time required to minify/maxify a list. Importantly, this structure will have the max/min as the first element.



An example tree-based implementation of a min-heap for the input (19, 14, 8, 29, 7, 4)

Index	0	1	2	3	4	5
Value	4	8	7	29	19	14

An example list-based implementation of the min-heap above, where the index corresponds to the blue number next to a value

From these examples, you can check to find that the parent-child index relationship holds.

I will be using Min-Heaps to implement a priority queue, as mentioned later in 2.4.9 Priority Queue.

This data structure is built into python through the heapq module.

The binary characteristic

Simply, each node has exactly 0, 1 or 2 children.

The balanced/complete characteristic

The depth of a tree is the greatest number of children to reach a leaf from the root. In the example above, the tree has a depth of 2. To determine the depth of a tree, you can use the following recursive algorithm:

```
Method get_depth(node):
    If node is a leaf:
        Return 0
    childDepth = 0
    for each child of node:
        childDepth = max of childDepth and get_depth(child)
    return childDepth + 1
```

The balanced characteristic (for a binary tree) is when the depth of the left subtree is equal or one greater than the depth of the right subtree. This will make the tree complete, as all nodes that aren't leaves/parents of leaves have exactly 2 children.

This is important, as the runtime of the insertion and removal functions of a min/max heap depend on the number of nodes we need to consider, which is equal to the depth of the heap. By balancing the heap, we can have a consistent runtime for these functions, as the depth is constant for each side of the tree (or constant + 1).

This (in conjunction with the binary characteristic) results in the $O(\log_2(n))$ complexity we want.

2.4.7 Queue

A queue is another variant of an array known as a First-In-First-Out data structure. This structure only allows for the first element to be looked at (called peeking) or removed (called pop or dequeue). I will be using a circular queue, which stores a pointer called "head" and a pointer called "tail". Elements are added by storing them in position last (then last increments by 1) and are popped by deleting the element at position first (then first increments by 1). This allows data to be quickly added and removed, at the cost of 2 variables, and making sure that first/last are always valid indices for the array.

Index	0	1	2	3	4	5	6	7
Value	"e"	"O"	"l"	"d"	"Q"	"u"	"e"	"u"
Head: 4				Tail: 0				

An example of a circular queue.

Note how the tail pointer is less than the head pointer, meaning the queue is wrapping back around the static list. Also note how indices 1, 2, and 3 contain values, but are not in the range indicated by Head and Tail. This is another benefit of a circular queue, as we don't need to care about deleting old values, but rather overwriting values when necessary.

I will not be using queues themselves, but rather priority queues, which are covered below (2.4.8 Priority Queue).

2.4.8 Priority Queue

A priority queue is a variant of the queue, but rather than dequeuing according to the FIFO rule, it first dequeues according to a priority. Priority is just an additional value assigned to each entry. Any priority ties are broken according to FIFO. One implementation is to just use a queue to maintain FIFO, then whenever a dequeue occurs, you search for the first occurrence of the highest priority in the queue. This is O(n) time and is why I will instead implement a priority queue using a min heap. This will organise the queue according to priority but damages the FIFO rule. I am currently using priority queues for the A* algorithm, which don't care for FIFO, only which element has the highest priority, so losing this property is not an issue. Also, for the A* algorithm, I will be using the lowest priority in the queue, as the algorithm wants the "closest" node to be selected next, hence the min heap over the max heap.

Index	0	1	2	3	4	5	6	7
Priority	2	3	4	1	5	4	1	-
Value	"e"	"d"	"b"	"f"	"a"	"c"	"g"	-

A FIFO priority queue – implemented as a linear queue.

"a"	"b"	"c"	"d"	"e"	"f"	"g"

The above queue's outputs.

To get the next value from this queue, you would loop through all values, storing the first element of the highest priority. That element is then removed, and the rest shuffled along to remove gaps. This allows each priority tier to be FIFO, but the overall queue is not FIFO.

Index	0	1	2	3	4	5	6
Priority	5	4	3	1	2	3	1
Value	"a"	"b"	"d"	"f"	"e"	"c"	"g"

The same priority queue - implemented as a max-heap.

"a"	"b"	"c"	"d"	"e"	"g"	"f"

The above queue's outputs.

As shown by the examples, the binary heap implementation won't necessarily retain the FIFO quality that a priority queue should have, but as the priority queue is being used for the A* algorithm, the order in which elements are added is irrelevant, only the priority.

Big O Notation

Above O(n) and $O(\log_2(n))$ are used in reference to time. This is Big O Notation and refers to how time taken (or space used) changes as number of elements (n) changes. In O(n), it means that as you double the items, the time taken doubles – linear time – and $O(\log_2(n))$ means that as you double the items, the time taken increases by $\log_2(2n) = \log_2(n) + \log_2(2) = \log_2(n) + 1$ – logarithmic time, as doubling items increases time taken by a constant amount.

There are multiple time complexity notations. Big O specifically denotes the worst-case runtime, so while an algorithm may run on $\Theta(1)$ (Big Theta of 1 – average case is constant time), it may come with an $O(n^3)$ runtime, which is undesirable. We use Big O, as while on average an algorithm might be very fast, we don't want to risk the worst case of it being VERY slow.

2.5 Data Requirements

As a database is not required, there are few data requirements for this project. As discussed in 2.7, specific formats will be used, but the program will ensure that user inputs comply with these formats by either limiting what keys the game "listens" to (used when a name is required), or only allowing the user to input with preformatted elements (in the case of the level editor).

2.6 File Organisation and Processing

File Name	Access	Description	Master/Transaction
Save.txt	Random	Stores all save data	Master file
Levels	Random	Folder storing all level data files	Master folder
*.csv	Random	A file storing the tile-map for a level (stored in the levels folder)	Master file

As this is an offline game, all files will be master files that have their contents updated when a save button is pressed. In the save file, there will be 3 lines which will be independently updated based on which player is saving. Whenever a new level is made, it is created and saved in the Levels folder as a list of comma separated values (csv).

2.7 Table/Record Structures

2.7.1 Save.txt

The variables in the table below will be stored on one line to represent a save file (of which there will be 3 lines for 3 files)

Data Item	Data Type	Length	Format	Validation	Source	Notes
Username	String	16	AAAAAAAA	1≤Length≤16	FILES	---
Lives	Integer	2	12	>0	GAME	Initially set to 5
Health	Integer	2	12	>0	GAME	Initially set to 10
Level	Integer	3	123	>0, ≤100	GAME	Initially set to 1*

When a new file is created, the player will be prompted to enter a name, between 1 and 8 characters. This name will be in all caps.

All values must be present.

*-1 will be used to indicate an empty save as to remove an unnecessary restriction on the username.

2.7.2 Level.csv

This file will simply be a csv file, with a value representing a tile id for a cell, and a line of values representing a row of cells.

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X
1	Start																							
2																								
3														Crawler								Flyer		
4			Flyer										Spike	Spike	Spike									
5														Spike										
6														Spike	Crawler									
7														Spike	Spike									
8						Lway									Spike					Walker		Walker		
9						Floor	Floor	Uway							Spike				Floor	Floor	Floor	Floor	Floor	
10														Spike	Spike	Spike								
11														Crawler										
12																								
13	Player									Floor	Floor													
14	Floor	Floor	Floor																					
15	Floor	Floor	Floor															Rway	Walker	Walker	Walker		Flag	
16	Floor	Floor	Floor		Floor	Honey	Honey	Floor						Floor	Floor	Dway	Dway	Dway	Floor	Floor	Floor	Floor	Floor	Floor

(This is being displayed in Microsoft Excel, hence the numbered rows and alphabetized columns. These are not present in the file inherently).

2.8 Class Structure

Main Structure

This project will have its sections split into different modules. A consequence of this is a lack of true global variables. As such, most of the code will be stored in classes to mimic the behaviour of global variables, by referencing a property of the respective element.

As a consequence of this, the driver code for the project – such as the game loop – will have a "controller" class which stores the driver code as a function. So, there will be a controller class for the menus, one for the game, and one for the editor, as each one has distinct logic.

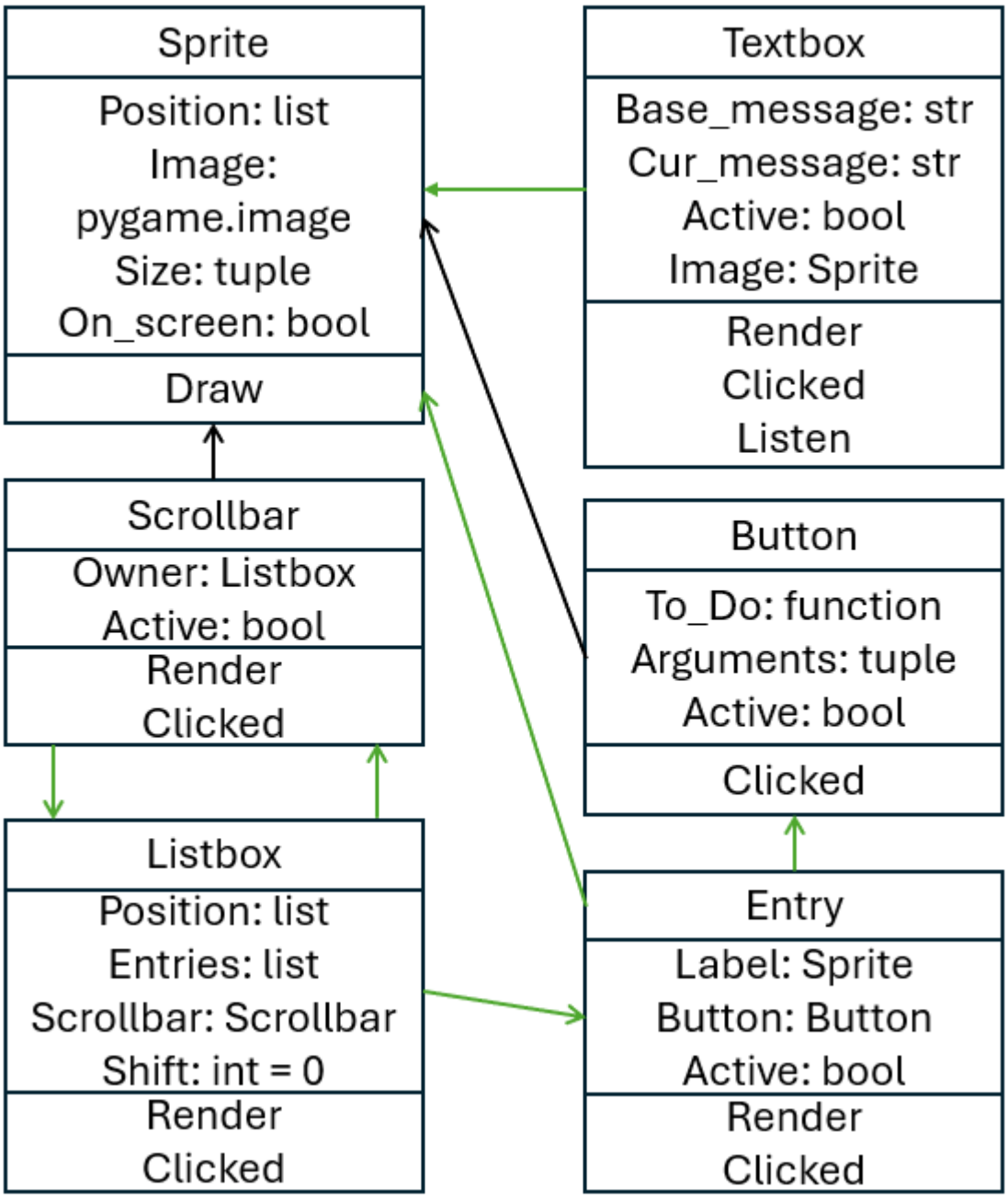
Pygame has two important classes which I will be using. The first is the surface, which is simply just an image of predefined size which you can paint onto – called blitting from bit block transfer. The screen that is displayed is one such surface, which we blit all our objects onto. The other is the rect, which just stores the position, width, and height of a rectangle. This is used to produce any hitboxes/colliders we need for the project.

All other classes will be used for actual objects. Each class will be put into its own module, unless they are very closely related – such as the different platform types.

The class structure for these classes can be displayed as a tree, with the "root" being the Sprite class. Most other classes are subclasses of the Sprite class, as most elements are visible, and thus need a sprite.

Note: All classes have a constructor method (__init__) which initialises all the properties of an object. This function is not listed in the class diagrams, as every class has one.

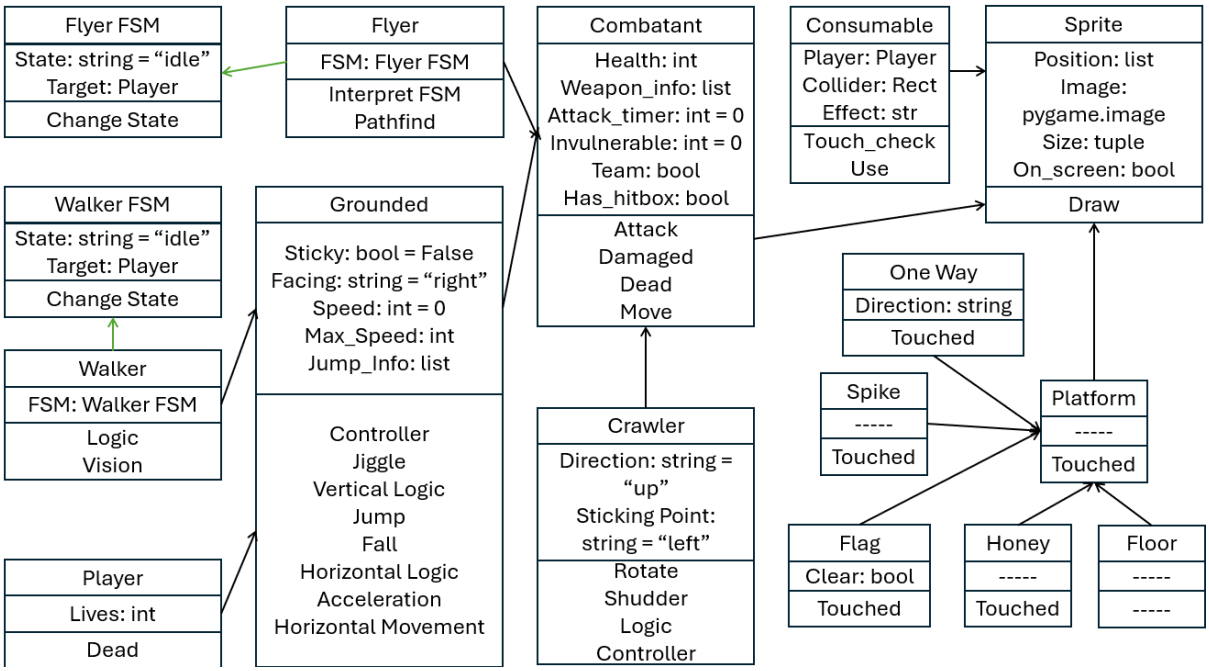
The rest of the class structure diagram will be split into two parts, one for the game classes, and one for the UI classes. The two diagrams can be combined through the common Sprite class.



Black arrows point to a superclass.
Green arrows indicate a reference to another class.

The UI class diagram has less inheritance and is more about references. This is because an entry is just a label and a button, and a listbox is just a list of entries.

The scrollbar and listbox reference one another, as not all listboxes need a scrollbar, and the scrollbar is used to scroll/shift its associated listbox.



Black arrows point to a superclass.
Green arrows indicate a reference to another class.

The game class diagram is just as shown, with the only note being that the different platform types are not strictly subclasses of some Platform superclass but are simply illustrated this way to show how they are all closely related, differing through polymorphic touched methods.

2.8.1 Sprite

This is the main class for most objects in the game, as we need a way to see them.

Property	Position	The absolute position of the sprite's top left corner in the level
Property	Image	The image of the sprite
Property	Size	The width and height of the image
Property	On-Screen	Is the sprite on the screen
Method	Draw	Draws the sprite onto its relative position on screen

Note: Pygame comes with an image class preinstalled, which will handle drawing the png file to the screen.

2.8.2 Platform

This class inherits from the sprite class and will thus inherit all the properties and methods of the sprite class – which will not be relisted.

There will not be a platform class, but rather a class for each type of platform (Standard, Honey, Spike, Flag, One Way). These classes will only have one method*, used to determine what should happen if something stands on it.

Standard	Nothing
Honey	Increase drag, prevent jumping
Spike	Deal damage
Flag	Tell the game controller to progress to the next level
One Way	Nothing

*One Way platforms will come with a property indicating their active side.

2.8.3 Combatant

This class inherits the sprite class and will similarly inherit all properties and methods. This class will act as the main parent for enemies and the player and contains the properties/methods to handle the combat side of the game.

Property	Health	How close is the entity to death
Property	Weapon_Info	How much damage does the attack do, how far can the entity attack, how long is the attack in frames.
Property	Invulnerable	How many frames is the entity immune to damage
Property	Attack_timer	Is the entity attacking – Used as a timer for how long an attack has taken
Property	Team	Is the entity on the player's team?
Method	Attack	Determine what entities are in range of the attack
Method	Damaged	Used to decrease a combatant's health
Method	Dead	Used to flag the combatant for deletion
Method	Move	Move the absolute position of the entity

2.8.4 Grounded Combatant

This class inherits from the combatant class and acts as a template for a grounded entity (player and enemy will have some difference, so will have to inherit from this class).

Property	Sticky	Used to activate honey's effects
Property	Facing	Used to determine which way the entity was moving, and which direction to draw an attack's hitbox
Property	Speed	The entity's current speed
Property	Max_Speed	The maximum speed the entity can reach
Property	Jump_Info	How long the entity has fallen for, can they coyote jump.
Method	Controller	Converts arrow key inputs (in player case) into instructions on how to move the entity (called by the Move Method)
Method	Jiggle	See 2.2.13
Method	Vertical Logic	Used to determine if the entity should jump or fall
Method	Jump	Moves the entity "up" until a full jump is done, or a collision occurs
Method	Fall	Moves the entity "down" until a full fall is done, or a collision occurs
Method	Horizontal Logic	Used to determine which way the entity should move (if any)
Method	Acceleration	Used to accelerate/decelerate the entity
Method	Horizontal Movement	Used to move the entity horizontally

2.8.5 Finite State Machines

These classes are base classes, and act as a finite state machine for their respective enemy class. While the two machines are different, that is only in the states which can be held, and their state transition tables. The properties and methods are identical.

Property	State	What state is the enemy in right now
Property	Target	What is the enemy targeting
Method	Change State	Changes the enemy's state (Covered in 2.2.7)

2.8.6 Walking Enemy (Walker)

This class inherits from the grounded combatant class and will contain an instance of the FSM class.

Property	FSM	Stores a reference to the associated finite state machine
Method	Logic	Uses the FSM state to choose what to do (2.2.7)
Method	Vision	Determine if the player is visible

Note: Idle state – do nothing, moving state –move, attacking state – attempt to attack the player, targeting state – don't turn around if an edge is reached (just stop).

2.8.7 Flying Enemy (Flyer)

This class inherits directly from the combatant class and will contain an instance of the FSM class.

Property	FSM	Same as in 2.6.6
Method	Interpret FSM	Determine if pathfinding is necessary, or the next move is known (2.2.7)
Method	Pathfind	Use the A* algorithm (2.2.9) to find the shortest path from flyer to target

Note: Idle state – do nothing, active state – pathfind then move, moving state – just move.

2.8.8 Crawling Enemy (Crawler)

This class inherits directly from the combatant class and has much simpler logic than the other enemy types.

Property	Direction	Which direction is the crawler moving
Property	Sticking point	Which direction is the wall the crawler is sticking to in relation to the crawler (90 degrees clockwise from the direction property)
Method	Rotate	Change the direction and sticking point
Method	Shudder	See 2.2.12
Method	Logic	Move around the platform (top-down controls) as indicated by the result of Logic
Method	Controller	Determine how to actually move

2.8.9 Player

This class inherits from the combatant class and will be the entity which the player can play as.

Property	Lives	How many times can the player die before game over
Method	Dead	Decrement the lives counter and reset the level

The Dead method is a polymorphism of the combatant's Dead method, as when the player dies, you want to reset the level, not delete it from memory.

2.8.10 Consumable

This class inherits from the sprite class and will be the entity which stores information for a modification to the player.

Property	Player	A reference to the player object, to prevent enemies from claiming consumables
Property	Collider	The pygame rectangle representing the consumables collider
Property	Effect	The effect which the consumable will have
Method	Touch_check	A method to validate if the player is touching the consumable, and if so, use it
Method	Use	A method to use and delete the consumable when touched (depends on the effect property).

There will be 2 main types of consumables: heals and upgrades. Heals simply increase the player's health, whilst upgrades provide a new movement option to the player.

Consumable Type	Effect
Heal	+1 health
Heal	+2 health
Heal	+5 health
Upgrade	Double jump
Upgrade	Wall jump
Upgrade	Double attack (attack on both sides at once)
Upgrade	Run
Upgrade	Dash

2.8.11 Label

Pygame offers a font render function to convert a string, font, and colour into an image. This image can then be used as the image in a sprite. As a result, all labels will just be sprites with a little extra setup.

2.8.12 Button

This class will inherit from the sprite class.

Property	To_Do	The function to be carried out if the button is clicked
Property	Arguments	The arguments to be passed in To_Do
Property	Active	Used as the latch in 2.2.4
Method	Clicked	Monitors whether the left mouse button is pressed whilst hovering over the button

2.8.13 Entry

This class will not inherit from anything, but will have reference to a Sprite and a Button

Property	Label	The text of the entry – Stored as a Label object
Property	Button	The button used when this entry is selected – Stored as a Button Object
Property	Active	Is this entry visible on the associated Listbox?
Method	Render	Combine the Label and Button into one image
Method	Clicked	Perform the referenced Button's Clicked method with a position relative to the entry

2.8.14 Listbox

This class will not inherit from anything but will have reference to many entries.

Property	Position	Where the visible Listbox should be drawn
Property	Entries	The actual elements of the list
Property	Scrollbar	The associated scrollbar
Property	Shift	How far has already been scrolled
Method	Render	Draw only the relevant entries
Method	Clicked	Perform the Clicked method of all the active entries with a position relative to the Listbox.

2.8.15 Scrollbar

This class is a subclass of Sprite and will inherit all of its properties and methods. This is a class which can optionally be created along with a Listbox when the Listbox has more than 3 elements (when scrolling becomes a necessity).

Property	Owner	References the associated Listbox
Property	Active	Used as the latch in 2.2.4
Method	Render	Used to reposition the scrollbar next to the Listbox
Method	Clicked	If the up or down buttons are pressed, then increment/decrement the associated Listbox's Shift property (must be in bounds). If the scrollbar portion is clicked, then alter shift according to what percentage of the way down the Scrollbar the click was.

2.8.16 Textbox

This class will not inherit from anything, but will contain a reference to the sprite class.

Property	Base Message	Stores what the textbox should display when the user has not inputted anything (or deleted their message)
Property	Current Message	Stores what the textbox is currently displaying
Property	Active	Used as the latch in 2.2.4
Property	Image	A sprite to display the current message
Method	Render	A method to convert the current message into a sprite
Method	Clicked	A method to detect whether the player is currently using the textbox or not
Method	Listen	A method to detect a user's keyboard inputs to modify the current message

Meta Structure

As stated previously, there are some classes which act as controllers for the different game states. There are three such classes: MAIN, GAME, and EDITOR.

2.9.17 MAIN

This is the class which is generated when the game is first loaded. It stores the main game loop, and all the UI functions to navigate between game states.

Property	Screen	A pygame screen object, which will be used to display my game to the real life monitor
Property	Clock	A pygame clock object, which ensures that the game updates every 60th of a second
Property	Game	A reference to the GAME object – needed to run the platformer
Property	Editor	A reference to the EDITOR object – needed to run the editor
Property	Game UI	A reference to the Game UI container object (see below) – needed to select and display the correct UI whilst the platformer is active
Property	Editor UI	A reference to the Editor UI container object (see below) – needed to display the UI elements of the editor, such as the tile palette
Property	Active UI	A list of UI elements which may be required on a given screen
Property	Running	A Boolean value to identify if the game is running (needed to safely quit the application)
Method	In Control	This method stores the main loop, and is called to start the game
Method	Change Screen	This method loads the UI of the requested screen, and (if necessary) runs the platformer/editor.
Method	Click Event	This method is ran whenever a click occurs, and loops through all UI elements to run their clicked() methods
Method	Listener	This method listens for keyboard input, and passes it to a Textbox (only ran if a Textbox is on screen)
Method	Run Game	This method tells the GAME object to load a file
Method	Delete Save	This method checks for a captcha, then deletes a save file

2.9.18 GAME

This class encapsulates all the properties and methods relating to the platformer itself, such as the player object, and the level information.

Property	Main Menu	A reference to the MAIN object – needed to load the correct UI during a pause, and change screens as necessary
Property	File	The index of the current save file in the main save .txt file
Property	Running	A Boolean storing whether the platformer is active or not
Property	Paused	A Boolean storing whether the platformer is paused or not
Property	Pause Latch	Used as the latch in 2.2.4
Property	Sent From	Stores which state the platformer was loaded from (Menu or Editor), needed to return to the correct part of the program
Property	Max Level	The number of levels in the main game – prevent loading a non-existent level

Property	Grid	A 2D array of all the tiles in the .csv file
Property	Graph Info	The dimensions of the grid (and thus the level)
Property	Tiles	A list of all the tile objects
Property	Flag	A reference to the flag object
Property	Tile Colliders	A list of all the tile colliders – otherwise each frame each entity would create the list again, so this saves time
Property	Player	A reference to the player object
Property	Crawlers	A list of all the crawler objects
Property	Flyers	A list of all the flyer objects
Property	Walkers	A list of all the walker objects
Property	Hitboxes	A list of all attack hitboxes objects (Sprite objects which disappear after 6 frames)
Property	Timer	The number of frames remaining until the level ends (countdown from 100)
Method	Load File	A method to load a level's .csv and instantiate all the required objects
Method	Clear Game	A method to delete all the old objects between levels to prevent old objects referencing deleted objects
Method	New File	A method to create a new save file from an inputted name
Method	Run Frame	A method to perform call all the logic methods – effectively the game loop
Method	Draw Game	A method to draw all visible objects onto the screen
Method	Forced Draw	A method to fully clear the screen and just display the game mid-frame (only called on level clear or player death)
Method	Movement	A method to call all the entity move methods
Method	Gather Attacks	A method to gather the attacks (See 2.2.8)
Method	Resolve Attacks	A method to resolve the attacks (See 2.2.8)
Method	On Screen	A method to only draw objects which are on-screen (See 2.2.5)
Method	Save to File	A method to modify the save file
Method	Reset	A method to play an animation then reset the level if the player dies.

2.9.19 EDITOR

This class encapsulates all the properties and methods relating to the level editor, such as the current grid.

Property	Main Menu	A reference to the MAIN object – needed to load the correct UI during a pause, and change screens as necessary
Property	Focuses	A set of two "focus" Sprites, which hover over the selected tile and mode so the user knows what a click will do
Property	Positions	A dictionary of positions for each image, and thus where to move the focus to when selected
Property	Images	A dictionary of all the images
Property	Active	A Boolean storing whether the editor is active or not
Property	File	The name of the current level file
Property	Mode	The current mode (Draw or Erase)
Property	Tile	The current tile

Property	Scroll	How much has the level been scrolled
Property	Player Position	What is the current position of the player (A player must exist to run the level, and there can only be one player)
Property	Flag Position	What is the current position of the flag (A flag must exist to run the level, and there can only be one flag)
Property	Previous Position	What was the previous position that a click occurred (used as a guard clause to save time when someone holds the mouse down over the same tile)
Property	Grid	A 2D array of strings storing the tile in each position – this is saved to the .csv
Property	Objects	A 2D array of Objects – this is to show the level so far
Property	Hidden	A Boolean storing whether the UI should be hidden or not
Property	Latches	A set of 9 Booleans, each used as the latch in (2.2.4) for the different key binds the player can use
Property	Palette	A string storing which palette is active
Property	Palette Pointer	An index for the active palette, so the player can "scroll" through the palettes
Method	New File	A method to create an empty .csv file for a new level
Method	Load	A method to load a csv file, and re-initialise the properties
Method	Unload	A method to save the csv file, then send to either the level select screen, or to game (if a flag and player also exist)
Method	Shift Focus	A method to move a focus over the selected tile/mode
Method	Change Mode	A method to update the mode (and then call Shift Focus)
Method	Change Scroll	A method to scroll the visible part of the level
Method	Button Clicked	A method to determine what each button should do
Method	Manage Flag And Player	A method to make sure only one flag/player exists, and that a reference only exists to a real object
Method	Modify Grid	A method to update the grid and object list according to the tile, mode, and mouse position
Method	Run Frame	A method to check for key bind presses, and mouse clicks – effectively the editor loop
Method	Shortcuts	A method to perform the latching logic (2.2.4) for each of the 9 key binds
Method	Hide Buttons	A method to load/unload the UI elements
Method	Change Palette	A method to change between the palettes

UI Container Structure

To separate logic from UI, each screen has its own module to generate the UI elements for that screen. As most screens are static, they just use a polymorphic function called generate_UI(). For these static screens, not classes are required, but it is still important to note how the UI is actually being generated.

What matters more here are the dynamic UI screens, such as the game overlay, the pause menu, and the editor. To accommodate these, I will use a game_ui class and an editor_ui class which will manage what to display.

2.8.20 Game UI

This screen requires a class to dynamically create the game overlay whilst unpaused (such as keeping the timer up to date) and to statically generate the pause menu when paused. This in itself does not warrant a class, but my implementation of the Button requires a function to be passed into it. So the buttons on the pause menu need a function to determine what to do. Whilst this could go into a meta class (like GAME), it would be easier to encapsulate it with the generate_UI functions for the game overlay and the pause menu.

Property	GAME	A reference to the GAME object
Property	MAIN_MENU	A reference to the MAIN object
Property	Common_images	A place to store the heart and health bar images which are needed in both the game overlay and the pause menu
Property	Main UI	A list of UI elements for the game overlay
Property	Pause UI	A list of UI elements for the pause menu
Method	Generate main UI	Generates the game overlay
Method	Generate pause UI	Generates the pause screen UI
Method	Resolve Pause	A function to determine what to do when the return button is pressed, or what to do if the quit button is pressed depending on which state loaded the level

The MAIN object will choose between Main UI and Pause UI when it needs to draw the UI.

2.8.21 Editor UI

A class is strictly required to store all the UI and only display it when the editor is not in the "Hidden" state. This class contains a method to generate the buttons which are on every palette, an abstract method to generate a palette from a list of button names, and a method for each palette which calls the generic method, passing that palette's elements.

These methods are just to make the code easier to read, and is functionally equivalent to the generate_UI method of all the other UI containers – just spread into small chunks.

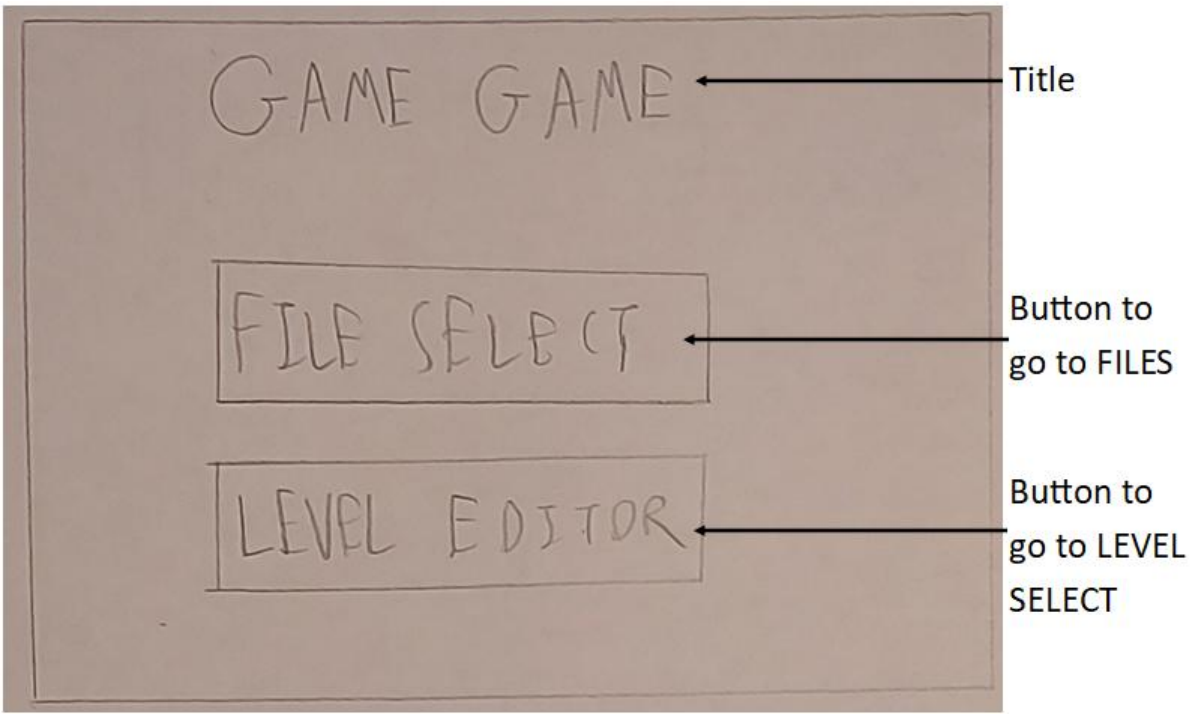
The importance of a class is to store all these UI elements, and select which to show based on the active palette and editor state.

Property	Tiles	A list of UI elements on the "Tiles" palette
Property	Entities	A list of UI elements on the "Entities" palette
Property	Mods	A list of UI elements on the "Mods" palette
Property	Static	A list of UI elements that are present on all palettes
Method	Generate UI	A set of methods to generate the above properties
Method	Assemble	A method to determine which palette's UI should be displayed, and then whether any UI should be displayed (dependant on EDITOR.hidden)

2.9 UI Design

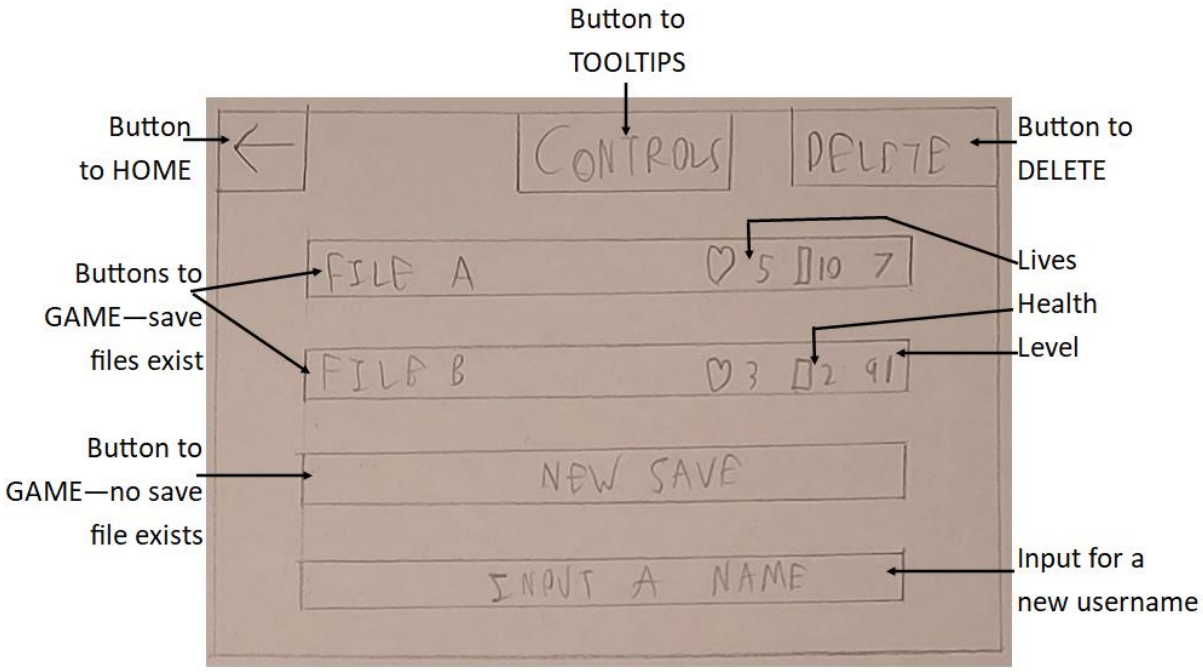
I will use a GUI for my program, due to the nature of a game already including graphical assets.

2.9.1HOME



Object	Event	Description
btnFILES	Click	Close HOME and load FILES
btnLEVELS	Click	Close HOME and load LEVEL SELECT

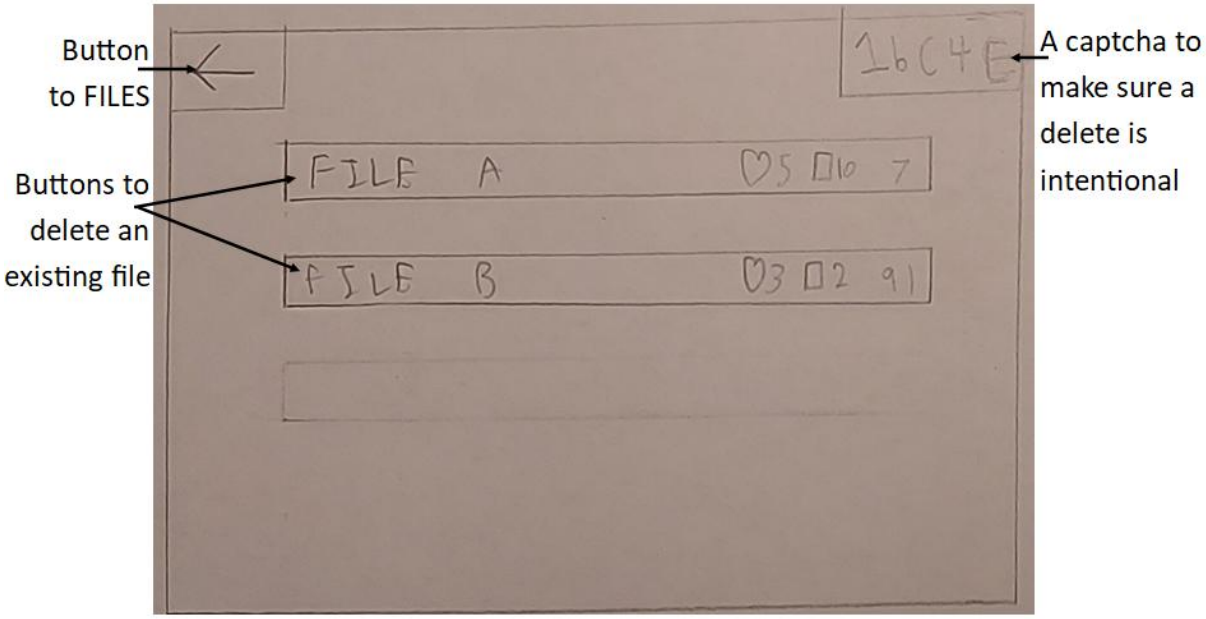
2.9.2 FILES



Object	Event	Description
FILES	Load	On load, the form reads the save data file to show the correct information on the file buttons
btnHOME	Click	Close FILES and load HOME
btnTOOLTIPS	Click	Close FILES and load TOOLTIPS
btnDELETE	Click	Close FILES and load DELETE
btnFileA	Click	Close FILES and load GAME, using the first save file's data
btnFileB	Click	Close FILES and load GAME, using the second save file's data
btnFileC	Click	Close FILES and load GAME, starting a new save file*
txtName	Typing	Add/Remove characters from the textbox based on what the user types
FILES	Unload	If a new save file needs to be created, use the contents of txtName as the save file's name

*The three buttons will work differently based on which saves have an active save file. In this case, saves A and B both have saves, whilst C does not.

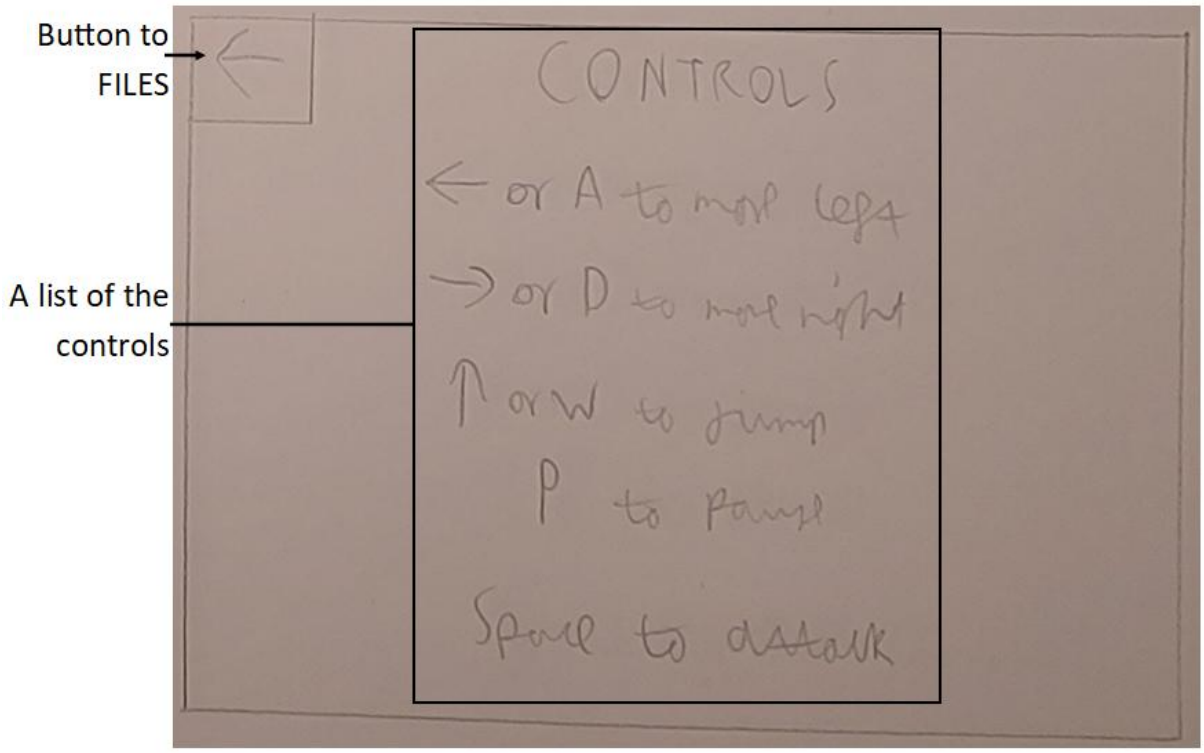
2.9.3 DELETE



Object	Event	Description
DELETE	Load	On load, reread the save data file to show the correct information for each file. Also, generate a new random captcha from numbers, lowercase, and uppercase letters.
btnFILES	Click	Close DELETE and load FILES
btnFileA	Click	Close DELETE and load FILES, and delete the information for the first save file if the captcha was successful
btnFileB	Click	Close DELETE and load FILES, and delete the information for the second save file if the captcha was successful
txtCaptcha	Typing	Add/Remove characters from the textbox based on what the user types

Because no save data exists for file C in this example, a button to delete file C is not generated. If save data were to exist for file C, then a button would appear in the faint area.

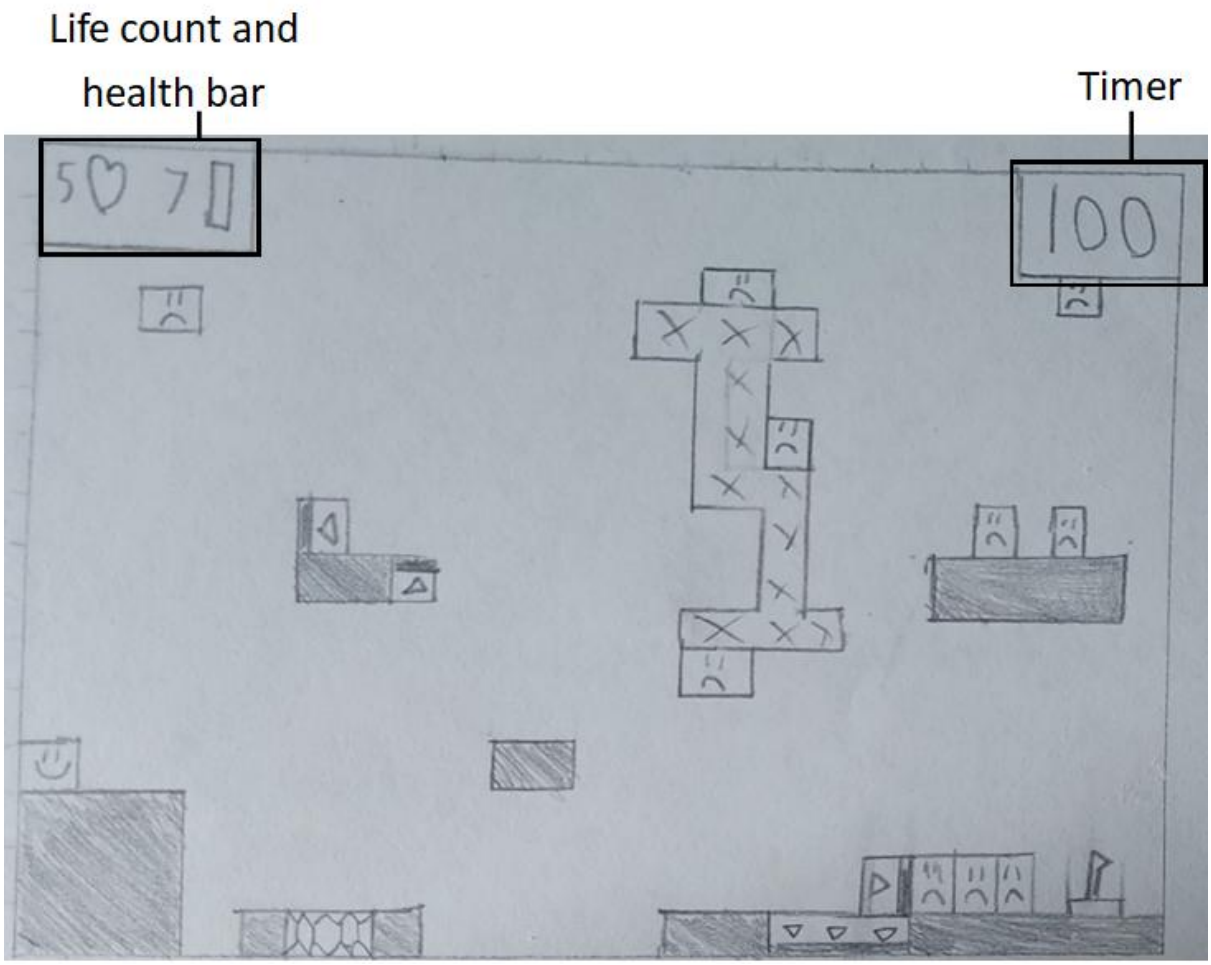
2.9.4 TOOLTIPS



Object	Event	Description
btnFILES	Click	Close TOOLTIPS and load FILES

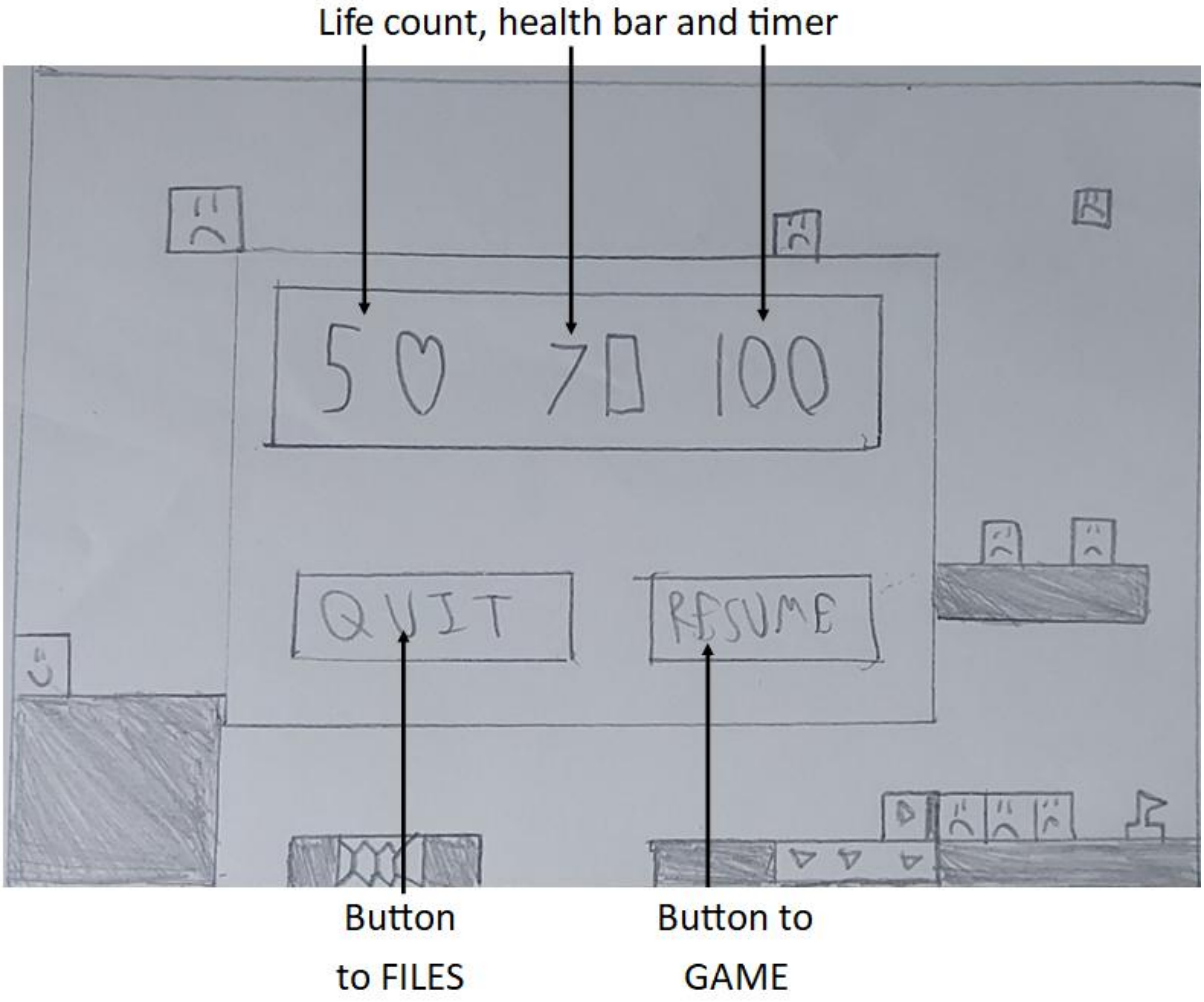
This form is used just to show the controls for the game/editor. The screen will display different things according to if the form was loaded by FILES – showing the controls for the game – or if the form was loaded by LEVEL SELECT – showing the controls for the editor.

2.9.5 GAME



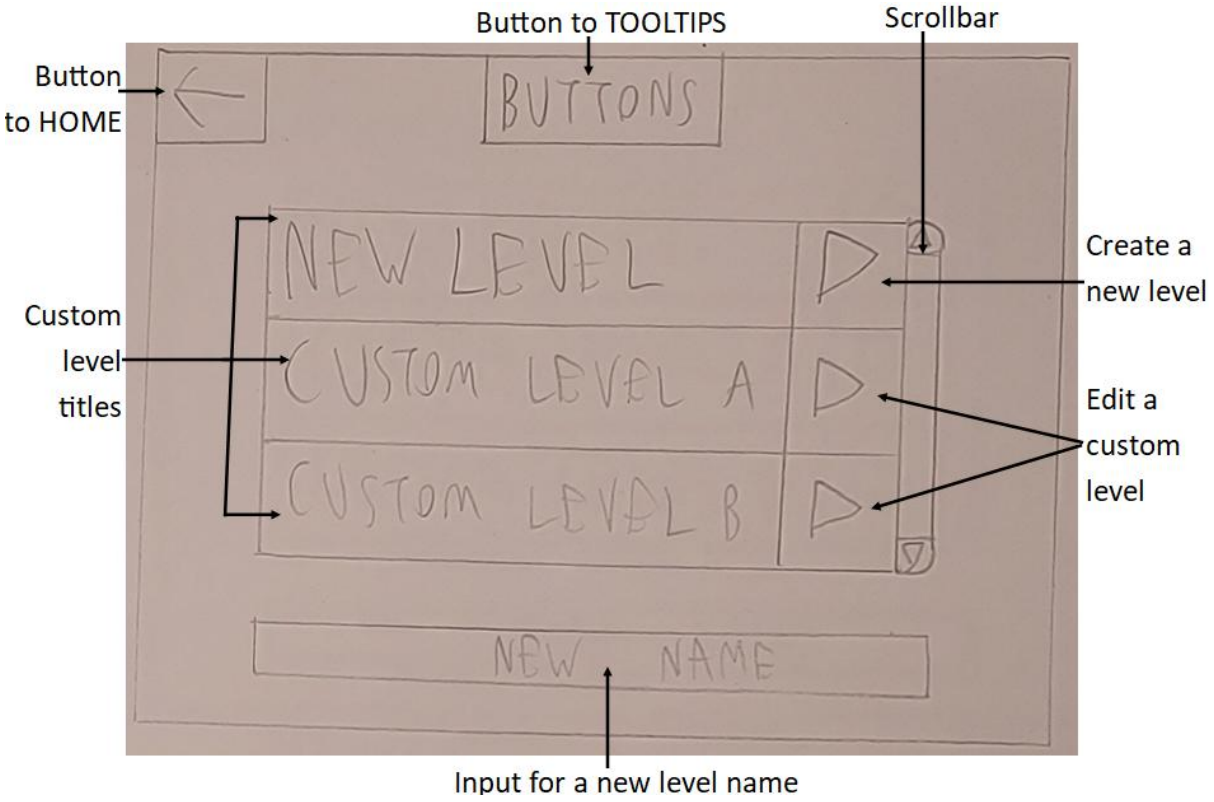
Object	Event	Description
GAME	Load	Based on the passed save file (if from FILES) or the passed level name (if from LEVEL SELECT), load the relevant level.csv file and instantiate all the objects, and initialise the timer to 100 and the health display to show the save file's information (a custom level will have 1 life and 10 health)
Health Display	Player is damaged	Decrement the value displayed based on how much damage was taken (Healing just increments the counter instead)
Timer	1 second passes	Decrement the timer display by 1
GAME	The p key was pressed	Load PAUSED – the game logic has stopped, but GAME is still active
GAME	The player dies	If playing a custom level: reload GAME Otherwise, if on 0 lives: close GAME and load FILES Otherwise: reload GAME
GAME	The player beats the level	Reload GAME, with either the next level, or the same level if a custom level is being played

2.9.6 PAUSED



Object	Event	Description
PAUSED	Load	Move the health and timer displays to the centre of the screen
btnFILES	Click	Close PAUSED and GAME and load FILES
btnGAME	Click	Close PAUSED and continue the game logic

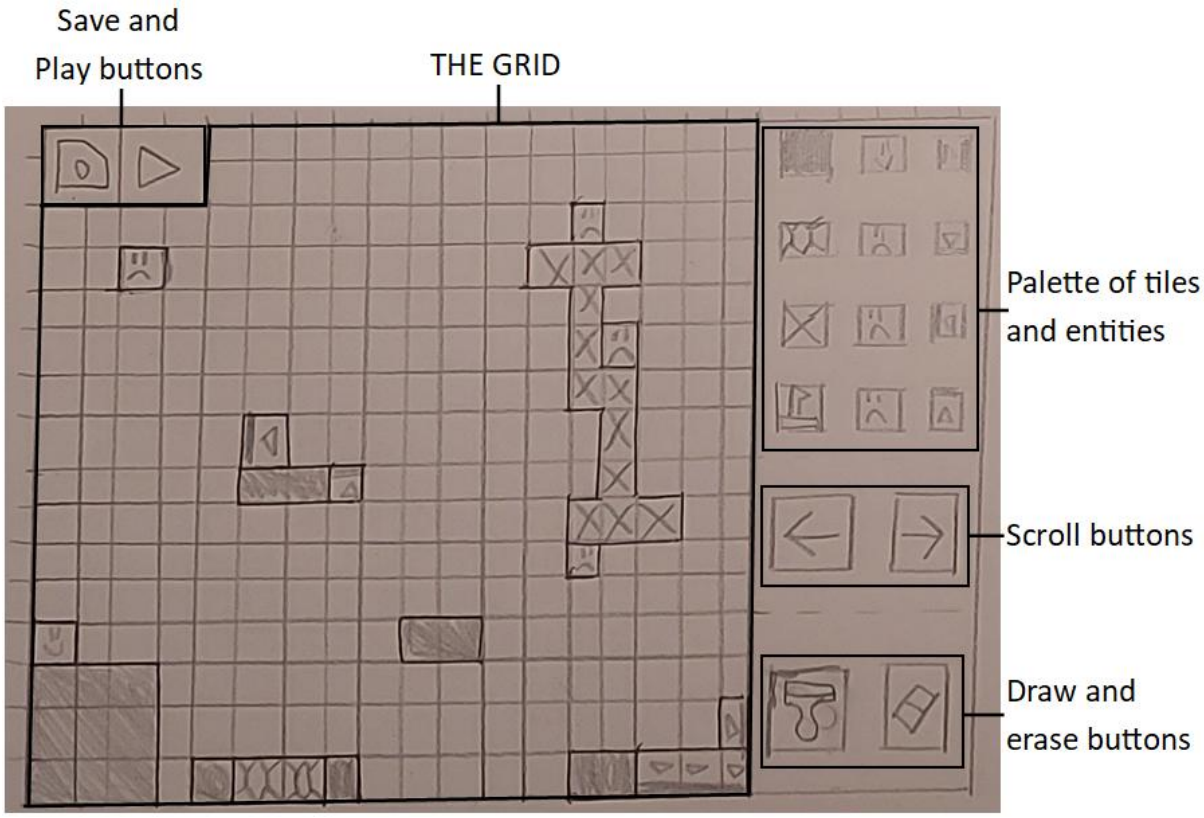
2.9.7 LEVEL SELECT



Object	Event	Description
LEVEL SELECT	Load	Open the custom levels folder and create a list of custom levels. If 0, 1, or 2 custom levels exist, don't create a scrollbar – as there is no need to scroll
btnHome	Click	Close LEVEL SELECT and load HOME
btnTOOLTIPS	Click	Close LEVEL SELECT and load TOOLTIPS
btnNEW	Click	Close LEVEL SELECT and load EDITOR, creating an empty level
btnCustom	Click	Close LEVEL SELECT and load EDITOR, loading that custom level's csv file*
Scrollbar – Arrow	Click	Scroll up/down by 1 entry
Scrollbar - Body	Click	Jump as far down the list as the click specifies – clicking the middle jumps to the middle
txtName	Typing	Add/Remove characters from the textbox based on what the user types
LEVEL SELECT	Unload	If a new level needs to be created, use the value in txtName as that level's name

*One button will exist for each custom level file, but they all have the same behaviour: load the respective file in the EDITOR.

2.9.8 EDITOR



Object	Event	Description
EDITOR	Load	Load the respective level file, drawing each tile to the grid
btnSave	Click	Close EDITOR and load LEVEL SELECT, saving the level in the process
btnPlay	Click	Close EDITOR and load GAME, saving the level and passing the file to GAME
Grid	Click	Effectively one big button which changes to show the level as it is being edited
btnPalette	Click	Change the "tile to draw" variable to the button that was clicked*
btnScroll	Click	Scroll the grid left/right according to which button was pressed*
btnDraw	Click	Change the mode variable to "draw"
btnErase	Click	Change the mode variable to "erase"

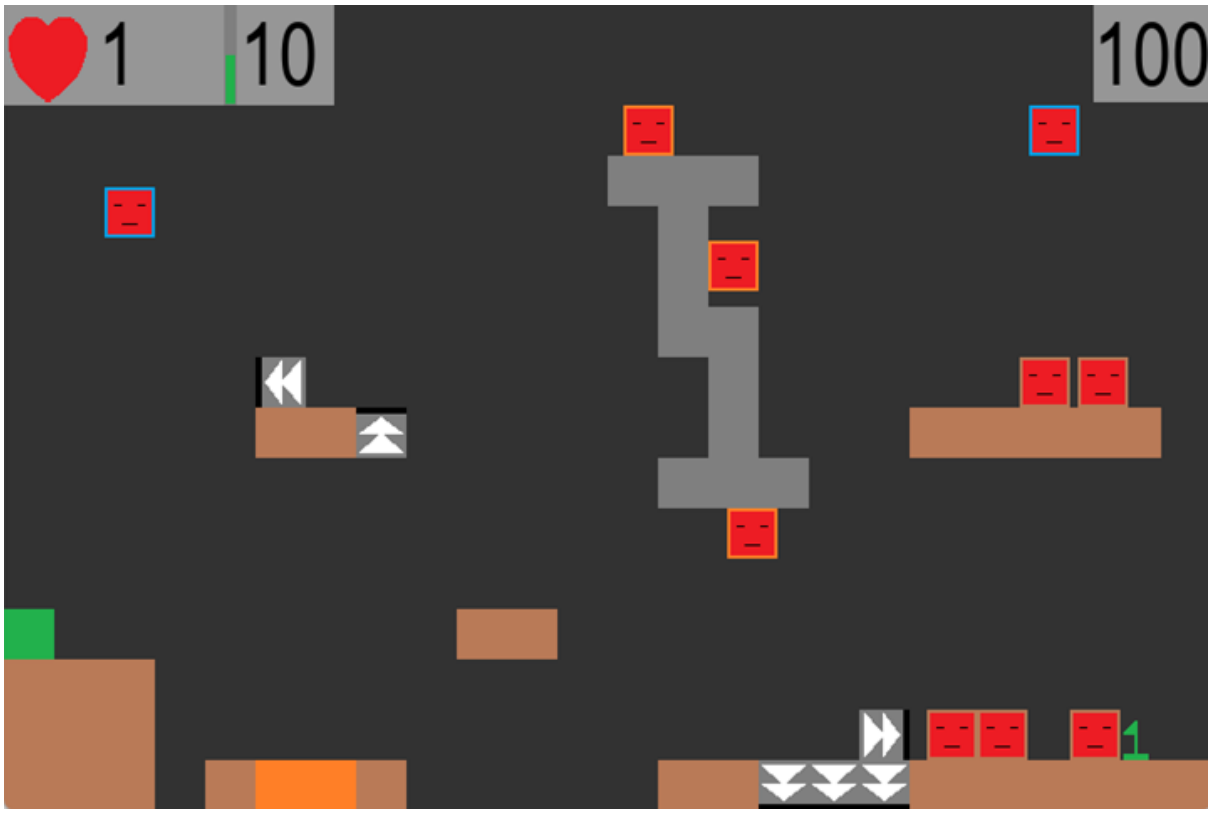
*One button will exist for each tile option and each direction of scroll. Their function is the same as the function specified above but they have not been shown to save space.

2.9.9 Implementations

HOME

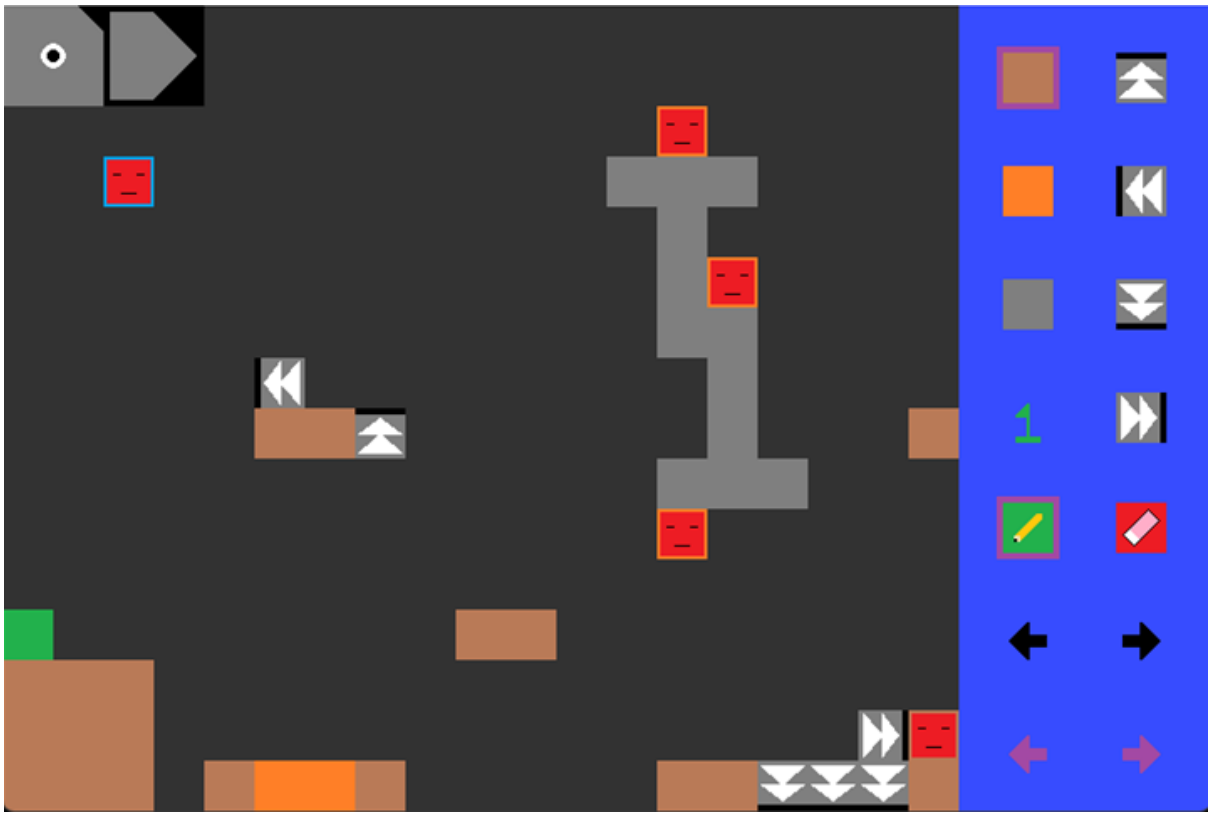
One of the simplest forms in the program. I have chosen a dark background to stay on theme with the game taking place in an evil magic tower. I have chosen bright colours for my text to stand out, with different colours differentiating the sections beyond the text itself. The buttons have a solid black background to clearly indicate the active region of each button.

GAME



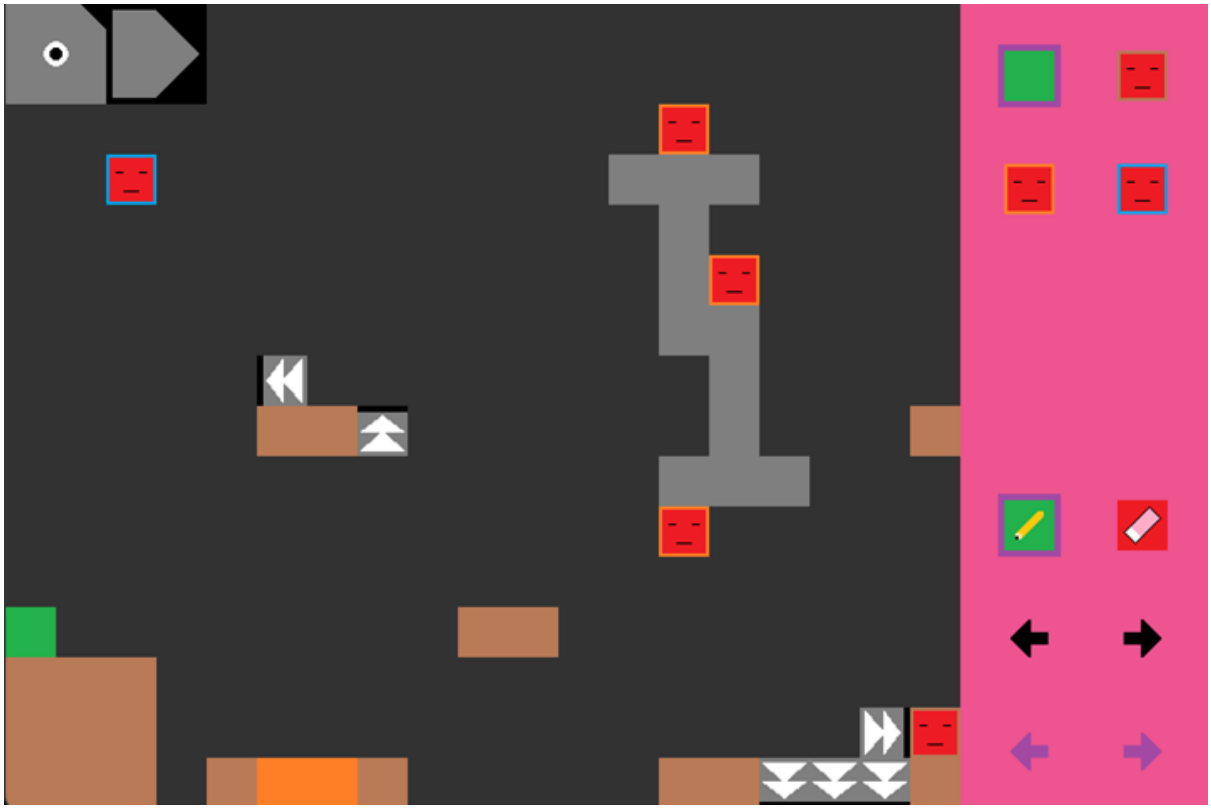
This is an updated implementation of the level shown in 1.5. The enemies have different positions from the example as they have had time to move whilst I was taking a screenshot. The one-way graphic has also been updated as John suggested – they now clearly indicate that motion is allowed in the shown direction, with a solid line indicating that the given side is solid. A lighter grey has been used for the UI elements to separate them from the background, whilst not drawing attention away from the game itself.

EDITOR

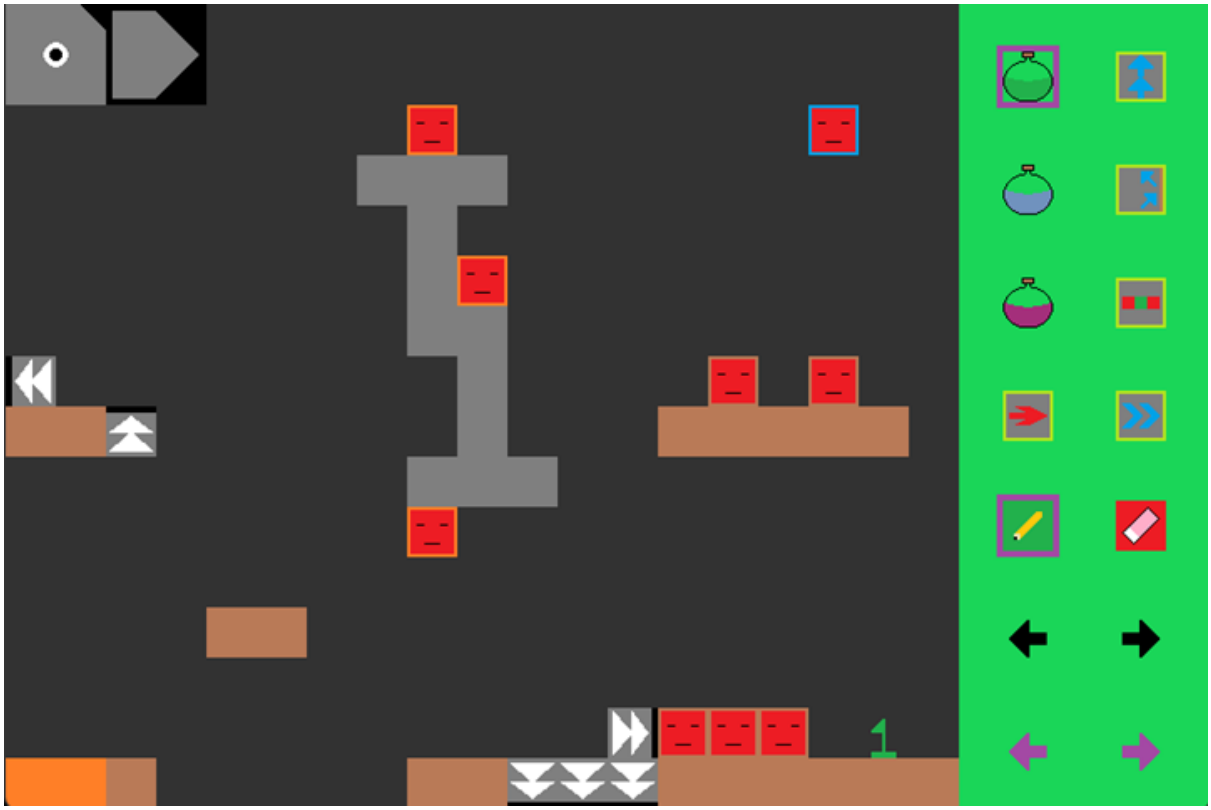


This is just a copy of the above design, but with each enemy having a border colour to indicate their type. The palette in the design was very busy, so I have separated it into three palettes, grouped by type. This is the "tile" palette, with one for "entities" and one for "consumables". To accommodate the change, I have added purple arrows at the bottom to change palette. The palette section has been made smaller to show more of the grid as a result of less tiles being present on the palette. Another change is the purple borders around the floor tile and the draw mode. These indicate what is the current tile and mode, and will move to provide a reminder of what a click would do.

I have used purple as it stands out on all the background colours, and made the scroll arrows black to differentiate them from the palette change buttons.



This is the same form, but with the "entities" palette loaded. Pink was used to make it distinct from the "tile" palette's blue, and the "consumables" palette's green.



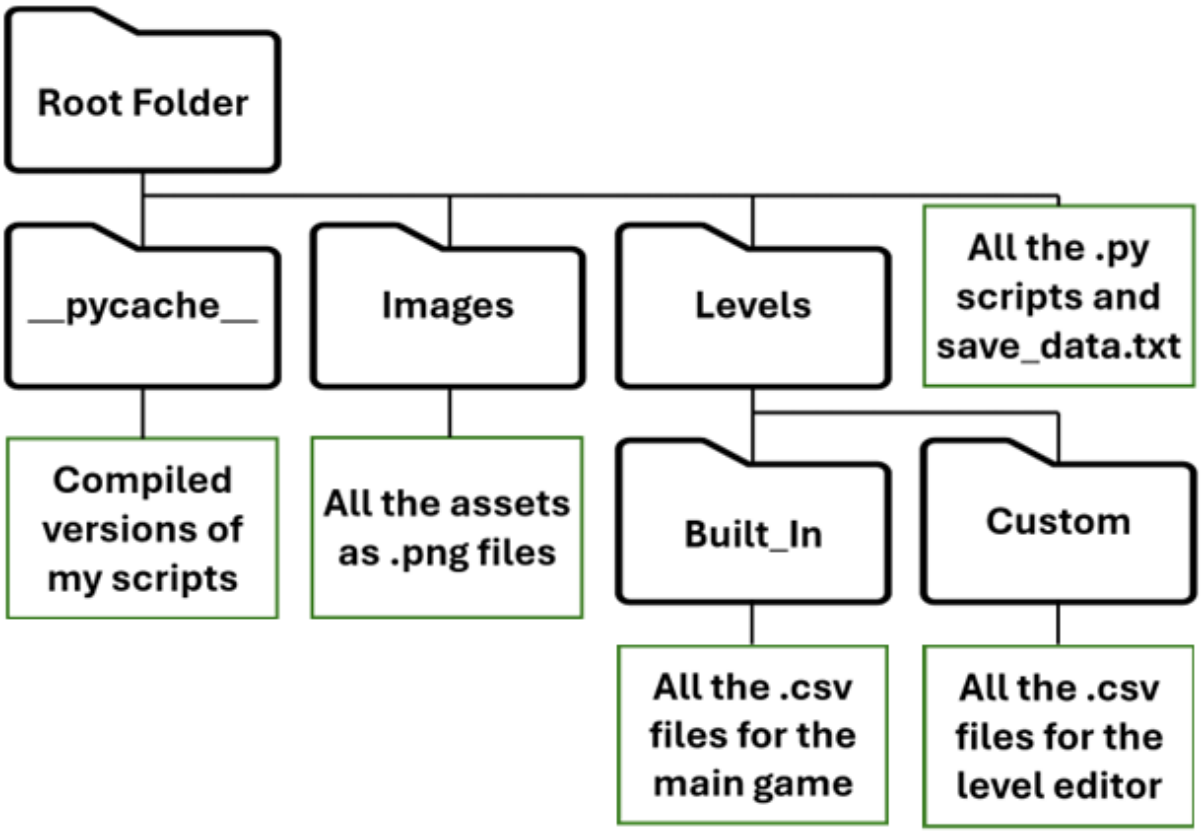
This is the "consumables" palette. By separating into multiple palettes, I can provide more options. The consumables are as follows:

Small Heal (1 health)	Double Jump
Medium Heal (2 health)	Wall Jump
Large Heal (5 health	Double Strike (allows the player to attack on both sides)
Run (doubles max speed)	Dash (allows the player to dash 3 tiles)

The screen has also been scrolled to show the rest of the level.

3 Program

3.1 File Structure



3.2 Drivers

3.2.1 MAIN.py

```
###IMPORTS###
import pygame
import sys
import glob
###UI Modules###
import Home_Screen_UI
import Files_Screen_UI
import Levels_Screen_UI
import Delete_Screen_UI
import Control_Screen_UI
import Editor_Tip_Screen_UI as ETSUI
import Editor_Screen_UI
from Game_Screen_UI import Game_UI
###Logic Modules###
import GAME
import EDITOR


#A class to display the relevant objects (the driver object)
class Menu_Controller:
    def __init__(self):
        pygame.init()                                                   #Create a window
        pygame.display.set_caption("GAME GAME")
        self.screen = pygame.display.set_mode((768, 512))
        self.clock = pygame.time.Clock()

        self.change_screen("#HOME")                                     #Load the home screen
        self.game = GAME.Game_Controller(self)                          #Instantiate the logic modules
        self.game_ui = Game_UI(self.game)
        self.editor = EDITOR.Editor_Controller(self)
        self.editor_ui = Editor_Screen_UI.Editor_UI(self.editor)

    #A function to delete a save file
    def delete_save(self, args):
        for part in self.active_UI:                                     #Find the textbox object
            try:
                answer = part.get_message()
                captcha = part.base_message
            except: pass

        if(captcha == answer):                                          #Determine if the captcha was correct
            file = args[0]

            saves = open("save_data.txt", "r").read().split("\n")       #If so load the save data
            this_save = "new, 5, 10, -1"                                #Delete the relevant save
```

```
        saves[file] = this_save

        f = open("save_data.txt", "w")                                          #Then store to the file
        f.write("\n".join(saves))
        f.close()

    self.change_screen("#FILES")

#A function to load the platformer
def run_game(self, context, caller):
    self.game.load_file(context, caller)                                        #Just load the platformer
    self.active_UI = []

#Sets the UI elements and passes "control"
def change_screen(self, target, caller = "Menu"):
    if(type(target) is tuple or type(target) is list):                          #Split the arguments if necessary
        args = target[1:]
        target = target[0]

    match target:                                                               #Load the relevant screen
        case "#HOME":
            self.active_UI = Home_Screen_UI.generate_UI(self.change_screen)
        case "#FILES":
            self.active_UI = Files_Screen_UI.generate_UI(self.change_screen)
        case "#LEVELS":
            self.active_UI = Levels_Screen_UI.generate_UI(self.change_screen)
        case "#GAME":
            self.run_game(args[0], caller)
        case "#DELETE":
            self.active_UI = Delete_Screen_UI.generate_UI(self.change_screen, self.delete_save)
        case "#CONTROLS":
            self.active_UI = Control_Screen_UI.generate_UI(self.change_screen)
        case "#TIPS":
            if(args[0] == "Buttons"):                                           #The editor has two tip screens
                self.active_UI = ETSUI.generate_buttons_UI(self.change_screen)
            else:
                self.active_UI = ETSUI.generate_keybinds_UI(self.change_screen)
        case "#NEW GAME":
            name = ""
            marked_part = None
            for part in self.active_UI:                                         #Find the Textbox
                try:
                    name = part.get_message().strip()                           #And check if a username was passed
                    marked_part = part
                except: pass
            if(name == ""):                                                     #If not
                marked_part.update("Input a name first")                        #Update the textbox
                marked_part.change_focus(True)
                return
            self.game.new_file(name, args[0])                                   #Else create a new file
            self.run_game(args[0], caller)                                      #And run the game
        case "#NEW LEVEL":
            name = ""
            marked_part = None
            for part in self.active_UI:                                         #Same thing as #NEW GAME
                try:                                                            #Make sure a level name was given
                    name = part.get_message().strip()
                    marked_part = part
                except: pass
            if(name == ""):
                marked_part.update("Input a name first")
                marked_part.change_focus(True)
                return
            existing_files = [lvl[14:len(lvl)-4] for lvl in glob.glob('Levels/Custom/*.csv')]   #Ensure a level name is unique
            if(name in existing_files):
                marked_part.update("Input a name first")
                marked_part.change_focus(True)
                return

            self.editor.load(name, new_level = True)                            #And only load if so
        case _:                                                                 #The default case is to load a
            self.editor.load(target)                                            #custom level of the passed name

#A function to monitor for clicks
def click_event(self, mouse_engaged: bool):
    mouse_position = list(pygame.mouse.get_pos())                               #Determine where the click is
    for part in self.active_UI:                                                 #And for each object
        try: part.clicked(mouse_engaged, mouse_position)                        #Try to perform clicked function
        except: pass                                                            #But ignore non buttons

#A function to pass a Textbox keyboard inputs
def listener(self, part, keys):
    part.del_timer = max(0, part.del_timer - 1)                                 #Decrement the backpace timer
    for key in keys:                                                            #Then for each key press
        part.listen(key)                                                        #Pass the key to the Textbox

#The main driver loop
def in_control(self):
    '''
    On loop:
    Display the active elements
    Manage UI
    '''
    self.running = True
    pause_started = False
    mouse_down = False
    while self.running:                                                         #MAINLOOP
        key_buffer = []
        for event in pygame.event.get():                                        #For each pygame event
            if(event.type == pygame.MOUSEBUTTONDOWN):                           #If the left mouse is down
                if(event.button == 1):
                    mouse_down = True                                           #Flag mouse as down
                    self.click_event(mouse_down)                                #Run click event
            if(event.type == pygame.MOUSEBUTTONUP):                             #If the left mouse is up
                if(event.button == 1):
                    mouse_down = False                                          #Flag mouse as up
                    self.click_event(mouse_down)                                #Run click event
            if(event.type == pygame.KEYDOWN):                                   #If a key was pressed
                key_buffer.append(pygame.key.name(event.key))                   #Add to the buffer
            if(event.type == pygame.QUIT):                                      #If the game was exited
                pygame.display.quit()                                           #Close the game
                sys.exit()

        self.screen.fill((50,50,50))                                            #Clear the screen

        if(self.game.running):                                                  #If the game is running
            if(not self.game.paused):                                           #And unpaused
                self.game_ui.generate_main_ui()                                 #Load the unpaused UI
                self.active_UI = self.game_ui.main_UI
                pause_started = False
```

```
                elif(not pause_started):                                    #Else if paused and it is the first pause frame
                    self.game_ui.generate_pause_ui()                        #Load the paused UI
                    self.active_UI = self.game_ui.pause_UI
                    pause_started = True
                self.game.run_frame()                                       #Then run the game

            if(self.editor.active):                                         #If the editor is active
                self.editor.run_frame(mouse_down)                           #Monitor for mouse presses
                self.active_UI = self.editor_ui.assemble()                  #And load the relevant UI

            for part in self.active_UI:                                     #For each UI element
                part.draw(self.screen)                                      #Overlay it
                try:
                    part.listen("")                                         #Then "listen" if it is a Textbox
                    self.listener(part, key_buffer)
                except: pass

            pygame.display.flip()                                           #Draw to the screen

            self.clock.tick(60)                                             #Then tick 1 60th of a second

if __name__ == "__main__":                                                  #Only run if this file was the initial file
    controller = Menu_Controller()                                          #Instantiate the controller
    controller.in_control()                                                 #And begin the driver code
```

3.2.2 GAME.py

```
###IMPORTS###
###Assets###
from Sprite import Sprite
import Crawler
import Flyer
import Walker
import Platforms as P
import Player
import Consumables as C
###Premade functions
import csv
import pygame
import time
import glob


#A class to store the methods and variables needed to run the platformer
class Game_Controller:
    def __init__(self, menu_control):
        self.main_menu = menu_control
        self.file = None
        self.running = False
        self.paused = False
        self.pause_latch = False
        self.sent_from = "Menu"
        self.max_level = len(glob.glob('Levels/Built_In/*.csv'))            #Determines the number of levels that exist

    #A function to create a new save
    def new_file(self, name, file: int):
        saves = open("save_data.txt", "r").read().split("\n")               #Load the save files
        this_save = f"{name}, 5, 10, 1"                                     #Update the relevant row
        saves[file] = this_save

        f = open("save_data.txt", "w")                                      #Then rewrite to the file
        f.write("\n".join(saves))
        f.close()

    #Auxiliary function to free up memory
    def clear_game(self):
        self.grid = None
        self.graph_info = None
        self.tiles = None
        self.flag = None
        self.tile_colliders = None
        self.player = None
        self.crawlers = None
        self.flyers = None
        self.walkers = None
        self.hitboxes = []

    #A function to load a level
    def load_file(self, file, sent_from: str):
        self.sent_from = sent_from
        self.clear_game()
        self.scroll = 0

        #Loading screen
        self.main_menu.screen.fill((0,0,0))                                 #Fill the screen with black
        pygame.display.flip()
        time.sleep(1)                                                       #And stall for 1 second

        self.file = file
        self.running = True
        self.paused = False

        #Load File
        if(self.sent_from == "Menu"):                                       #Loads a built_in level
            this_save = open("save_data.txt", "r").read().split("\n")[file]
            player_info = [int(x) for x in this_save.split(", ")[1:3]]
            this_level = this_save.split(", ")[3]
            if(int(this_level) > 100):
                this_level = 100
                print("Congratulations, you have beaten Game Game :D")
            level_file = f"Levels/Built_In/{this_level}.csv"
        else:                                                               #Loads a custom level
            player_info = [1, 10]
            level_file = f"Levels/Custom/{file}.csv"

        #Read Level File
        with open(level_file, "r") as lvlfile:                              #Actually read the level file
            csv_reader = csv.reader(lvlfile)
            row_data = [row for row in csv_reader]

        #Gridify
        to_generate = {"Player":[], "Flag":[],                              #Create a dictionary for each object
                       "Crawler":[], "Walker":[], "Flyer":[],
                       "Floor":[], "Honey":[], "Spike":[],
                       "Uway":[], "Dway":[], "Lway":[], "Rway":[],
                       "SHeal":[], "MHeal":[], "LHeal":[],
```

```
                "DJump":[], "WJump":[], "DStrike":[],
                "Run":[], "Dash":[]}
    self.grid = [[""]*len(row_data[0]) for i in range(len(row_data))]                    #And a blank array for the grid
    for y,row in enumerate(row_data):                                                    #For each tile
        for x,tile in enumerate(row):
            if(tile == "Start"): continue                                                #Ignore non_solid objects
            if(tile != ""): to_generate[tile].append((x,y))
            if(tile in ["Player","Flag","Crawler","Walker","Flyer"]):                    #Treat the entities as non_solid objects
                tile = ""
            self.grid[y][x] = tile                                                       #Store the platform type to the grid
    level_bound = len(self.grid[0]) * 32
    self.camera_edge = level_bound - 768

    ROWS = len(self.grid)                                                                #Set and group these constants for later use
    COLS = len(self.grid[0])
    self.graph_info = (ROWS, COLS)

    #GENERATE OBJECTS - Tiles, Entities
    img_dict = {}
    for filename in [image[7:] for image in glob.glob('Images/*')]:                      #For each image file
        this_img = pygame.image.load(f"Images/{filename}").convert_alpha()               #Load the image
        img_dict[filename[:-4]] = this_img                                               #And store to the dictionary

    ##TILES
    self.tiles = []
    for position in to_generate["Floor"]:                                                #For each floor:
        true_position = [position[0] * 32, position[1] * 32]                             #Find the in game position
        new_floor = P.Std_Platform(true_position, img_dict["Floor"])                     #Instantiate an object
        self.tiles.append(new_floor)                                                     #Store to the list

    for position in to_generate["Honey"]:                                                #Repeat for each honey tile
        true_position = [position[0] * 32, position[1] * 32]
        new_honey = P.Honey_Platform(true_position, img_dict["Honey"])
        self.tiles.append(new_honey)

    for position in to_generate["Spike"]:                                                #And for each spike
        true_position = [position[0] * 32, position[1] * 32]
        new_spike = P.Spike_Platform(true_position, img_dict["Spike"])
        self.tiles.append(new_spike)

    for position in to_generate["Uway"]:                                                 #And for each upwards one_way
        true_position = [position[0] * 32, position[1] * 32]
        new_uway = P.One_Way_Platform(true_position, img_dict["Oway"], "down")
        self.tiles.append(new_uway)

    for position in to_generate["Dway"]:                                                 #And do mostly the same for downwards one_ways
        true_position = [position[0] * 32, position[1] * 32]
        img = pygame.transform.rotate(img_dict["Oway"], 180)                             #Just rotating the one_way image accordingly
        new_dway = P.One_Way_Platform(true_position, img, "up")
        self.tiles.append(new_dway)

    for position in to_generate["Lway"]:                                                 #Repeat the new method for left one_ways
        true_position = [position[0] * 32, position[1] * 32]
        img = pygame.transform.rotate(img_dict["Oway"], 90)
        new_lway = P.One_Way_Platform(true_position, img, "right")
        self.tiles.append(new_lway)

    for position in to_generate["Rway"]:                                                 #And then for right one_ways
        true_position = [position[0] * 32, position[1] * 32]
        img = pygame.transform.rotate(img_dict["Oway"], 270)
        new_rway = P.One_Way_Platform(true_position, img, "left")
        self.tiles.append(new_rway)

    flag_position = (to_generate['Flag'][0][0] * 32, to_generate['Flag'][0][1] * 32)     #Determine where the flag is
    self.flag = P.Flag(flag_position, img_dict["Flag"])                                  #And instantiate the flag object
    self.tiles.append(self.flag)                                                         #Then add to the list

    self.tile_colliders = [pygame.Rect(tile.position, tile.size) for tile in self.tiles] #Then create a collider for all the tiles

    ##ENTITIES
    player_pos = [to_generate["Player"][0][0] * 32, to_generate["Player"][0][1] * 32]    #Determine where the player is
    lives = player_info[0]                                                               #And the lives/health
    health = player_info[1]
    self.player = Player.Player(player_pos, img_dict["Player"], lives, health, level_bound)  #Instantiate the Player object

    self.crawlers = []
    for position in to_generate["Crawler"]:                                              #For each Crawler:
        true_position = [position[0] * 32, position[1] * 32]                             #Find the in_game position
        new_crawler = Crawler.Crawler(true_position, img_dict["Crawler"], self.grid)     #Then instantiate the Crawler
        try:
            if(new_crawler.destroy_flag): continue                                       #This is a check to remove trapped/floating crawlers
        except:
            pass
        self.crawlers.append(new_crawler)                                                #Then add to the crawler list

    self.flyers = []
    for position in to_generate["Flyer"]:                                                #Repeat for all flyers
        true_position = [position[0] * 32, position[1] * 32]
        new_flyer = Flyer.Flyer(true_position, img_dict["Flyer"], self.player)
        self.flyers.append(new_flyer)

    self.walkers = []
    for position in to_generate["Walker"]:                                               #And then for all walkers
        true_position = [position[0] * 32, position[1] * 32]
        new_walker = Walker.Walker(true_position, img_dict["Walker"], self.player, level_bound)
        self.walkers.append(new_walker)

    ##CONSUMABLES/MODS
    self.consumables = []
    heal_amounts = [1,2,5]
    for x, heal in enumerate(["SHeal", "MHeal", "LHeal"]):                               #For each heal type
        for position in to_generate[heal]:                                               #For each instance of that type
            true_position = [position[0] * 32, position[1] * 32]                         #Find the in game position
            new_heal = C.Heal(true_position, img_dict[heal], self.player, heal_amounts[x])  #Instantiate a Heal object
            self.consumables.append(new_heal)                                            #And add to the list

    for upgrade in ["DJump", "WJump", "DStrike", "Run", "Dash"]:                         #Then repeat for each upgrade type
        for position in to_generate[upgrade]:
            true_position = [position[0] * 32, position[1] * 32]
            new_upgrade = C.Upgrade(true_position, img_dict[upgrade], self.player, upgrade)
            self.consumables.append(new_upgrade)

    self.timer = 6060                                                                    #100 second level with 1 grace second

#The driver code for the game controller
def run_frame(self):
    keys = pygame.key.get_pressed()                                                      #Determine what keys were pressed

    ##Pause logic
    if(keys[pygame.K_p]): self.pause_latch = True                                        #Perform the pause latch
```

```
    else:
        if(self.pause_latch):
            self.paused = not self.paused                                   #Changing state if necessary
        self.pause_latch = False

    self.draw_game()                                                        #Always draw the game
    if(self.paused): return                                                 #And end the frame early if paused

    ##Move
    movementKeys = (keys[pygame.K_UP] or keys[pygame.K_w],                  #Check the movement keys
                    keys[pygame.K_LEFT] or keys[pygame.K_a],
                    keys[pygame.K_RIGHT] or keys[pygame.K_d],
                    keys[pygame.K_LSHIFT] or keys[pygame.K_RSHIFT])
    self.movement(movementKeys)                                             #Then perform movement for all entities

    ##Level clear check
    if(self.flag.complete):                                                 #Check if the level flag was hit
        self.draw_game()                                                    #Make sure the player can see they hit the flag
        if(self.sent_from == "Menu"):                                       #If actually playing the game
            self.save_to_file(self.player.lives, self.player.health, level_shift = 1)   #Progress to the next level
            self.forced_draw()
            time.sleep(1)
        self.load_file(self.file, self.sent_from)                           #Then load the file again
        return                                                              #Ending the frame early

    ##Timer check
    self.timer -= 1                                                         #Decrement the timer
    if(self.timer <= 0):                                                    #If timer falls to 0
        self.player.lives -= 1                                              #Lose a life
        self.reset(False)                                                   #And reset
        return                                                              #Ending the frame early

    ##Consumable logic
    self.resolve_consumables()                                              #Perform the function

    ##Attack logic
    attacks = self.gather_attacks(keys[pygame.K_SPACE])                     #Gather all the attacks
    self.resolve_attacks(attacks)                                           #And then resolve them

    ##Hitbox logic
    to_destroy = []
    for x, hitbox in enumerate(self.hitboxes):                              #For each hitbox to draw
        hitbox.timer -= 1                                                   #Reduce its timer
        if(hitbox.timer < 0):                                               #Flagging hitboxes with a timer < 0
            to_destroy.insert(0, x)
        else:
            hitbox.draw(self.main_menu.screen)                              #And drawing the others
    for index in to_destroy:                                                #Then for each flagged hitbox
        this_hitbox = self.hitboxes.pop(index)                              #Remove the reference
        del this_hitbox                                                     #Then delete the object

    ##Update FSMs
    fsms = [f.fsm for f in self.flyers] + [w.fsm for w in self.walkers]     #Group the objects with FSM's
    for fsm in fsms: fsm.change_state()                                     #And update their state

    ##On_screen checks
    self.on_screen()                                                        #Finally, check which objects are on screen

#A function to draw the game
def draw_game(self):
    visible_tiles = [tile for tile in self.tiles if tile.on_screen]         #Determine which tiles are visisble
    entities = self.consumables + self.crawlers + self.flyers
    entities = entities + self.walkers + [self.player] + [self.flag]
    visible_entities = [entity for entity in entities if entity.on_screen]  #And which entities are visible

    for visible in visible_tiles:                                           #Draw the visible tiles
        visible.draw(self.main_menu.screen, self.scroll)

    for visible in visible_entities:                                        #Then for each visible entitiy
        try:
            if((visible.invulnerable // 5) % 2 == 1): continue              #Flash if invulnerable
        except: pass
        visible.draw(self.main_menu.screen, self.scroll)                    #Else draw

#A function to force the screen to update mid frame
def forced_draw(self):
    self.main_menu.screen.fill((50,50,50))                                  #Clear the screen
    self.draw_game()                                                        #Draw the game
    pygame.display.flip()                                                   #Then show it

#A function to run the movement function of each entity
def movement(self, player_keys: list):
    self.player.controller(player_keys, self.tiles, self.tile_colliders)
    for crawler in self.crawlers: crawler.logic(self.tile_colliders)
    for walker in self.walkers: walker.logic(self.grid, self.tiles, self.tile_colliders)
    for flyer in self.flyers: flyer.interpret_fsm(self.grid, self.graph_info)

#A function to determine what consumables have been touched by the player
def resolve_consumables(self):
    touched = []
    for x, consumable in enumerate(self.consumables):                       #For each consumable
        if(consumable.touch_check()):                                       #Flag the ones which were touched
            touched.insert(0, x)

    for deletion in touched:                                                #Then for each flagged object
        target = self.consumables.pop(deletion)                             #Remove any reference to the object
        del target                                                          #And delete the object

#A function to gather the attacks of all combatant objects
def gather_attacks(self, player_key: bool) -> list:
    attacks = []
    if(player_key): attacks = self.player.attack(attacks, facing = self.player.facing)  #If the player attacks, run the player attack
    for crawler in self.crawlers:                                           #For each crawler
        attacks = crawler.attack(attacks)                                   #Try to attack
    for flyer in self.flyers:                                               #For each flyer
        attacks = flyer.attack(attacks)                                     #Try to attack
    for walker in [w for w in self.walkers if w.fsm.current_state == "attacking"]:  #For each attacking walker
        attacks = walker.attack(attacks, facing = walker.facing)            #Try to attack
    return attacks

#A function to actually deal damage and remove dead objects
def resolve_attacks(self, attack_list: list):
    enemies = self.walkers + self.flyers + self.crawlers                    #Group the enemy type objects
    dead_list = []
    for packet in attack_list:                                              #Packet format: Hitbox, Attacker
        hitbox = packet[0]
        attacker = packet[1]
        damage = attacker.weapon_damage
        player_team = attacker.is_player_team
        draw_hitbox = attacker.draw_hitbox
```

```
        if(draw_hitbox):                                                    #If the hitbox is to be drawn
            position = (hitbox.left - self.scroll, hitbox.top)              #Determine its position
            image = pygame.Surface((hitbox.width, hitbox.height))           #Create a red square
            image.fill((255,0,0))
            new_hitbox = Sprite(position, image)                            #Instantiate a Sprite object
            new_hitbox.timer = 6                                            #Start a 6 frame counter
            self.hitboxes.append(new_hitbox)                                #And save to the hitbox list

        if(player_team): attackable = enemies                               #Prevent enemies hurting enemies
        else: attackable = [self.player]

        for opponent in attackable:                                         #For each opponent
            collider = pygame.Rect(opponent.position, opponent.size)        #Create their collider
            if(not pygame.Rect.colliderect(collider, hitbox)): continue     #Test for a collision

            is_dead = opponent.damaged(damage)                              #Deal damage
            if(is_dead): dead_list.append(opponent)                         #And if dead, flag

    for combatant in enemies + [self.player]:                               #For each combatant
        combatant.invulnerable = max(combatant.invulnerable - 1, 0)         #Decrement their invulnerable timer
        combatant.locked = max(combatant.locked - 1, 0)                     #And decrement their attack lock timer

    if(self.player.position[1] > 512): self.reset(True)                     #If the player is off_screen, reset
    if(self.player.health <= 0): self.reset(False)                          #If the player is dead, reset

    for dead in dead_list:                                                  #For each dead object, remove all references
        for x, flyer in enumerate(self.flyers):
            if(flyer != dead): continue
            del self.flyers[x]
        for x, walker in enumerate(self.walkers):
            if(walker != dead): continue
            del self.walkers[x]
        for x, crawler in enumerate(self.crawlers):
            if(crawler != dead): continue
            del self.crawlers[x]
        if(self.player == dead):
            self.reset(False)                                               #Don't delete the player, just reset
            break
        del dead                                                            #Then delete the object

#A function to update a save file
def save_to_file(self, lives: int, health: int, level_shift = 0):
    saves = open("save_data.txt", "r").read().split("\n")                   #Read the relevant file
    this_save = saves[self.file].split(", ")

    level = int(this_save[3]) + level_shift
    if(level > self.max_level):                                             #Stop the game from loading a
        print("Congratulations, you have beaten GAME GAME :D")              #non_existant level (the game has been beaten)
        level = int(this_save[3])

    alternate_save = [this_save[0], str(lives), str(health), str(level)]    #Group the updated save info
    saves[self.file] = ", ".join(alternate_save)

    f = open("save_data.txt", "w")                                          #Then save to the file
    f.write("\n".join(saves))
    f.close()

#A function to reset the level
def reset(self, screen_death: bool):
    self.forced_draw()                                                      #Force the game to draw
    def silly(shift: int):                                                  #Auxiliary function to just play an animation
        self.player.move(0,shift)
        self.forced_draw()
        time.sleep(0.01)

    self.running = False                                                    #Stop running the game
    if(screen_death):                                                       #Determine which type of death happened
        self.player.lives -= 1                                              #In off screen, the player loses a life
        self.player.position[1] = 480                                       #Reposition the player
        self.forced_draw()                                                  #Prevent smearing of the player
        toRise = 0
        toFall = 32
    else:
        toRise = 16
        toFall = (544 - self.player.position[1]) // 4

    time.sleep(1)                                                           #Pause to let player reflect
    for a in range(toRise):                                                 #Then play the animation
        silly(-2)
    for b in range(toFall):
        silly(4)
    time.sleep(1)

    if(self.sent_from == "Editor"):                                         #Reload the level if sent from the editor
        self.load_file(self.file, self.sent_from)
        return

    if(self.player.lives <= 0):                                             #Else check for a game over
        self.save_to_file(5, 10)                                            #Reseting the lives and health if so
        self.main_menu.change_screen("#HOME")                               #And sending back to the home screen
        return

    self.save_to_file(self.player.lives, 10)                                #Otherwise save and reset
    self.load_file(self.file, self.sent_from)

#A function to determine which objects are on screen
def on_screen(self):
    margin = [self.scroll + 128, self.scroll + 640]                         #Determine the margins of the screen
    left_conditions = self.player.facing == "left" and self.scroll > 0      #Determine if a left scroll is possible
    if(self.player.position[0] < margin[0] and left_conditions):            #If moving into the left margin
        self.scroll = max(0,self.player.position[0] - 128)                  #Scroll as far left as possible

    right_conditions = self.player.facing == "right" and self.scroll < self.camera_edge    #Determine if a right scroll is possible
    if(self.player.position[0] > margin[1] and right_conditions):           #If moving into the right margin
        self.scroll = min(self.camera_edge, self.player.position[0] - 640)  #Scroll as far right as possible

    screen_bounds = [self.scroll - 128, self.scroll + 896]                  #Determine the bounds of the screen
    objects = self.tiles + [self.flag] + [self.player]                      #Group all the objects
    objects = objects + self.crawlers + self.flyers + self.walkers
    for obj in objects:                                                     #For each object
        if(obj.position[0] < screen_bounds[0] or obj.position[0] > screen_bounds[1]):    #Check if on screen
            obj.on_screen = False
        else: obj.on_screen = True
```

3.2.3 EDITOR.py

```
###IMPORTS###
from Sprite import Sprite
import pygame
from Common_Functions import click_in_bounds
###Premade functions###
import glob
import csv

#A class to control what the editor is doing
class Editor_Controller:
    def __init__(self, menu_control):
        self.main_menu = menu_control

        border_img = pygame.image.load('Images/Border.png').convert_alpha()         #Load the border image
        focusA = Sprite((632,26), border_img)                                       #Instantiate sprites for the focuses
        focusB = Sprite((632,312), border_img)                                      #-These identify what tile and mode is selected
        self.focuses = [focusA, focusB]

        self.positions = {}
        self.img_dict = {}
        for filename in [image[7:] for image in glob.glob('Images/*')]:             #For each image file
            this_img = pygame.image.load(f"Images/{filename}").convert_alpha()      #Load the image
            self.img_dict[filename[:-4]] = this_img                                 #And store to the dictionary
        variants = ["R","U","L","D"]
        for i, var in enumerate(variants):                                          #Rotate the Oway image to produce the individual one-way platforms
            degrees = -90 + 90 * i                                                  #Determine the rotation amount
            rotated = pygame.transform.rotate(self.img_dict["Oway"], degrees)
            self.img_dict[f"{var}way"] = rotated

        self.active = False                                                         #Not active on initial load

    #A function to prepare a new csv file
    def new_file(self, name: str):
        temp_grid = [[""] * 240 for x in range(16)]                                 #Create a max_size blank grid
        with open(f"Levels/Custom/{name}.csv", "w", newline='') as newfile:         #Then write to the csv file
            writer = csv.writer(newfile)
            writer.writerows(temp_grid)

    #A function to initialise the variables
    def load(self, file, new_level = False):
        if(new_level):                                                              #Create a file for a new level
            self.new_file(file)
        self.file = file

        self.shift_focus((632,26), 0)                                               #Reset the tile and move
        self.shift_focus((632,312), 1)
        self.mode = "Draw"
        self.tile = "Floor"

        self.scroll = 0
        self.active = True
        self.player_pos = None                                                      #Must have 1 player
        self.flag_pos = None                                                        #and 1 flag
        self.previous_position = [-1,-1]
        self.grid = [[""] * 240 for x in range(16)]                                 #Create a grid for 10 screens of level <- for fast lookup
        with open(f"Levels/Custom/{self.file}.csv", "r") as lvlfile:                #Load the relevant file
            csv_reader = csv.reader(lvlfile)
            row_data = [row for row in csv_reader]
        for y, row in enumerate(row_data):                                          #Then add to the grid
            for x, tile in enumerate(row):
                self.grid[y][x] = tile
                if(tile == "Player"):                                               #Flagging the player and flag if they exist
                    self.player_pos = (x, y)
                elif(tile == "Flag"):
                    self.flag_pos = (x, y)

        self.objects = [[None] * 240 for x in range(16)]                            #Create an array to store the objects <- for drawing
        for y, row in enumerate(self.grid):                                         #Instantiate a Sprite for each tile
            for x, tile in enumerate(row):
                if(tile == "Start"): continue                                       #Ignore the air tiles
                if(tile == ""): continue
                new_pos = (x * 32, y * 32)                                          #Determine the actual position
                new_img = self.img_dict[tile]                                       #And the relevant image
                new_obj = Sprite(new_pos, new_img)                                  #Then instantiate
                if(new_pos[0] > 576): new_obj.on_screen = False                     #Flag as off_screen if off screen
                self.objects[y][x] = new_obj                                        #Store to the array

        self.hide_latch = False                                                     #A set of latches used for the keybinds
        self.hidden = False

        self.mode_latch = False
        self.erase_latch = False
        self.draw_latch = False

        self.tiles_latch = False
        self.entity_latch = False
        self.mods_latch = False

        self.left_latch = False
        self.right_latch = False

        self.palette = "Tiles"                                                      #Initialise the palette to Tiles
        self.p_pointer = 0

    #A function to save the file and change screen
    def unload(self, mode: list):
        self.active = False                                                         #Deactivate
        if(self.grid[0][0] == ""):                                                  #Ensure the first tile has a value (quirk of .csv format)
            self.grid[0][0] = "Start"

        #Trim empty columns (save space)
        empty_cols = [True] * 240
        for row in self.grid:                                                       #Loop through each tile to see which columns are empty
            for x, tile in enumerate(row):
                if(tile != ""):
                    empty_cols[x] = False
                    continue

        max_x = 24                                                                  #Ensure at least 1 screen exists
        for pos in range(239, 23, -1):                                              #Loop through each column (from the right)
            if(empty_cols[pos]): continue                                           #And find the first non_empty column
            max_x = pos + 1                                                         #Flag the first non_empty column
            break

        to_save = []
        for row in self.grid:
```

```
        to_save.append(row[:max_x])                                             #Slice each row to the size determined

    with open(f"Levels/Custom/{self.file}.csv", "w", newline='') as newfile:    #Then actually save
        writer = csv.writer(newfile)
        writer.writerows(to_save)

    if(mode[0] == "Save"):                                                      #If just saving:
        self.main_menu.change_screen("#LEVELS")                                 #Change the screen to the Level Select screen
    elif(self.player_pos is None or self.flag_pos is None):                     #Prevent testing a level without a player/flag
        print("You need a flag and a player")                                   #Feedback to the player the error
        self.active = True                                                      #Reactivate
    else:
        self.main_menu.change_screen(("#GAME",self.file), "Editor")             #Play the level, telling the GAME object to return to the editor

#A function to change the position of a focus
def shift_focus(self, position, focus):
    self.focuses[focus].position = position                                     #Just move the relevant focus
    self.previous_position = [-1,-1]                                            #And declare the last click as offscreen (used later)

#A function to change the mode of the editor
def change_mode(self, new_mode: str):
    self.mode = new_mode                                                        #Set the mode
    if(new_mode == "Erase"):                                                    #Then shift the focus accordingly
        self.shift_focus((704,312), 1)
    else:
        self.shift_focus((632,312), 1)
    self.previous_position = [-1,-1]                                            #And declare the last click as offscreen

#A function to scroll the editor
def change_scroll(self, amount: int):
    self.scroll += amount                                                       #Move
    self.scroll = max(0, self.scroll)                                           #Make sure we're not negative
    if(self.hidden):                                                            #If the palette is hidden:
        self.scroll = min(216, self.scroll)                                     #Can't scroll past 9 screens
        shift = 23                                                              #There are 24 tiles on screen
    else:
        self.scroll = min(221, self.scroll)                                     #Can't scroll past 9 screens and 5 tiles
        shift = 18                                                              #There are19 tiles on screen

    bounds = (self.scroll, self.scroll + shift)                                 #Determine what x positions are on screen
    for y in range(16):                                                         #Loop through each coordinate
        for x in range(240):
            obj = self.objects[y][x]
            if(obj is None): continue                                           #Ignoring empty tiles

            if(x < bounds[0] or x > bounds[1]):                                 #Flag off screen objects as off_screen
                obj.on_screen = False
            else:
                obj.on_screen = True                                            #And on screen objects as on_screen

#An interface function between the palette and the editor
def btn_clicked(self, button):
    action = button[0]
    match action:                                                               #Determine the button clicked and perform its action
        case "Draw": self.change_mode("Draw")
        case "Erase": self.change_mode("Erase")
        case "Left": self.change_scroll(-1)
        case "Right": self.change_scroll(1)
        case "LPalette": self.change_palette(-1)
        case "RPalette": self.change_palette(1)
        case _:                                                                 #In this case we are drawing a tile
            self.tile = action                                                  #So update the tile selected
            self.shift_focus(self.positions[action], 0)                         #And shift focus accordingly

#A function to check the existance of the player/flag when the grid changes
def revoke_core_objects(self, hovered_tile):
    if(hovered_tile == "Player"):                                               #The tile updated contained the player
        self.player_pos = None
    elif(hovered_tile == "Flag"):                                               #The tile updated contained the flag
        self.flag_pos = None

#The function to keep the grid up to date
def modify_grid(self):
    mouse_position = list(pygame.mouse.get_pos())                               #Deterime where the mouse is

    right_side = 607                                                            #Determine where the right side of the screen is
    if(self.hidden):                                                            #More screen available when the palette is hidden
        right_side = 767

    if(not click_in_bounds(mouse_position, [0, right_side, 0, 511])): return    #Check if the mouse is on_screen (ignores palette)
    if(click_in_bounds(mouse_position, [0, 127, 0, 63]) and not self.hidden):   #Ignore if the player is trying to save/play
        return

    current_position = [mouse_position[x] // 32 for x in range(2)]              #Convert mouse_position to grid indices
    current_position[0] += self.scroll                                          #And adjust to scroll
    if(current_position == self.previous_position): return                      #Only consider new clicks
    self.previous_position = current_position

    hovered_tile = self.grid[current_position[1]][current_position[0]]          #Then update the relevant tile
    self.revoke_core_objects(hovered_tile)                                      #Check if the flag/player was overwritten
    if(self.mode == "Erase"):                                                   #Erase if in erase mode
        self.grid[current_position[1]][current_position[0]] = ""
        self.objects[current_position[1]][current_position[0]] = None           #Garbage collection will delete an overwritten object
    else:                                                                       #Draw if in draw mode
        self.grid[current_position[1]][current_position[0]] = self.tile         #Set the grid to store the tile
        new_pos = (current_position[0] * 32, current_position[1] * 32)          #Determine the true position
        new_img = self.img_dict[self.tile]                                      #And image
        new_obj = Sprite(new_pos, new_img)                                      #To Instantiate a Sprite of the tile
        self.objects[current_position[1]][current_position[0]] = new_obj        #Then store to the object array
        if(self.tile == "Player"):                                              #Then flag the player
            if(self.player_pos is not None):                                    #Only allow one player at a time
                self.grid[self.player_pos[1]][self.player_pos[0]] = ""          #So erase a duplicate
                self.objects[self.player_pos[1]][self.player_pos[0]] = None
            self.player_pos = current_position                                  #Perform the flagging
        elif(self.tile == "Flag"):                                              #Repeat for the flag
            if(self.flag_pos is not None):
                self.grid[self.flag_pos[1]][self.flag_pos[0]] = ""
                self.objects[self.flag_pos[1]][self.flag_pos[0]] = None
            self.flag_pos = current_position

#A function to actually run the editor each frame
def run_frame(self, mouse_down):
    self.shortcuts()                                                            #Check if any shortcuts/keybinds were used
    if(mouse_down): self.modify_grid()                                          #Then update the grid if the mouse is down

    visible = []
    for row in self.objects:                                                    #Flag all the on_screen objects
        for obj in row:
            if(obj is None): continue
            if(not obj.on_screen): continue
```

```
                visible.append(obj)
        for vis in visible:                                             #Then draw the on_screen objects
            vis.draw(self.main_menu.screen, self.scroll * 32)

    #A function to check if the player is using any shortcuts
    def shortcuts(self):                                                #No logic, just latching
        keys = pygame.key.get_pressed()                                 #Get a list of all keys pressed this frame

        if(keys[pygame.K_h]): self.hide_latch = True                    #Hide/show the save and play buttons as well as the palette
        else:
            if(self.hide_latch): self.hide_buttons(not self.hidden)
            self.hide_latch = False

        if(keys[pygame.K_m]): self.mode_latch = True                    #Alternate the mode
        else:
            if(self.mode_latch):
                if(self.mode == "Draw"): self.change_mode("Erase")
                else: self.change_mode("Draw")
            self.mode_latch = False

        if(keys[pygame.K_e]): self.erase_latch = True                   #Set mode to Erase
        else:
            if(self.erase_latch): self.change_mode("Erase")
            self.erase_latch = False

        if(keys[pygame.K_d]): self.draw_latch = True                    #Set mode to Draw
        else:
            if(self.draw_latch): self.change_mode("Draw")
            self.draw_latch = False

        if(keys[pygame.K_1]): self.tiles_latch = True                   #Change palette to the Tiles palette
        else:
            if(self.tiles_latch): self.change_palette("Tiles")
            self.tiles_latch = False

        if(keys[pygame.K_2]): self.entity_latch = True                  #Change palette to the Entitites palette
        else:
            if(self.entity_latch): self.change_palette("Entities")
            self.entity_latch = False

        if(keys[pygame.K_3]): self.mods_latch = True                    #Change palette to the Mods palette
        else:
            if(self.mods_latch): self.change_palette("Mods")
            self.mods_latch = False

        if(keys[pygame.K_LEFT]): self.left_latch = True                 #Scroll left
        else:
            if(self.left_latch): self.change_scroll(-1)
            self.left_latch = False

        if(keys[pygame.K_RIGHT]): self.right_latch = True               #Scroll right
        else:
            if(self.right_latch): self.change_scroll(1)
            self.right_latch = False

    #A function to adjust the screen when hidden
    def hide_buttons(self, new_state):
        self.hidden = new_state                                         #Update the hide flag
        if(self.scroll == 221 and self.hidden):                         #If hiding and at the end of the file
            self.change_scroll(-5)                                      #Move forward 5 tiles
        elif(self.scroll == 216 and not self.hidden):                   #If unhiding and at the end of the file
            self.change_scroll(5)                                       #Move back 5 tiles
        else:
            self.change_scroll(0)                                       #Make sure that tiles behind the palette are revealed

    #A function to change the palette
    def change_palette(self, target):
        self.hide_buttons(False)                                        #Unhide the buttons
        palettes = ["Tiles", "Entities", "Mods"]                        #A list of palette options
        palette_primaries = ["Floor", "Player", "SHeal"]                #A list of the first tile in each palette
        if(type(target) is str):                                        #If directly updating (the target is known)
            self.palette = target                                       #Update the palette
            self.p_pointer = palettes.index(target)                     #And move the pointer
        else:                                                           #Else you're "scrolling" the palettes
            self.p_pointer += target                                    #Determine the next palette
            self.p_pointer = self.p_pointer % 3
            self.palette = palettes[self.p_pointer]                     #And update accordingly
        self.tile = palette_primaries[self.p_pointer]                   #Update the selected tile to reflect the palette
        self.shift_focus(self.positions[self.tile], 0)                  #Then shift focus accordingly
```

3.3 Assets

3.3.1 Sprite.py

```
#The base class of all visual elements
#stores the information needed to draw the object to the correct part of the screen
class Sprite:
    def __init__(self, position: tuple, image):
        self.position = list(position)                                  #Position refers to the top-left corner
        self.image = image
        self.size = (self.image.get_width(), self.image.get_height())  #Size should reflect the image size in pixels
        self.on_screen = True

    #Function to draw the Sprite onto a surface (typically the screen)
    def draw(self, surface, shift = 0):
        relative_position = [self.position[0] - shift, self.position[1]]    #Adjust the image position if necessary
        surface.blit(self.image, relative_position)                     #Then draw
```

3.3.2 Platforms.py

```
###IMPORTS###
from Sprite import Sprite
import pygame

#touched() in all cases returns whether a collision should block movement
#Other platform effects are covered in the main document

#The standard platform that just blocks movement
class Std_Platform(Sprite):                                     #All platforms are subclasses of the Sprite class
    def __init__(self, position: tuple, image):
        super().__init__(position, image)                       #Instantiate the superclass (see Sprite for details)

    def touched(self, *_) -> bool:                              #*_ allows for a polymorphic touched function, ignoring all parameters
        return True                                             #Always block

#The honey platform blocks VERTICAL movement
class Honey_Platform(Sprite):
    def __init__(self, position: tuple, image):
        super().__init__(position, image)                       #Instanatiate the superclass

    def touched(self, collider, direction_of_motion: str) -> bool:
        try: collider.is_sticky = True                          #Mark the object as sticky (if possible)
        except: pass
        return direction_of_motion in ["up","down"]             #Only block vertical motion

#The spike platform blocks all movement and deals damage
class Spike_Platform(Sprite):
    def __init__(self, position: tuple, image):
        super().__init__(position, image)                       #Instantiate the superclass

    def touched(self, collider, _) -> bool:                     #Ignore the direction of motion
        collider.damaged(1)                                     #Deal 1 damage
        return True

#The oway blocks entry from one direction
class One_Way_Platform(Sprite):
    def __init__(self, position: tuple, image, direction: str):
        super().__init__(position, image)                       #Instantiate the superclass
        self.direction = direction                              #Note: an up one_way has a down blocking direction
        match direction:                                        #Determine where the blocking edge is
            case "down":
                alt_position = position
                alt_size = (32, 1)
            case "up":
                alt_position = (position[0], position[1] + 31)
                alt_size = (32, 1)
            case "left":
                alt_position = (position[0] + 31, position[1])
                alt_size = (1, 32)
            case "right":
                alt_position = position
                alt_size = (1, 32)
        self.edge = pygame.Rect(alt_position, alt_size)         #Generate a collider for that edge

    def touched(self, collider, direction_of_motion: str) -> bool:
        if(self.direction != direction_of_motion): return False #If not moving in the blocked direction, then allow motion

        position = collider.position
        match self.direction:                                   #Generate a collider for the relevant edge on the colliding object
            case "up":
                alt_position = position
                alt_size = (32, 1)
            case "down":
                alt_position = (position[0], position[1] + 31)
                alt_size = (32, 1)
            case "right":
                alt_position = (position[0] + 31, position[1])
                alt_size = (1, 32)
            case "left":
                alt_position = position
                alt_size = (1, 32)
        col_edge = pygame.Rect(alt_position, alt_size)          #Generate the collider

        return self.edge.colliderect(col_edge)                  #Only block if the colliding object is ENTERING the platform

#A special platform, which doesn't block motion, but flags if the level is completed
class Flag(Sprite):
    def __init__(self, position: tuple, image):
        super().__init__(position, image)                       #Instantiate the superclass
        self.complete = False

    def touched(self, collider, _) -> bool:
        if(collider.is_player_team): self.complete = True       #Only mark as complete if collided with the player
        return False                                            #DON'T BLOCK MOTION
```

3.3.3 Combatant.py

```
###IMPORTS###
from Sprite import Sprite
import pygame

#A class used to represent any damageable object (player and enemies)
class Combatant(Sprite):                                                    #Combatant is a subclass of the Sprite class
    def __init__(self, position: tuple, image, start_health: int,
                 weapon_info: tuple, team: bool, draw_hitbox: bool):
        super().__init__(position, image)                                   #Instantiate the superclass (see Sprite for details)
        self.health = start_health

        self.weapon_damage = weapon_info[0]
        self.weapon_range = weapon_info[1]
        self.weapon_duration = weapon_info[2]
        self.locked = 0
        self.invulnerable = 0

        self.is_player_team = team
        self.draw_hitbox = draw_hitbox
        self.upgrade = None                                                 #Only used for the player

    #A function to determine if the object is attacking
    def attack(self, packet_list, facing = False, first_call = True) -> list:
        '''
```

```
        Check if the attack is possible
        Create the hitbox
        Create a packet
        Start the lock timer
        '''
        if(self.locked > 0 or not self.on_screen): return packet_list                   #Can't attack if invisible, or attacked recently

        if(not facing):                                                                 #Flyers and Crawlers are their own hitbox
            hitbox = pygame.Rect(self.position, self.size)                              #Generate a collider for the object
        else:                                                                           #Player and Walkers use a "weapon"
            if(facing == "left"):                                                       #Determine what direction the object is attacking
                left = self.position[0] - self.weapon_range
            else:
                left = self.position[0] + self.size[0]
            hitbox_position = (left, self.position[1])                                  #Generate a collider for the weapon
            hitbox_size = (self.weapon_range, self.size[1])
            hitbox = pygame.Rect(hitbox_position, hitbox_size)

        packet = (hitbox, self)
        packet_list.append(packet)                                                      #Add to the current list of attacks

        if(self.upgrade == "DStrike" and first_call):                                   #If the player has double strike, and this is the first strike
            if(facing == "left"):                                                       #strike again in the opposite direction
                packet_list = self.attack(packet_list, "right", first_call = False)
            else: packet_list = self.attack(packet_list, "left", first_call = False)

        self.locked = self.weapon_duration                                              #Prevent spam attacking

        return packet_list                                                              #Return the list of attacks

    #A function to determine whether the attack killed the combatant
    def damaged(self, damage_amount: int, piercing = False) -> bool:
        if(self.invulnerable > 0 and not piercing): return False                        #If in an invulnerable state

        self.health -= damage_amount

        if(self.health <= 0):                                                           #If health <= 0, you die
            return self.dead()                                                          #A function is used specifically for the player

        self.invulnerable = 60                                                          #Mark the object as invulnerable for 60 frames
        return False                                                                    #Mark the object as alive

    def dead(self) -> bool:
        return True

    #LOGIC-FREE MOVEMENT - JUST REPOSITION
    def move(self, x_shift: int, y_shift: int):
        self.position[0] += x_shift
        self.position[1] += y_shift
```

3.3.4 Grounded.py

```
###IMPORTS###
from Combatant import Combatant
import pygame

#A class which walks left to right
class Grounded(Combatant):                                                              #Grounded is a subclass of Combatant
    def __init__(self, position: tuple, image, start_health: int, weapon_info: tuple,
                 team: bool, has_hitbox: bool, max_speed: int, edge: int):
        super().__init__(position, image, start_health, weapon_info, team, has_hitbox)  #Instantiate the superclass (see Combatant for details)

        self.is_sticky = False
        self.facing = "right"
        self.speed = 0
        self.max_speed = max_speed

        self.jump_count = 0                                                             #0 = grounded, 16 = jump maxxed
        self.fall_count = 0
        self.coyote_frames = 8                                                          #8 = grounded
        self.double_jump = False
        self.jump_release = False

        self.dash_timer = 0

        self.screen_edge = edge - 32
        self.upgrade = None

    #A function which takes boolean inputs and causes the object to move accordingly
    def controller(self, inputs: tuple, platform_list: tuple, collider_list: tuple):
        self.vertical_logic(inputs[0], platform_list, collider_list)
        self.position[1] = max(self.position[1], 0)                                     #The object can't go off screen

        if(inputs[1] and not inputs[2]):                                                #Move horizontally if the left and right
            self.horizontal_logic("left", platform_list, collider_list)                 #inputs satisfy an XOR gate
        elif(inputs[2] and not inputs[1]):
            self.horizontal_logic("right", platform_list, collider_list)
        else: self.horizontal_acceleration("none")

        if(self.upgrade == "Dash"):                                                     #If the player has the dash upgrade
            if(inputs[3] and self.dash_timer <= 0):                                     #If a dash is desired and possible
                self.speed = self.max_speed                                             #Max out speed
                self.dash_timer = 15                                                    #Prevent dashing again for 15 frames
                for x in range(24):                                                     #Try to move 3 tiles in the current direction
                    self.horizontal_movement(self.facing, platform_list, collider_list)
            else: self.dash_timer -= 1                                                  #Decrement the timer

    #A function to incrementally move the objects forward
    def jiggle(self, direction: str, x_shift: int, y_shift: int, distance_to_move: int,
               platform_list: tuple, collider_list: tuple) -> bool:
        '''
        Move a little bit
        Check for collisions
        If collided, perform that platform's touched function
        Repeat until fully moved or collided
        '''

        displacement = 0
        collided = False
        while displacement < distance_to_move:                                          #While I still want to move
            displacement += 1
            self.move(x_shift,y_shift)                                                  #Move

            my_collider = pygame.Rect(self.position, self.size)                         #Generate my collider
            collision_index = my_collider.collidelist(collider_list)                    #Check if I collide with any platforms
```

```
            if(collision_index >= 0):
                if(platform_list[collision_index].touched(self, direction)):                #Check if the movement is blocked
                    collided = True                                                         #If so, undo move and stop moving further
                    self.move(-x_shift,-y_shift)
                    break
        self.position[0] = min(self.position[0], self.screen_edge)                          #Keep myself on screen
        self.position[0] = max(self.position[0], 0)

        return collided                                                                     #Return whether a collision occured

    #Polymorphically used in the player class - Here it is just a fall
    def vertical_logic(self, _, platform_list: tuple, collider_list: tuple):
        try:
            if(self.grounded): return                                                       #Do nothing if on a platform
        except: pass

        self.fall_count += 1                                                                #Else accelerate downwards
        speed = self.fall_count // 2
        self.grounded = self.jiggle("down", 0, 1, speed, platform_list, collider_list)      #Set the grounded flag when you land

    #A function to accelerate and move horizontally
    def horizontal_logic(self, direction: str, platform_list: tuple, collider_list: tuple):
        self.horizontal_acceleration(direction)
        self.horizontal_movement(self.facing, platform_list, collider_list)

    #A function to accelerate/decellerate horizontally
    def horizontal_acceleration(self, direction: str):
        if(self.facing == direction):                                                       #If moving in the previous direction
            self.speed = min(self.speed + 1, self.max_speed)                                #Get faster (up to a terminal velocity
        else:
            self.speed -= 1                                                                 #Else get slower

        if(self.speed < 0):                                                                 #If speed is negative, turn around
            if(direction != "none"): self.facing = direction
            self.speed = 0

    #A function to move the object horizontally
    def horizontal_movement(self, direction: str, platform_list: tuple, collider_list: tuple):
        match direction:                                                                    #Determine the direction of movement
            case "right": shift = 1
            case "left": shift = -1

        self.jiggle(direction, shift, 0, self.speed, platform_list, collider_list)          #Then call jiggle to move as far as possible
```

3.3.5 Player.py

```
###IMPORTS###
import Grounded
import pygame

#This is the class to represent the player's character
class Player(Grounded.Grounded):                                                            #This is a subclass of Grounded
    def __init__(self, position: tuple, image, start_lives: int, start_health: int, edge: int):
        super().__init__(position, image, start_health, (1,32,60), True, True, 4, edge)    #Instantiate the superclass (See Grounded for details)

        self.lives = start_lives

    #A function to determine how to move vertically each frame (overrides the superclass)
    def vertical_logic(self, jump_pressed: bool, platform_list: tuple, collider_list: tuple):
        '''
        Being sticky prevents vertical movement
        Jumping takes precendence over falling
        '''

        if(self.is_sticky):                                                                 #Check if you are on/under a honey tile
            self.coyote_frames = 0                                                          #Prevent coyote jumping out of honey
            honey_list = [pygame.Rect(plat.position, plat.size)                             #Create a simplified list of just honey tiles
                          for plat in platform_list
                          if str(type(plat))[18:23] == "Honey"]
            modded_position = (self.position[0], self.position[1] - 1)                      #Make the player temporarily 2 pixels taller
            modded_size = (self.size[0], self.size[1] + 2)
            big_collider = pygame.Rect(modded_position, modded_size)                        #Create a collider for the modified player
            collision_index = big_collider.collidelist(honey_list)                          #Test if the player collides with a honey tile
            if(collision_index < 0): self.is_sticky = False                                 #If not, then unflag sticky
            return                                                                          #Always end vertical movement if you were sticky
        if(self.jump_conditions(jump_pressed, collider_list)):                              #Check if you can/want to jump
            self.jump(platform_list, collider_list)                                         #-Jump if so
        else: self.fall(jump_pressed, platform_list, collider_list)                         #Else fall

    #A function to check if the player can jump and wants to jump
    def jump_conditions(self, jump_pressed: bool, collider_list: tuple) -> bool:
        '''
        Double Jump Conditions:
        1 - Double jump available
        2 - First jump complete
        '''
        d_jump_condition = self.upgrade == "DJump" and not self.double_jump                 #Check if a double jump is an option
        first_jump_complete = self.jump_release and self.jump_count == 16                   #Check that the first jump is done
        if(d_jump_condition and first_jump_complete):                                       #If both are the case
            if(jump_pressed):                                                               #AND you want to jump
                self.jump_count = 0                                                         #Perform a double jump by resetting the jump_count
                self.double_jump = True                                                     #Then flag the double jump
                return True
            return False

        '''
        Wall Jump Conditions:
        1 - Touching a wall
        2 - First jump complete
        '''
        if(self.upgrade == "WJump" and first_jump_complete):                                #Check if wall jumping is an option
            left_position = [self.position[0] - 1, self.position[1]]                        #Determine the pixels directly to the left
            right_position = [self.position[0] + 32, self.position[1]]                      #And right of the player
            modded_size = [1, self.size[1]]
            left_collider = pygame.Rect(left_position, modded_size)                         #Create a collider for each
            right_collider = pygame.Rect(right_position, modded_size)
            left_collision = left_collider.collidelist(collider_list) >= 0                  #Test for a collision on each (i.e. there is a wall)
            right_collision = right_collider.collidelist(collider_list) >= 0

            condition = left_collision or right_collision
            if(condition and jump_pressed):                                                 #If there is a wall and you want to jump
                self.jump_count = 4                                                         #A wall jump is slightly weaker
                self.fall_count = 0

                                                                                            #XOR the collision conditions to:
```

```
            if(left_collision and not right_collision):                         #Bounce right off of a left wall
                self.facing = "right"
                self.speed = 4
            elif(right_collision and not left_collision):                       #Bounce left off of a right wall
                self.facing = "left"
                self.speed = 4

            return True                                                         #If XOR fails then we are between two walls


        try_jumping = jump_pressed or (self.jump_count > 0 and self.jump_count < 8)     #A jump must last at least 8 frames
                                                                                        #Check if the player pressed jump or a jump is in progress
        return try_jumping and self.coyote_frames > 0 and self.jump_count < 16          #Then jump if you want to AND coyote frames remain
                                                                                        #AND you've not reached the top of a jump
    #A function to actually perform a jump
    def jump(self, platform_list: tuple, collider_list: tuple):
        self.jump_release = False                                               #A flag to say that the jump key was/wasn't released

        self.coyote_frames = 1                                                  #Prevent double_jumping by quickly tapping
        collided = self.jiggle("up", 0, -1, 8, platform_list, collider_list)    #Then jiggle upwards into place

        if(collided): self.jump_count = 16                                      #Colliding is the same as reaching the top of a jump
        else: self.jump_count += 1                                              #Else just increment the timer

    #A function to accelerate the player downwards
    def fall(self, jump_pressed: bool, platform_list: tuple, collider_list):
        self.fall_count += 1                                                    #Just a timer
        speed = self.fall_count // 2                                            #V = U + AT => U = 0, A = 1/2 => V = T/2 (Kinematics)
        grounded = self.jiggle("down", 0, 1, speed, platform_list, collider_list)   #Jiggle downwards into place

        if(not jump_pressed):                                                   #Check if the jump key was released
            self.jump_release = True
        if(grounded):
            if(self.jump_release):                                              #Only allow jump when the jump key is released
                self.jump_count = 0
                self.jump_release = False
            self.fall_count = 0                                                 #Reset the other variables
            self.coyote_frames = 8
            self.double_jump = False
        else:
            self.coyote_frames -= 1                                             #Reduce coyote_frames
            if(self.coyote_frames <= 0): self.jump_count = 16                   #If coyote jump is no longer an option then max out jump

    #A function to decrement the life counter rather than delete the player (override the superclass)
    def dead(self):
        if(self.health >= 0): self.lives -= 1
        return True
```

3.3.6 Walker.py

```
###IMPORTS###
import Grounded
from Finite_State_Machine import Walker_FSM
import random

#This class is for an enemy which walks left to right on a platform
class Walker(Grounded.Grounded):                                                #Walker is a subclass of Grounded
    def __init__(self, position: tuple, image, player_object, edge: int):
        health, damage, attack_time, speed = self.mutations()                   #Randomise the stats

        weapon = (damage, 32, attack_time)                                      #Grouping weapon info just to make the next line shorter
        super().__init__(position, image, health, weapon, False, True, speed, edge)     #Instantiate the superclass (see Grounded for details)

        self.fsm = Walker_FSM(self, player_object)                              #Instantiate an FSM

    #A function to randomise the stats of the class
    def mutations(self):
        health = random.randint(2,4)                                            #Pick a random number from 2 -> 4

        weapon_roll = random.randint(0,7)                                       #Pick a random number from 0 -> 7
        damage, attack_time = 1, 15                                             #The basic stats
        if(weapon_roll < 2):                                                    #On a low roll (0, 1), attack slower
            attack_time = 30
        elif(weapon_roll >= 4 and weapon_roll < 6):                             #On a good roll (4, 5), do more damage or attack faster
            if(weapon_roll % 2 == 0): damage = 2
            else: attack_time = 10
        elif(weapon_roll >= 6):                                                 #On an exceptional roll (6, 7), do more damage and attack faster
            damage = 2
            attack_time = 10

        speed_roll = random.randint(0,3)                                        #Pick a random number from 0 -> 3
        if(speed_roll == 0): speed = 1                                          #On a bad roll (0), move slowly
        elif(speed_roll == 3): speed = 4                                        #On a good roll (3), move quickly
        else: speed = 2                                                         #Else move normal speed

        return health, damage, attack_time, speed

    #A function to determine how the Walker should move
    def logic(self, grid, platform_list, collider_list):
        move_inputs = self.interpret_fsm(grid)                                  #Reads the state of the FSM to determine how to move
        self.controller(move_inputs, platform_list, collider_list)              #Then perform the move (method of the superclass)

    #A function to read/interpret the FSM
    def interpret_fsm(self, grid) -> tuple:
        state = self.fsm.current_state

        player_visible = self.is_player_visible(grid)                           #Determine if the player is visible
        if(state == "attacking" and player_visible): return (False,False,False,False)   #Don't move if in combat with the player
        return [False] + self.determine_move(player_visible, grid) + [False]    #Else determine how to move

    #Check if the player is in-line with the walker and not behind a wall
    def is_player_visible(self, grid) -> bool:
        x_index = self.position[0] // 32                                        #Determine the grid position
        y_index = self.position[1] // 32

        player_position = self.fsm.target.position                              #Detemrine the player's position
        player_indices = (player_position[0] // 32, player_position[1] // 32)    #And where that is on the grid
        if(player_indices[1] != y_index): return False                          #Check if the two objects are in-line

        if(self.facing == "left"):                                              #Determine:
            if(player_indices[0] > x_index): return False                       #If the player is even in this direction
            shift = -1                                                          #How to traverse the grid
            bound = -1                                                          #When to stop
            block_list = ["Spike","Floor","Rway"]                               #And what counts as a wall
        else:
```

```
            x_index -= 1                                                    #Make sure we're considering the left side of the walker
            if(player_indices[0] < x_index): return False
            shift = 1
            bound = len(grid[0])
            block_list = ["Spike","Floor","Lway"]

        for this_x in range(x_index, bound, shift):                         #Step across the grid in the determine direction
            if(grid[y_index][this_x] in block_list): return False           #If you reach a wall, then the player is hiden
        return False                                                        #The search failed otherwise

    #A function to determine how to move
    def determine_move(self, player_visible: bool, grid) -> list:
        y_index = self.position[1] // 32                                    #Find what row the walker is on

        match self.facing:
            case "left":                                                    #Determine:
                vector = -1                                                 #How to move
                x_index = ((self.position[0] - 2) // 32) + 1                #Which side (left or right) to consider <- this case is left
                block_list = ["Spike","Floor","Rway"]                       #What stops movement
                new_face = "right"                                          #And how to turn around
            case "right":
                vector = 1
                x_index = (self.position[0] + 2) // 32
                block_list = ["Spike","Floor","Lway"]
                new_face = "left"

        state = "normal"
        next_x = x_index + vector                                           #Determine the tiles in front
        unwalkable = ["","Uway","Spike"]
        if(next_x < 0 or next_x >= len(grid[0])): state = "turn"            #Turn if you reach the screen edge
        elif(grid[y_index][next_x] in block_list): state = "turn"           #Turn if there is something in the way
        elif(grid[y_index + 1][next_x] in unwalkable): state = "turn"       #Turn if you reach a platform's edge

        moves = [False, False]
        if(state == "normal" or player_visible):                            #Only turn if the state is turn and the player isn't visible
            moves[(vector + 1) // 2] = True                                 #Covert the vector to an index
        elif(state == "turn"):
            self.facing = new_face

        return moves                                                        #Tell the logic method how to move
```

3.3.7 Flyer.py

```
###imports###
from Combatant import Combatant
from Finite_State_Machine import Flyer_FSM
import AStar
import random

#An enemy which flies around the screen, targeting the player
class Flyer(Combatant):                                                     #The flyer is a subclass of the Combatant class
    def __init__(self, position: tuple, image, player_object):
        health, damage = self.mutations()                                   #Randomise the flyer's stats

        super().__init__(position, image, health, (damage,4,1), False, False)   #Instantiate the superclass (see Combatant for details)

        self.fsm = Flyer_FSM(self, player_object)                           #Instantiate a finite state machine for the flyer
        self.move_timer = 0
        self.target_position = [0,0]

    #A function to randomise the flyer's stats
    def mutations(self):
        health_roll = random.randint(0,3)                                   #Randomly choose a number from 0 to 3
        if(health_roll == 3): health = 2                                    #Good roll = double health
        else: health = 1                                                    #Else, normal health

        damage_roll = random.randint(0,3)                                   #Randomly choose a number from 0 to 3
        if(damage_roll == 3): damage = 2                                    #Good roll = double damage
        else: damage = 1                                                    #Else, normal damage
        return health, damage                                               #Return the randomised stats

    #A function to choose an action based on the finite state machine
    def interpret_fsm(self, grid, graph):
        state = self.fsm.current_state
        if(state == "idle"): return                                         #Don't do anything if idle
        if(state != "moving"):                                              #If there is not a path found
            next_indices = self.pathfind(grid, graph)                       #Find the next indices on the path
            self.target_position = [32 * index for index in next_indices]   #Convert to a position
            self.move_timer = 16                                            #Only pathfind after 16 frames
        self.move_timer -= 1                                                #Reduce timer

        horiz_direction = self.target_position[0] - self.position[0]        #Determine where the target is horizontally
        if(horiz_direction == 0): x_shift = 0                               #Then move accordingly
        elif(horiz_direction > 0): x_shift = 2
        else: x_shift = -2

        vert_direction = self.target_position[1] - self.position[1]         #Determine where the target is vertically
        if(vert_direction == 0): y_shift = 0                                #Then move accordingly
        elif(vert_direction > 0): y_shift = 2
        else: y_shift = -2
        self.move(x_shift, y_shift)                                         #Then actually move

    #A function to find the closest position on the path to the player
    def pathfind(self, grid, graph) -> tuple:
        my_indices = (self.position[0] // 32, self.position[1] // 32)       #Determine the indices of the flyer and the player
        player_indices = (self.fsm.target.position[0] // 32,
                          self.fsm.target.position[1] // 32)

        result = AStar.a_star(grid, my_indices, player_indices, *graph)     #Use the AStar algorithm to find the shortest path

        if(type(result) is str):                                            #If "Fail" was returned, don't move
            result = my_indices

        return result                                                       #Return the indices of the position to target
```

3.3.8 Crawler.py

```
###IMPORTS###
from Combatant import Combatant
import pygame
import random

#A class for an enemy which crawls around a platform
class Crawler(Combatant):                                                   #The crawler is a subclass of the Combatant
    def __init__(self, position: tuple, image, grid):
        #Auxiliary function to make sure the neighbouring tile is real
        def validate(indices, shifts, grid, walls):
            modded_indices = [indices[x] + shifts[x] for x in range(2)]     #Determine the position of the neighbour

            if(modded_indices[0] < 0): return False                         #Guard clauses to reject non existant tiles
            if(modded_indices[1] < 0): return False
            if(modded_indices[0] >= len(grid[0])): return False
            if(modded_indices[1] > 15): return False

            tile = grid[modded_indices[1]][modded_indices[0]]               #If the tile is real
            return tile in walls                                            #Only valid if the tile is a platform

        health, damage, attack_time = self.mutations()                      #Mutate the crawler to make it unique

        super().__init__(position, image, 2, (1,4,15), False, False)        #Instantiate the superclass (see Combatant for details)

        my_indices = (position[0]//32, position[1]//32)                      #Convert position to indicies
        walls = ["Floor", "Honey", "Spike", "Uway", "Dway", "Rway", "Lway"] #The solid objects
        self.direction = "up"
        options = 0
        if(validate(my_indices, (-1,0), grid, walls)):                      #For each direction, mark if the crawler could spawn there
            self.direction = "up"                                           #i.e. has something to stick to
            options += 1
        if(validate(my_indices, (1,0), grid, walls)):
            self.direction = "down"
            options += 1
        if(validate(my_indices, (0,-1), grid, walls)):
            self.direction = "right"
            options += 1
        if(validate(my_indices, (0,1), grid, walls)):
            self.direction = "left"
            options += 1

        if(options == 0 or options == 4): self.destroy_flag = True          #Destroy if there's nothing to stick to, or fully trapped
        self.set_sticking_point()                                           #Determine the sticking direction

    #A function to randomise the stats of the crawler
    def mutations(self):
        health = random.randint(2,4)                                        #Health is 2 -> 4

        attack_roll = random.randint(0,7)                                   #Choose a random number from 0 to 7
        damage, attack_time = 1, 15                                         #Basic stats
        if(attack_roll < 2):                                                #Low roll (<2)
            attack_time = 30                                                #slow attacker
        elif(attack_roll >= 4 and attack_roll < 6):                         #Above average roll (4, 5)
            if(attack_roll % 2 == 0): damage = 2                            #either give a damage boost
            else: attack_time = 10                                          #or attack faster
        elif(attack_roll >= 6):                                             #Exceptional roll (6, 7)
            damage = 2                                                      #High damage and fast attacker
            attack_time = 10

        speed_roll = random.randint(0,3)                                    #Choose another random number from 0 to 3
        if(speed_roll == 0): self.speed = 1                                 #Bad roll = half speed
        elif(speed_roll == 3): self.speed = 4                               #Good roll = double speed
        else: self.speed = 2                                                #Average roll = normal speed

        return health, damage, attack_time                                  #Return the randomised stats

    #A function to determine the "sticking point" of the crawler
    def set_sticking_point(self):
        match self.direction:                                               #The sticking point is 90 degrees clockwise from the direction of motion
            case "up": self.sticking_point = "left"
            case "left": self.sticking_point = "down"
            case "down": self.sticking_point = "right"
            case "right": self.sticking_point = "up"

    #A function to rotate the crawler's direction by 90 degrees
    def rotate(self, clockwise: str):
        salted_direction = self.direction + clockwise                       #Clockwise and anticlockwise rotations are different
        match salted_direction:                                             #Just rotate 90 degrees in the indicated direction
            case "upc": self.direction = "right"
            case "upa": self.direction = "left"
            case "leftc": self.direction = "up"
            case "lefta": self.direction = "down"
            case "downc": self.direction = "left"
            case "downa": self.direction = "right"
            case "rightc": self.direction = "down"
            case "righta": self.direction = "up"

        self.set_sticking_point()                                           #Then update the sticking point

   #A function to determine if there is a platform in the indicated direction
    def shudder(self, shudder_direction: str, collider_list) -> bool:
        modded_position = [self.position[0], self.position[1]]              #Create a copy opf the current position to not have to undo a shift

        match shudder_direction:                                            #Shift the position in the indicated direction
            case "up": modded_position[1] -= self.speed
            case "down": modded_position[1] += self.speed
            case "left": modded_position[0] -= self.speed
            case "right": modded_position[0] += self.speed

        my_collider = pygame.Rect(modded_position, self.size)               #Generate a collider of where the crawler would be if it moved this way

        return my_collider.collidelist(collider_list) != -1                 #Return True if there is something in the way

    #The main function to determine what the crawler should do
    def logic(self, collider_list):
        if(self.shudder(self.direction, collider_list)):                    #If the next tile is blocked
            self.rotate("c")                                                #rotate clockwise
            return

        self.controller()                                                   #Move

        if(not self.shudder(self.sticking_point, collider_list)):           #If the crawler is no longer connected to a platform
            self.rotate("a")                                                #rotate anticlockwise

    #A function to logically move the crawler
    def controller(self):
```

```
        x_shift = y_shift = 0

        match self.direction:                                       #Determine what movement is necessary
            case "up": y_shift = -self.speed
            case "down": y_shift = self.speed
            case "left": x_shift = -self.speed
            case "right": x_shift = self.speed

        self.move(x_shift, y_shift)                                 #Then move
```

3.3.9 Finite_State_Machine.py

```
###IMPORTS###
from Common_Functions import pythagorus

#THESE CLASSES JUST TELL THE ENEMY WHAT THEY SHOULD BE DOING GENERALLY, NOT HOW TO DO IT

#A class storing the finite state machine for a walker enemy
class Walker_FSM:
    def __init__(self, owner, target):
        self.current_state = "idle"
        self.owner = owner
        self.target = target

    #See Finite State Machine Transition Diagram
    #Function to determine (and perform) if a state change is necessary
    def change_state(self):
        match self.current_state:
            case "idle":                                                                        #The do nothing state
                if(self.owner.on_screen):                                                       #Move when visible
                    self.current_state = "moving"

            case "moving":                                                                      #The just move state
                if(pythagorus(self.owner.position, self.target.position) <= self.owner.weapon_range):   #Attack when you can hit
                    self.current_state = "attacking"

                elif(not self.owner.on_screen):                                                 #Stay still when you're off screen
                    self.current_state = "idle"

            case "attacking":                                                                   #The attack the player state
                if(pythagorus(self.owner.position, self.target.position) > self.owner.weapon_range):    #Move when you can't hit
                    self.current_state = "moving"


#A class storing the finite state machine for a flyer enemy
class Flyer_FSM:
    def __init__(self, owner, target):
        self.current_state = "idle"
        self.owner = owner
        self.target = target

    #See Finite State Machine Transition Diagram
    def change_state(self):
        match self.current_state:
            case "idle":                                                                        #The do nothing state
                if(self.owner.on_screen):                                                       #If visible, "turn on"
                    self.current_state = "active"

            case "active":                                                                      #The pathfinding state
                if(not self.owner.on_screen):                                                   #If off screen, deactivate
                    self.current_state = "idle"

                elif(self.owner.move_timer > 0):                                                #If path found, don't waste
                    self.current_state = "moving"                                               #resources pathfinding

            case "moving":                                                                      #The path found, just move state
                if(self.owner.move_timer == 0):                                                 #If destination has been reached
                    self.current_state = "active"                                               #pathfind again
```

3.3.10 Consumables.py

```
###IMPORTS###
from Sprite import Sprite
import pygame

#A class used to store a consumable entity in the game
class Consumable(Sprite):                                           #Consumable is a subclass of the Sprite class
    def __init__(self, position: tuple, image, player):
        super().__init__(position, image)                           #Instantiate the superclass (see Sprite for details)
        self.player = player
        self.collider = pygame.Rect(self.position, self.size)       #Create a collider for the consumable

    #Function to test if the player has touched the consumable
    def touch_check(self) -> bool:
        player_collider = pygame.Rect(self.player.position, self.player.size)   #Create the player collider
        if(not self.collider.colliderect(player_collider)): return False        #Test for collision

        self.use()                                                  #Perform use if collided
        return True                                                 #Then flag for deletion

#A class used to store the method for an upgrade consumable
class Upgrade(Consumable):                                          #Upgrade is a subclass of the Consumable class
    def __init__(self, position: tuple, image, player, upgrade: str):
        super().__init__(position, image, player)                   #Instantiate the superclass
        self.upgrade = upgrade

    #A function to "use" the consumable
    def use(self):
        if(self.player.upgrade == "Run"):                           #If the player has the run upgrade
            self.player.max_speed //= 2                             #half the player's speed
        if(self.upgrade == "Run"):                                  #If this is a run upgrade
            self.player.max_speed *= 2                              #Double the player's speed

        self.player.upgrade = self.upgrade                          #Then just set the upgrade variable

#A class used to store the method for a healing consumable
class Heal(Consumable):                                             #Heal is a subclass of the Consumable class
    def __init__(self, position: tuple, image, player, amount: str):
```

```
        super().__init__(position, image, player)                           #Instantiate the superclass
        self.amount = -amount                                               #Healing is just reverse damage

    #A function to "use" the consumable
    def use(self):
        self.player.damaged(self.amount, piercing = True)                   #Healing can happen if the player is invulnerable
```

3.4 UI

3.4.1 Button.py

```
###IMPORTS###
from Sprite import Sprite
from Common_Functions import click_in_bounds

#An object which runs a function when clicked
class Button(Sprite):                                                       #Button is a subclass of Sprite
    def __init__(self, position: tuple, image, action, *args):              #*args collates all final arguments into a list
        super().__init__(position, image)                                   #Instantiate the superclass (see Sprite for details)
        self.action = action                                                #The function and parameters to be ran when clicked
        self.args = args
        self.active = False                                                 #A variable acting as the latch

    #Validate if a click starts and ends within the button
    def clicked(self, mouse_down: bool, mouse_position):
        bounds = (self.position[0], self.position[0] + self.size[0],        #The button's bounds
                  self.position[1], self.position[1] + self.size[1])
        if(not click_in_bounds(mouse_position, bounds)):                    #If not clicking the button
            self.active = False                                             #Unlatch and end
            return

        if(self.active and not mouse_down):                                 #If latched AND releasing the mouse
            self.action(self.args)                                          #Run the function
            self.active = False                                             #Unlatch
        if(mouse_down): self.active = True                                  #If pressing down, latch
```

3.4.2 Entry.py

```
###IMPORTS###
import pygame

#An object which stores a label and a button, used as an entry of a listbox
class Entry:
    def __init__(self, position: list, label, button, is_active: bool):
        self.position = position                                            #Determine and set the dimensions of the object
        image_width = label.size[0] + button.size[0]
        image_height = max(label.size[1], button.size[1])
        self.image = pygame.Surface((image_width, image_height))            #Generate a background for the label and button

        label.position = [0,0]                                              #Reposition the label and button to be
        button.position = [label.size[0],0]                                 #relative to the entry
        label.draw(self.image)                                              #Then draw onto the background
        button.draw(self.image)

        del label                                                           #Destroy the label to save memory
        self.button = button                                                #Keep track of the button
        self.is_active = is_active                                          #Stores whether the entry is currently ON the listbox

    #A function to draw the entry to the screen
    def draw(self, surface):
        surface.blit(self.image, self.position)

    #A function to check if the label's button was clicked
    def clicked(self, mouse_down: bool, mouse_position):
        relative_position = (mouse_position[0] - self.position[0],          #Reposition the click position to be relative
             mouse_position[1] - self.position[1])                          #to the entry (like the button is)
        self.button.clicked(mouse_down, relative_position)                  #Then let the button perform the click checks
```

3.4.3 Listbox.py

```
###IMPORTS###
import pygame

#An object which acts as a container of Entry objects
class Listbox:
    def __init__(self, position: tuple, entry_list: tuple, scrollbar):
        #Reposition all the entries
        self.position = list(position)                                              #The listbox can change position, so make sure it is mutable
        self.entries = entry_list
        total_height = 0
        self.entry_height = entry_list[0].image.get_height()                        #Determine the height of an entry
        for entry in entry_list:                                                    #Then for each entry
            entry.position = [0, total_height]                                      #Reposition to be relative to the listbox
            total_height += self.entry_height                                       #then update the current height of the listbox

        #Draw onto one image
        this_width = entry_list[0].image.get_width()                                #Determine the width of an entry
        self.image = pygame.Surface((this_width, total_height))                     #Generate a big background
        for entry in entry_list:                                                    #Then draw all the entries onto it
            entry.draw(self.image)

        self.shift = 0
        self.shift_bound = len(self.entries)-3                                      #Can scroll up to number of entries - 3 time
        surface_height = self.entry_height * 3
        self.to_draw = pygame.Surface((this_width, surface_height))                 #Generate a new image which is actually to be drawn
        self.to_draw.fill((50,50,50))                                               #Fill with a grey background
        self.shift_and_render(0)                                                    #Perform the function to put it together

        self.scrollbar = scrollbar
        if(scrollbar is not None): self.scrollbar.make_relative(self)               #If a scrollbar is present, move it next to the listbox

    #A function which shifts the listbox up and down, and then renders the new image
```

```
    def shift_and_render(self, shift_amount: int):
        self.shift += shift_amount
        self.shift = min(self.shift, self.shift_bound)                    #Keep in bounds
        self.shift = max(self.shift, 0)

        self.to_draw.blit(self.image, (0, -self.entry_height * self.shift)) #Draw the active section of the big image onto the little image

        for x, entry in enumerate(self.entries):                          #Then active/deactivate the entries
            if(x < self.shift): entry.is_active = False                   #to make sure only displayed entries are active
            elif(x > self.shift + 2): entry.is_active = False
            else: entry.is_active = True

    #A function to draw the listbox onto a surface
    def draw(self, surface):
        surface.blit(self.to_draw, self.position)                         #Draw the active listbox (little image)
        if(self.scrollbar is not None): self.scrollbar.draw(surface)      #Draw the scrollbar if it exists

    #A function to perform the clicked check for each of the contained objects
    def clicked(self, mouse_down: bool, mouse_position):
        '''
        The "active" listbox moves, whereas the scrollbar remains the same
        -Thus different mouse position lists are necessary
        '''
        effective_top = (self.position[1] - self.entry_height * self.shift) #Determine where the top of the active listbox is
        mpA = [mouse_position[0] - self.position[0],                      #Make the mouse position relative to the active listbox
               mouse_position[1] - effective_top]                         #-for entries
        mpB = [mouse_position[0] - self.position[0],                      #Make the mouse position relative to the listbox
               mouse_position[1] - self.position[1]]                      #-for the scrollbar

        for entry in self.entries:                                        #Then for each ACTIVE entry, perform the clicked check
            if(entry.is_active): entry.clicked(mouse_down, mpA)
        if(self.scrollbar is not None):
            self.scrollbar.clicked(mouse_down, mpB)                       #Perform the scrollbar clicked check if it exists
```

3.4.4 Scrollbar.py

```
###IMPORTS###
from Sprite import Sprite
from Common_Functions import click_in_bounds

#An object which is used to scroll a listbox
class Scrollbar(Sprite):                                                  #Scrollbar is a subclass of Sprite
    def __init__(self, position: tuple, image):
        super().__init__(position, image)                                 #Instantiate the superclass (see Sprite for details)
        self.click_position = [0,0]
        self.owner = None
        self.active = False

    #A function to reposition the scrollbar next to a listbox
    def make_relative(self, new_owner):
        relative_x = new_owner.position[0] + new_owner.to_draw.get_width()  #Scrollbar goes on the right side of the listbox
        self.position = [relative_x, new_owner.position[1]]               #Set the position relative to the listbox
        self.click_position = [new_owner.to_draw.get_width(), 0]

        self.owner = new_owner

    #Validate if a click starts and ends within the button
    def clicked(self, mouse_down: bool, mouse_position):
        bounds = (self.click_position[0], self.click_position[0] + self.size[0],  #The bounds of the scrollbar
                  self.click_position[1], self.click_position[1] + self.size[1])
        if(not click_in_bounds(mouse_position, bounds)):                  #If not in bounds, unlatch and end
            self.active = False
            return

        if(self.active and not mouse_down):                               #If latched and releasing the mouse
            if(mouse_position[1] < 14): self.owner.shift_and_render(-1)   #Shift 1 up if the top was clicked
            elif(mouse_position[1] > 178): self.owner.shift_and_render(1) #Shift 1 down if the bottom was clicked
            else:
                jump_to = ((mouse_position[1] - 14) * (self.owner.shift_bound + 1)) // 164  #Else determine what percentage of the
                                                                          #scrollbar the click was
                to_shift = jump_to - self.owner.shift                     #Then jump that percentage of the way into the listbox
                self.owner.shift_and_render(to_shift)
        if(mouse_down): self.active = True                                #Set the latch
```

3.4.5 Textbox.py

```
###IMPORTS###
from Sprite import Sprite
from Common_Functions import text_to_image, click_in_bounds
import pygame
###Premade lists of legal characters
from string import ascii_lowercase as LOW                                 #A list of lowercase letters (a -> z)
from string import digits as DIG                                          #A list of digits (0 -> 9)

#An object which allows the user to enter text based data into the program
class Textbox(Sprite):                                                    #Textbox is a subclass of Sprite
    def __init__(self, position: list, size: tuple, fg: tuple, active_bg: tuple,
                 inactive_bg: tuple, base_message: str, max_length: int):
        super().__init__(position, pygame.Surface(size))                  #Instantiate the superclass (see Sprite for details)
        self.base_message = base_message
        self.max_length = max_length
        self.current_message = ""
        self.font = pygame.font.SysFont("Arial", 54)                      #Load the Arial 54 font
        self.fg = fg
        self.active_bg = active_bg
        self.inactive_bg = inactive_bg

        self.active = False
        self.using = False

        self.generate_image()                                             #Generate the initial image

        self.caps = False
        self.del_timer = 0
        self.valid_keys = tuple([char for char in LOW + DIG] + ["space"])  #The player can only enter letters, digits or spaces

    #Auxiliary function for the main loop to read the textbox
    def get_message(self) -> str:
        return self.current_message
```

```
    #Auxiliary function to override the message
    def update(self, message: str) -> str:
        self.current_message = ""
        self.base_message = message


    #Function which updates the image to reflect the current message and state of the textbox
    def generate_image(self):
        message = self.current_message
        if(message == ""): message = self.base_message                          #If empty, use the base message

        if(self.using):                                                         #Select a background cover depending on the current state
            bg = self.active_bg
        else:
            bg = self.inactive_bg

        text_image = text_to_image(self.font, message, True, self.fg, bg)       #Generate an image from the text and background
        left = (self.image.get_width() - text_image.get_width()) // 2           #Centralise the text to the box

        self.image.fill(bg)                                                     #Clear the current image and fill with background colour
        self.image.blit(text_image, (left, 0))                                 #Then draw the new text image

    #Function to modify the current message
    def make_change(self, to_add = "", adding = True):
        if(not adding):                                                         #If backspace
            curmsg = self.current_message
            self.current_message = curmsg[:len(curmsg)-1]                       #trim one character
        elif(to_add.lower() == "space"):                                        #Else add space
            self.current_message += " "
        else:                                                                   #Or the relevant character
            self.current_message += to_add
        self.generate_image()                                                   #Then update the image

    #Function to activate/deactivate the textbox
    def change_focus(self, new_state):
        self.using = new_state                                                  #Change state
        self.generate_image()                                                   #Then update the image

    #Function to detect if a click started and ended in the textbox
    def clicked(self, mouse_down: bool, mouse_position):
        bounds = (self.position[0], self.position[0] + self.size[0],            #The bounds of the textbox
                  self.position[1], self.position[1] + self.size[1])
        if(not click_in_bounds(mouse_position, bounds)):                        #If not in bounds
            self.active = False                                                 #Unlatch
            self.change_focus(False)                                            #Deactivate
            return

        if(self.active and not mouse_down):                                     #If latched and releasing mouse
            self.change_focus(True)                                             #Activate
            self.active = False                                                 #Unlatch
        if(mouse_down): self.active = True                                      #Else latch

    #Function to read the keyboard inputs and add to the message
    def listen(self, this_key: str):
        if(this_key == "caps lock"):                                            #If caps lock pressed
            self.caps = not self.caps                                           #just invert the state of the caps lock
            return

        keys = pygame.key.get_pressed()                                         #This is here to let you hold backspace down
        deletable = self.current_message != "" and self.del_timer == 0          #Get all the keys which are currently down
                                                                                #Can only delete if there is message to delete
        if(keys[pygame.K_BACKSPACE] and deletable):                             #and its been 5 frames
            self.del_timer = 5                                                  #If backspace down
            self.make_change(adding = False)                                    #Start the timer
            return                                                              #Then remove a character (using the function)

        if(len(self.current_message) == self.max_length): return                #Can't add to a full textbox
        if(this_key not in self.valid_keys): return                             #Can't add an invalid character (see above)

        shift = keys[pygame.K_RSHIFT] or keys[pygame.K_LSHIFT]                  #Check if a shift key is down

        if(self.caps ^ shift):                                                  #Bitwise XOR of caps lock and shift
            to_add = this_key.upper()                                           #Upper case
        else:
            to_add = this_key                                                   #Lower case

        self.make_change(to_add = to_add)                                       #Then make the change
```

3.5 UI Screens

3.5.1 Home_Screen_UI.py

```
###IMPORTS###
from Sprite import Sprite
from Button import Button
from Common_Functions import text_to_image
import pygame

#Function to generate the UI for the home screen
def generate_UI(change_screen) -> list:
    '''
    Elements:
    -Title
    -File Select Button
    -Level Select Button
    '''
    big_font = pygame.font.SysFont("Arial", 72)                             #Load the Arial 72 font for the title
    lil_font = pygame.font.SysFont("Arial", 48)                             #Load the Arial 48 font for the buttons

    title_img = big_font.render("GAME GAME", True, (255,255,255))           #Create an image for the title, and two buttons
    files_img = text_to_image(lil_font, "FILE SELECT", True, (101,254,95))
    levels_img = text_to_image(lil_font,"LEVEL EDITOR", True, (255,0,0))

    lblTitle = Sprite((200,50), title_img)                                  #Instantiate a Sprite for the title
    btnFiles = Button((265,200), files_img, change_screen, "#FILES")        #Instantiate the two buttons
    btnLevels = Button((250,290), levels_img, change_screen, "#LEVELS")

    return [lblTitle, btnFiles, btnLevels]                                  #Return the elements
```

3.5.2 Files_Screen_UI.py

```
###IMPORTS###
from Button import Button
from Textbox import Textbox
from Common_Functions import text_to_image
import pygame

#Function to generate the UI for the file select screen
def generate_UI(change_screen) -> list:
    '''
    Elements:
    -Return Button
    -List of save file buttons
    -Delete Button
    '''
    return_img = pygame.image.load('Images/return_button.png').convert()        #Load the return, heart, and health bar images
    heart_img = pygame.image.load('Images/heart.png').convert_alpha()
    bar_img = pygame.image.load('Images/health.png').convert()

    btnReturn = Button((0,0), return_img, change_screen, "#HOME")               #Instantiate the return button

    font = pygame.font.SysFont("Arial", 54)                                     #Load the Arial 54 font
    fg, bg = (0,0,0), (0,160,255)                                               #Black Foreground, Blue Background
    saves = open("save_data.txt", "r").read().split("\n")                       #Load the save data file
    save_btn_list = [None] * 3
    save_found = False
    for x, save in enumerate(saves):                                            #For each save file
        position = (34, 152 + 80 * x)                                           #Determine its position

        parts = save.split(", ")                                                #Separate the parts of the save file

        save_img = pygame.Surface((700,64))                                     #Generate the background for the button
        save_img.fill(bg)

        if(parts[3] == "-1"):                                                   #If the save is empty
            txt_img = text_to_image(font, "NEW SAVE", True, fg, bg)             #Generate an image to show that
            save_img.blit(txt_img, (234,0))
            newBtn = Button(position, save_img, change_screen, "#NEW GAME",x)   #Instantiate a button of for the new save
        else:
            save_found = True

            name_img = text_to_image(font, parts[0], True, fg, bg)              #Generate an image to show the save file's information
            lives_img = text_to_image(font, parts[1], True, fg, bg)
            health_img = text_to_image(font, parts[2], True, fg, bg)
            level_img = text_to_image(font, parts[3], True, fg, bg)

            save_img.blit(name_img, (16,0))                                     #Assemble it all into one image
            save_img.blit(heart_img, (400,0))
            save_img.blit(lives_img, (464,0))
            save_img.blit(bar_img, (528,0))
            save_img.blit(health_img, (536, 0))
            save_img.blit(level_img, (632,0))
            newBtn = Button(position, save_img, change_screen, "#GAME",x)       #Instantiate a button for that save
        save_btn_list[x] = newBtn                                               #Add to save button list

    controls_img = text_to_image(font, "Controls", True, fg, bg)                #Generate an image for the control tooltip button
    btnControls = Button((301,0), controls_img, change_screen, "#CONTROLS")     #Instantiate the control tooltip button

    if(save_found):                                                             #Only generate if a save exists to be deleted
        delete_img = text_to_image(font, "DELETE", True, fg, (255, 0, 0))       #Generate an image to go to the delete form
        btnDelete = [Button((594,0), delete_img, change_screen, "#DELETE")]     #Instantiate the send to delete button
    else:
        btnDelete = []

    nameEntry = Textbox((34,420),(700,64), fg, bg, (0,100,160), "New Name", 16) #Instantiate a textbox to enter a new username

    return [nameEntry, btnReturn, btnControls] + btnDelete + [*save_btn_list]   #Return the UI elements
```

3.5.3 Game_Screen_UI.py

```
###IMPORTS###
from Sprite import Sprite
from Button import Button
from Common_Functions import text_to_image
import pygame

#The game screen has an overlay for the health bar and timer
#Or can be paused to reveal a pause screen
class Game_UI:
    def __init__(self, game_control):
        self.game = game_control                                                #The current object storing the game itself
        self.main_menu = game_control.main_menu                                 #The current object storing the menu
                                                                                #-importantly, it stores the UI driver code
        self.common_imgs = {"heart": pygame.image.load('Images/heart.png').convert_alpha(), #A dictionary of the heart and health images
                            "health": pygame.image.load('Images/health.png').convert()}     #which are used in both the game and the
                                                                                            #pause screens
        self.main_UI = []                                                       #Lists to store the UI generated
        self.pause_UI = []

    #Function to generate the UI overlay whilst the game is running
    def generate_main_ui(self):
        '''
        Elements:
        -Life count
        -Health bar
        -Timer
        '''
        life_UI = pygame.Surface((140,64))                                      #Create a light grey background for
        life_UI.fill((150,150,150))                                             #the life and health bar
        health_UI = pygame.Surface((70,64))
        health_UI.fill((150,150,150))

        lives = self.game.player.lives                                          #Read the player's current life and health
        health = self.game.player.health
        raw_time = self.game.timer                                              #Read the current time
        time_in_seconds = raw_time // 60                                        #Then convert to seconds

        font = pygame.font.SysFont("Arial", 54)                                 #Load the Arial 54 font
        fg, bg = (0,0,0), (150,150,150)                                         #Black foreground, Light Grey background
        life_img = text_to_image(font, str(lives), True, fg, bg)                #Create an image for the life, health and time
        health_img = text_to_image(font, str(health), True, fg, bg)
        timer_img = text_to_image(font, str(time_in_seconds), True, fg, bg)
```

```
        life_UI.blit(self.common_imgs["heart"], (0,0))                                  #Draw the life symbol and life image
        life_UI.blit(life_img, (60,0))                                                  #Onto the life display image
        health_UI.blit(self.common_imgs["health"], (0,0))                               #Do the same for the health display image
        health_UI.blit(health_img, (10,0))

        life_display = Sprite((0,0), life_UI)                                           #Instantiate a sprite for each image
        health_display = Sprite((140,0), health_UI)
        timer_x = 768 - timer_img.get_width()                                           #(The timer is on the right side)
        timer_display = Sprite((timer_x,0), timer_img)
        self.main_UI = [life_display, health_display, timer_display]                    #Store to the main UI list

    #Function to generate the UI for the pause screen
    def generate_pause_ui(self):
        '''
        Elements:
        -Repositioned Life, Health, and Timer displays
        -Background
        -Return Button
        -Quit Button
        '''
        self.generate_main_ui()                                                         #All elements on the overlay above can be
        for element in self.main_UI:                                                    #repositioned for the pause screen
            element.position[0] += 200                                                  #The repositioning method
            element.position[1] += 120
        self.main_UI[2].position[0] -= 400                                              #Pull the timer to the center of the screen

        background_img = pygame.Surface((500,400))                                      #Generate a background for the screen
        background_img.fill((30,30,30))                                                 #Dark grey fill

        font = pygame.font.SysFont("Arial", 66)                                         #Load the Arial 66 font
        fg = (0,0,0)                                                                    #Black foreground
        return_img = text_to_image(font, "RETURN", True, fg, (0,255,0))                 #Create the return button image (green bg)
        quit_img = text_to_image(font, "QUIT", True, fg, (255,0,0))                     #Create the quit button image (red background)

        background = Sprite((134,56), background_img)                                   #Instantiate the background as a Sprite
        returnBtn = Button((363,320), return_img, self.resolve_pause, True)             #Instantiate the two buttons
        quitBtn = Button((180,320), quit_img, self.resolve_pause, False)

        self.pause_UI = [background, returnBtn, quitBtn]                                #Store the UI in one place
        self.pause_UI += self.main_UI

    #Function to control what the return and quit buttons do
    def resolve_pause(self, return_to_game: bool):
        if(return_to_game[0]):                                                          #If returning to game
            self.game.paused = False                                                    #Just unpause
            return
        self.game.running = False                                                       #Else stop the game
        if(self.game.sent_from == "Menu"):                                              #Then if you are playing the actual game
            self.game.save_to_file(self.game.player.lives, self.game.player.health)     #Save
            self.main_menu.change_screen("#FILES")                                      #Return to the file select screen
        else:                                                                           #Otherwise you came from the level editor
            self.main_menu.change_screen(self.game.file)                                #So go back to the level editor
```

3.5.4 Control_Screen_UI.py

```
###IMPORTS###
from Sprite import Sprite
from Button import Button
from Common_Functions import text_to_image
import pygame

#Function to create the UI for the Control Screen
def generate_UI(change_screen) -> list:
    '''
    Elements:
    -Return Button
    -List of controls as images
    '''
    return_img = pygame.image.load('Images/return_button.png').convert()            #Load the return button image
    btnReturn = Button((0,0), return_img, change_screen, "#FILES")                  #Instantiate the return button

    font = pygame.font.SysFont("Arial", 54)                                         #Load the Arial 54 font
    fg, bg = (255,255,255), (50,50,50)                                              #White foreground, Grey background
    title_img = text_to_image(font, "CONTROLS", True, fg, bg)                       #Create images of the inputted text
    left_img = text_to_image(font, "Left or A to move left", True, fg, bg)
    right_img = text_to_image(font, "Right or D to move right", True, fg, bg)
    jump_img = text_to_image(font, "Up or W to jump", True, fg, bg)
    pause_img = text_to_image(font, "P to pause", True, fg, bg)
    attack_img = text_to_image(font, "Space to attack", True, fg, bg)

    title = Sprite((261,16), title_img)                                             #Instantiate a Sprite of each text
    left_tip = Sprite((181,96), left_img)
    right_tip = Sprite((149,176), right_img)
    jump_tip = Sprite((223,256), jump_img)
    pause_tip = Sprite((278,336), pause_img)
    attack_tip = Sprite((231,416), attack_img)

    return [btnReturn, title, left_tip, right_tip, jump_tip, pause_tip, attack_tip] #Return the generated objects
```

3.5.5 Delete_Screen_UI.py

```
###IMPORTS###
from Button import Button
from Textbox import Textbox
from Common_Functions import text_to_image
import pygame
###Premade lists and functions I need for the captcha###
from string import ascii_lowercase as LOW                       #A list of lowercase characters (a -> z)
from string import ascii_uppercase as UP                        #A list of uppercase characters (A -> Z)
from string import digits as DIG                                #A list of digits (0 -> 9)
from random import choices                                      #A function which generates a random string

#Function to generate the UI for the file delete screen
def generate_UI(change_screen, delete_save) -> list:
    '''
    Elements:
    -Return Button
    -List of save file buttons
```

```
    '''
    return_img = pygame.image.load('Images/return_button.png').convert()    #Load the return, heart, and healthbar images
    heart_img = pygame.image.load('Images/heart.png').convert_alpha()
    bar_img = pygame.image.load('Images/health.png').convert()

    btnReturn = Button((0,0), return_img, change_screen, "#FILES")          #Instantiate the return button

    font = pygame.font.SysFont("Arial", 54)                                 #Load the Arial 54 font
    fg, bg = (255,255,255), (0,0,0)                                         #White foreground, Black background
    saves = open("save_data.txt", "r").read().split("\n")                   #Load and read the save data file
    save_btn_list = [None] * 3                                              #Create an empty list for the save file buttons
    for x, save in enumerate(saves):                                        #For each save file:
        position = (34, 152 + 120 * x)                                      #Produce its position

        parts = save.split(", ")                                            #Separate the parts of a given save
        if(parts[3] == "-1"): continue                                      #If no save exists, then what would you delete?

        save_img = pygame.Surface((700,64))                                 #Create a background image for the button
        save_img.fill(bg)

        name_img = text_to_image(font, parts[0], True, fg, bg)              #Create an image for the username, lives
        lives_img = text_to_image(font, parts[1], True, fg, bg)             #health, and current level
        health_img = text_to_image(font, parts[2], True, fg, bg)
        level_img = text_to_image(font, parts[3], True, fg, bg)

        save_img.blit(name_img, (16,0))                                     #Draw each image onto the background image
        save_img.blit(heart_img, (400,0))
        save_img.blit(lives_img, (464,0))
        save_img.blit(bar_img, (528,0))
        save_img.blit(health_img, (536, 0))
        save_img.blit(level_img, (632,0))
        newBtn = Button(position, save_img, delete_save, x)                 #Instantiate a button for that save
        save_btn_list[x] = newBtn                                           #Add to list

    save_btn_list = list(filter((lambda x: x is not None), save_btn_list))  #Remove all "None" occurences in the list

    captcha = "".join(choices(LOW + UP + DIG, k = 5))                       #Generate a captcha
    nameEntry = Textbox((568,0),(200,64), fg, (160,0,0), bg, captcha, 5)    #Instantiate a textbox for the captcha

    return [nameEntry, btnReturn, *save_btn_list]                           #Return the list of elements
```

3.5.6 Levels_Screen_UI.py

```
###IMPORTS###
from Sprite import Sprite
from Button import Button
from Entry import Entry
from Listbox import Listbox
from Scrollbar import Scrollbar
from Textbox import Textbox
from Common_Functions import text_to_image
import pygame
import glob

#Generic function to generate the label for an entry object
def generate_label(text: str, font, bg: tuple):
    label = pygame.Surface((620,64))                                        #Generate a background
    label.fill(bg)                                                          #Using the passed bg colour
    label.blit(text_to_image(font, text, True, (0,0,0), bg), (0,0))         #Generate an image for the text and draw onto the bg
    return Sprite((0,0), label)                                             #Instantiate and return a Sprite for the image

#Function to generate the Level Select Screen UI
def generate_UI(change_screen) -> list:
    '''
    Elements:
    -Return Button
    -Tooltip Buttons
    -List of all custom levels
    '''
    return_img = pygame.image.load('Images/return_button.png').convert()    #Load the return, play, and scroll images
    play_img = pygame.image.load('Images/play.png').convert()
    scroll_img = pygame.image.load('Images/scroll_bar.png').convert_alpha()

    font = pygame.font.SysFont("Arial", 54)                                 #Load the Arial 54 font

    controls_img = text_to_image(font, "Buttons", True, (255,255,255), (63,72,204))     #Generate an image for the buttons which load
    keybinds_img = text_to_image(font, "Keybinds", True, (255,255,255), (200,191,231))  #the tool tips screens
    btnControls = Button((308, 0), controls_img, change_screen, "#TIPS", "Buttons")     #Instantiate the two buttons
    btnKeybinds = Button((292, 62), keybinds_img, change_screen, "#TIPS", "Keybinds")

    bgA, bgB = (0,160,255), (0,255,160)                                     #Blue background, Mint background
    levels = [level[14:len(level)-4] for level in glob.glob('Levels/Custom/*.csv')]     #Find all the custom level files
                                                                            #then trim to just show the level name
                                                                            #not the file path and file format
    entry_list = [None] * (len(levels) + 1)                                 #Initialise a list
    lbl_new_level = generate_label("NEW LEVEL", font, bgA)                  #Generate the label for the NEW LEVEL Entry
    btn_new_level = Button((0,0), play_img, change_screen, "#NEW LEVEL")    #Generate the button for the NEW LEVEL Entry
    entry_list[0] = Entry((0,0), lbl_new_level, btn_new_level, True)        #Insert the entry at position 0
    for x, level in enumerate(levels):                                      #For each level
        if(x % 2 == 0): this_bg = bgB                                       #Choose an alternating background colour
        else: this_bg = bgA

        this_label = generate_label(level, font, this_bg)                   #Generate the label for the entry
        this_button = Button((0,0), play_img, change_screen, level)         #Generate the button for the entry
        this_entry = Entry((0,0), this_label, this_button, True)            #Instantiate the entry using the label and button
        entry_list[x+1] = this_entry                                        #Add to the list

    btnReturn = Button((0,0), return_img, change_screen, "#HOME")           #Instantiate the return button
    #Minimum of 4 custom levels to scroll
    if(len(entry_list) > 3): scroll = Scrollbar((0,0), scroll_img)          #Only instantiate a scrollbar if scrolling is
    else: scroll = None                                                     #necessary
    lstLevels = Listbox((34,152), entry_list, scroll)                       #Then instantiate a listbox of the entries

    nameEntry = Textbox((34,420),(700,64), (0,0,0), bgA, (0,100,160), "New Name", 18)   #Instantiate a textbox for level name entry

    return [nameEntry, btnReturn, btnControls, btnKeybinds, lstLevels]      #Return the list of elements
```

3.5.7 Editor_Screen_UI.py

```
###IMPORTS###
from Sprite import Sprite
from Button import Button
import pygame

#Some UI turns on and off depending on the state of the editor, hence the class
class Editor_UI:
    def __init__(self, editor_control):
        self.editor = editor_control                                        #The current object used for the editor section

        self.meta = self.generate_meta_buttons()                            #Functions to generate each UI element by type
        self.static = self.generate_static_buttons()
        self.tiles = self.generate_tile_palette()
        self.entities = self.generate_entity_palette()
        self.mods = self.generate_mod_palette()

    #Generates the buttons which allow the user to change game states
    def generate_meta_buttons(self) -> list:
        '''
        META Elements:
        -Save button
        -Play button
        '''
        save_img = pygame.image.load('Images/save.png').convert()           #Load the save and play button images
        play_img = pygame.image.load('Images/play.png').convert()

        btnSave = Button((0,0), save_img, self.editor.unload, "Save")       #Instantiate the save and play buttons
        btnPlay = Button((64,0), play_img, self.editor.unload, "Play")
        return [btnSave, btnPlay]                                           #Return the button objects

    #Generates the buttons which appear on all 3 palettes
    def generate_static_buttons(self) -> list:
        '''
        Static Elements:
        -Mode change buttons
        -Scroll buttons
        -Change palette buttons
        '''
        draw_img = pygame.image.load('Images/Draw.png').convert()           #Load the images for the draw, erase
        erase_img = pygame.image.load('Images/Erase.png').convert()         #scroll, and palette change buttons
        scroll_img = pygame.image.load('Images/Arrow.png').convert_alpha()
        palette_scroll_img = pygame.image.load("Images/Palette_Arrow.png").convert_alpha()

        rotated_scroll_img = pygame.transform.rotate(scroll_img, 180)       #Rotate the two scroll images to use for
        rotated_ps_img = pygame.transform.rotate(palette_scroll_img, 180)   #a left-pointing arrow

        btnDraw = Button((636,316), draw_img, self.editor.btn_clicked, "Draw")      #Instantiate all the buttons
        btnErase = Button((708,316), erase_img, self.editor.btn_clicked, "Erase")
        btnRight = Button((708,388), scroll_img, self.editor.btn_clicked, "Right")
        btnLeft = Button((636,388), rotated_scroll_img, self.editor.btn_clicked, "Left")
        btnRPalette = Button((708,460), palette_scroll_img, self.editor.btn_clicked, "RPalette")
        btnLPalette = Button((636,460), rotated_ps_img, self.editor.btn_clicked, "LPalette")
        return [btnDraw, btnErase, btnRight, btnLeft, btnRPalette, btnLPalette]     #Return the button objects

    #Generic function to generate the palettes
    def generate_palette(self, colour: tuple, btns_to_make: list) -> list:
        '''
        Generic Palette Elements:
        -Background
        -Button for each "tile" type
        '''
        palette_img = pygame.Surface((160,512))                             #Generate a background image for the palette
        palette_img.fill(colour)
        palette = Sprite((608,0), palette_img)                              #Instantiate the image as a Sprite

        object_btns = []

        oway_img = pygame.image.load('Images/Oway.png').convert()           #One-Ways use the same image for
                                                                            #different directions of one-way
        for i, element in enumerate(btns_to_make):                          #Loop through each button
            match element:                                                  #Test for which image should be loaded
                case "Uway": img = oway_img                                 #Up oway uses the base oway image
                case "Lway": img = pygame.transform.rotate(oway_img, 90)    #Left oway is roated 90 degrees CW
                case "Dway": img = pygame.transform.rotate(oway_img, 180)   #Down oway is a flipped image
                case "Rway": img = pygame.transform.rotate(oway_img, -90)   #Right oway is rotated 90 degrees ACW
                case _: img = pygame.image.load(f'Images/{element}.png').convert_alpha()    #All other objects have different images
            x = 636 + (i % 2) * 72                                          #Determine the relevant x and y positions
            y = 30 + (i // 2) * 72
            btn = Button((x,y), img, self.editor.btn_clicked, element)      #Instantiate the button object
            self.editor.positions[element] = (x - 4, y - 4)                 #These are the positions for the "focuses"
            object_btns.append(btn)                                         #Add new object to the list

        return [palette] + object_btns                                      #Return a list of the palette and buttons

    #Specific function to generate the "tile palette"
    def generate_tile_palette(self) -> list:
        btns_to_make = ["Floor","Uway",                                     #Just a list of the different buttons
                        "Honey","Lway",                                     #on this palette
                        "Spike","Dway",
                        "Flag","Rway"]

        return self.generate_palette((55, 76, 255), btns_to_make)           #Call the generic function

    #Specific function to generate the "entity palette"
    def generate_entity_palette(self) -> list:
        btns_to_make = ["Player","Walker",                                  #Just a list of the buttons on this palette
                        "Crawler","Flyer"]

        return self.generate_palette((237,84,144), btns_to_make)            #Call the generic function

    #Specific function to generate the "modification palette"
    def generate_mod_palette(self) -> list:
        btns_to_make = ["SHeal","DJump",                                    #Just a list of the buttons on this palette
                        "MHeal","WJump",
                        "LHeal","DStrike",
                        "Run","Dash"]

        return self.generate_palette((26,214,88), btns_to_make)             #Call the generic function

    #Select the relevant elements to be displayed
    def assemble(self) -> list:
        current_UI = []

        match self.editor.palette:                                          #Selects the relevant palette
            case "Tiles":
                current_UI += self.tiles
```

```
            case "Entities":
                current_UI += self.entities
            case "Mods":
                current_UI += self.mods

        current_UI += self.static                                   #Add the static and meta UI
        current_UI += self.meta
        current_UI += self.editor.focuses                           #Add the "focuses" to show what is
                                                                    #currently selected
        if(not self.editor.hidden):                                 #Only display if not hidden
            return current_UI

        return []
```

3.5.8 Editor_Tip_Screen_UI.py

```
###IMPORTS###
from Sprite import Sprite
from Button import Button
from Common_Functions import text_to_image
import pygame

#Function to generate the UI for the Editor Palette Button Tooltip Screen
def generate_buttons_UI(change_screen) -> list:
    '''
    Elements:
    -Return Button
    -List of controls
    '''
    return_img = pygame.image.load('Images/return_button.png').convert()                    #Load the return, save, play
    save_img = pygame.image.load('Images/save.png').convert()                               #draw and erase button images
    play_img = pygame.image.load('Images/play.png').convert()
    draw_img = pygame.image.load('Images/Draw.png').convert()
    erase_img = pygame.image.load('Images/Erase.png').convert()

    draw_img = pygame.transform.scale(draw_img, (64, 64))                                   #Resize the draw and erase
    erase_img = pygame.transform.scale(erase_img, (64, 64))                                 #images to match return, etc

    btnReturn = Button((0,0), return_img, change_screen, "#LEVELS")                         #Instantiate the return button

    save_display = Sprite((128,32), save_img)                                               #Create a sprite for each of the
    play_display = Sprite((128,112), play_img)                                              #other images
    draw_display = Sprite((128,192), draw_img)
    erase_display = Sprite((128,272), erase_img)
    button_displays = [save_display, play_display, draw_display, erase_display]             #Then store in a list

    font = pygame.font.SysFont("Arial", 54)                                                 #Load the Arial 54 font
    fg, bg = (255,255,255), (50,50,50)                                                      #White foreground, Grey background
    save_msg_img = text_to_image(font, "Save the level", True, fg, bg)                      #Generate an image for each string
    play_msg_img = text_to_image(font, "Play the level", True, fg, bg)                      #which will be used to explain what
    draw_msg_img = text_to_image(font, "Set mode to 'draw'", True, fg, bg)                  #the buttons do
    erase_msg_img = text_to_image(font, "Set mode to 'erase'", True, fg, bg)
    scroll_msg_img = text_to_image(font, "Use black arrows to scroll", True, (0, 0, 0), bg)
    palette_msg_img = text_to_image(font, "Use purple arrows to change palette", True, (152, 68, 152), bg)

    save_msg = Sprite((224,32), save_msg_img)                                               #Instantiate a sprite for each msg
    play_msg = Sprite((224,112), play_msg_img)
    draw_msg = Sprite((224,192), draw_msg_img)
    erase_msg = Sprite((224,272), erase_msg_img)
    scroll_msg = Sprite((131,352), scroll_msg_img)
    palette_msg = Sprite((26,432), palette_msg_img)
    msg_displays = [save_msg, play_msg, draw_msg, erase_msg, scroll_msg, palette_msg]       #Then store in a list

    return [btnReturn] + button_displays + msg_displays                                     #Return the UI elements

#Function to generate the UI for the Editor Palette Keybinds Tooltip Screen
def generate_keybinds_UI(change_screen) -> list:
    '''
    Elements:
    -Return Button
    -List of keybinds
    '''
    return_img = pygame.image.load('Images/return_button.png').convert()                    #Load the return button image
    btnReturn = Button((0,0), return_img, change_screen, "#LEVELS")                         #Instantiate the return button

    messages = ["H: Hide save and play buttons", "M: Change mode",                          #A list of the messages to
                "E: Set mode to 'erase'", "D: Set mode to 'draw'",                          #be displayed
                "1: Change palette to tiles", "2: Change palette to entities",
                "3: Change palette to consumables", "LEFT: Scroll left",
                "RIGHT: Scroll right"]
    msg_buttons = []
    font = pygame.font.SysFont("Arial", 32)                                                 #Load the Arial 32 font
    fg, bg = (255,255,255), (50,50,50)                                                      #White foreground, Grey background
    for k, msg in enumerate(messages):                                                      #For each message
        msg_img = text_to_image(font, msg, True, fg, bg)                                    #Generate an image of the message

        x = (768 - msg_img.get_width()) // 2                                                #Determine its position
        y = 32 + 48 * k                                                                     #horizontally centered
        new_msg = Sprite((x,y), msg_img)                                                    #Instantiate a Sprite object
        msg_buttons.append(new_msg)                                                         #Append to the list

    return [btnReturn] + msg_buttons                                                        #Return the UI elements
```

3.6 Miscellaneous

3.6.1 AStar.py

```
###IMPORTS###
import heapq
from math import inf
from Common_Functions import pythagorus

#A class to store the information about the node
#Just a data container
class Node:
    def __init__(self, x, y):
        self.parent_x = 0
        self.parent_y = 0
        self.f = inf
        self.g = inf
        self.h = 0

#Determine if a point is in the grid
def is_in_grid(x, y, ROWS, COLS) -> bool:
    return (x >= 0) and (x < COLS) and (y >= 0) and (y < ROWS)

#Determine if you can move along the given vector in the given tile
def is_movable(tile, vector) -> bool:                                   #Guard clauses to reject invalid moves
    if(tile in ["Floor","Spike"]): return False                         #No moving in solid objects
    if(tile in ["Honey","Dway"] and vector[1] == -1): return False      #No moving up in a down one-way
    if(tile in ["Honey","Uway"] and vector[1] == 1): return False       #No moving down in an up one-way
    if(tile == "Rway" and vector[0] == -1): return False                #No moving left in a right one-way
    if(tile == "Lway" and vector[0] == 1): return False                 #No moving right in a left one-way
    return True                                                         #Not bad -> Good :D

#Determine if you can move from (x,y) to (new_x,new_y)
def is_reachable(x, y, new_x, new_y, vector, grid) -> bool:
    curTile = grid[y][x]
    newTile = grid[new_y][new_x]

    if(not is_movable(newTile, vector)): return False                   #Do either of the tiles
    if(not is_movable(curTile, vector)): return False                   #reject the movement?

    if(abs(vector[0]) + abs(vector[1]) == 1): return True               #Orthogonal movement is valid at this stage

    horizTile = grid[y][new_x]                                          #A diagonal motion is a vertical and horizontal
    vertTile = grid[new_y][x]                                           #motion, so test if it is valid for both
    return is_movable(horizTile, vector) and is_movable(vertTile, vector)

#Determine if the position is the destination
def is_destination(x, y, dest) -> bool:
    return x == dest[0] and y == dest[1]

#Calculates the h_value for a given positoin
def calculate_h_value(x, y, dest) -> int:
    return pythagorus((x,y), dest, scalar = 10)

#Trace the path from the start node to destination, returning the next node on the path
def trace_path(nodes, dest) -> tuple:
    path = []
    x = dest[0]
    y = dest[1]

    while not (nodes[y][x].parent_x == x and nodes[y][x].parent_y == y):    #Just follow the pointers in the nodes
        path.append((x, y))
        temp_x = nodes[y][x].parent_x
        temp_y = nodes[y][x].parent_y
        x = temp_x
        y = temp_y

    return path[-1]                                                     #The path is traced from destination to start

#A shortest-path pathfinding algorithm from a source (src) node to a destination (dest) node
def a_star(grid, src, dest, ROWS, COLS):
    if(src == dest):                                                    #Don't move if already at the end
        return src

    closed_list = [[False for x in range(COLS)] for y in range(ROWS)]   #Initialise the closed list and graph (nodes)
    nodes = [[Node(x,y) for x in range(COLS)] for y in range(ROWS)]

    x = src[0]                                                          #(Initialise) and point the source node at itself
    y = src[1]
    source_node = nodes[y][x]
    source_node.f = source_node.g = source_node.h = 0
    source_node.parent_x = x
    source_node.parent_y = y

    open_list = []
    heapq.heappush(open_list, (0.0, x, y))

    vectors = [(0, 1), (0, -1), (1, 0), (-1, 0),                        #Just a list of vectors to adjacent nodes
               (1, 1), (1, -1), (-1, 1), (-1, -1)]

    while len(open_list) > 0:                                           #Loop until all options considered
        p = heapq.heappop(open_list)                                    #Select the closest option (smallest f)

        x = p[1]
        y = p[2]
        closed_list[y][x] = True                                        #Mark as visited (the shortest path here has been found)

        for vector in vectors:                                          #Consider each neighbouring node
            new_x = x + vector[0]
            new_y = y + vector[1]

            if(not is_in_grid(new_x, new_y, ROWS, COLS)): continue      #Ignore if not real
            if(not is_reachable(x, y, new_x, new_y, vector, grid)): continue    #Ignore if not reachable
            if(closed_list[new_y][new_x]): continue                     #Ignore if visited

            neighbour = nodes[new_y][new_x]

            if(is_destination(new_x, new_y, dest)):                     #If neighbour is destination then we're done
                neighbour.parent_x = x                                  #This only works as our nodes lie on a grid
                neighbour.parent_y = y
                trace_path(nodes, dest)
                return trace_path(nodes, dest)
            else:
                g_new = nodes[y][x].g + pythagorus((x, y), (new_x, new_y), scalar = 10) #Calculate updated values for f, g, and h
                if(neighbour.f == inf):                                 #Only update h if this is the first visit
```

```
                h_new = calculate_h_value(new_x, new_y, dest)
            else: h_new = neighbour.h
            f_new = g_new + h_new

            if(neighbour.f > f_new):                                    #If using this node is part of a shorter path to the neighbour
                heapq.heappush(open_list, (f_new, new_x, new_y))        #Add the neighbour to the open list

                neighbour.f = f_new                                     #Update the neighbour information
                neighbour.g = g_new
                neighbour.h = h_new
                neighbour.parent_x = x
                neighbour.parent_y = y

    return "Fail"                                                       #Failsafe if destination could not be reached
```

3.6.2 Common_Functions.py

```
###IMPORTS###
import pygame
from math import sqrt as root

#Calculates the pythagorean distance between two points
def pythagorus(positionA, positionB, scalar = 1) -> int:                #It's just pythagorus
    x_distance = positionA[0] - positionB[0]
    y_distance = positionA[1] - positionB[1]

    square_md = (x_distance ** 2) + (y_distance ** 2)
    true_md = root(square_md)
    return round(true_md * scalar)                                      #Scale, then return as an integer

#Determines if a point lies within some set of bounds
def click_in_bounds(mouse_position: list, bounds: tuple) -> bool:       #Guard clauses will reject invalid clicks
    if(mouse_position[0] < bounds[0]): return False                     #Mouse x < object left
    if(mouse_position[0] > bounds[1]): return False                     #Mouse x > object right
    if(mouse_position[1] < bounds[2]): return False                     #Mouse y < object top
    if(mouse_position[1] > bounds[3]): return False                     #Mouse y > object bottom
    return True                                                         #Not bad -> good :D

#Converts a string into a rectangular image
def text_to_image(font, text: str, bold: bool, fg_colour: tuple, bg_colour = (0,0,0)):
    txt = font.render(text, bold, fg_colour)                            #Premade function to convert a text to an image
    image = pygame.Surface((txt.get_width(), txt.get_height()))         #Create a drawable object for the image
    image.fill(bg_colour)                                               #Fill with the background colour
    image.blit(txt, (0,0))                                              #Then draw the image onto the background
    return image
```

4 Testing

4.1 Testing Approach

During the development of my program, I have used bottom-up testing to debug my functions as I produce them. Bottom-up testing is a type of testing where you test functions as you go, often in isolation from the system. Through this type of testing, I can ensure that individual functions work independently, so if bugs appear later it is due to an interaction between two functions, and not a fault of the logic of an individual function – that is to say a fault is due to the integration. I will then use top-down testing to ensure the program functions as a whole system. Top-down testing is the act of testing a function after it is integrated into the main program. This branch of testing is what is covered here, as bottom-up testing was used strictly to produce code.

We can further classify tests into white-box and black-box testing. In white-box testing, we intend to follow the logic of a function, often through print statements. This is used to test for logical faults or bottlenecks, and can be used to identify areas for optimization. In black-box testing, we don't really care how a function works, but rather that for a given input we get an expected output. This type of testing is useful for visual updates, whereas white-box testing is useful for checking the program is operating as expected (for example, lives counter is being kept up to date).

Most of my tests in this section will be black-box tests, as I know each function works by itself during development, and what matters most is that the user gets the intended feedback from the system for a given input.

4.2 Program Requirements

This is a copy of 1.3, just to have a copy of the requirements in this section.

1) The player can move left and right using A/D respectively, or using the relative arrow key, as well as being able to jump by pressing W or the Up Arrow Key.
2) Include Mario's movement improvements (see 1.2.2)
3) The game screen should scroll to keep the player on screen.
4) The game must contain the following tile types:
    - Floor/Wall tile
    - One way tile
    - Honey tiles (slow the player and prevent jumping)
    - Spike tile (damage the player)
5) Levels are loaded from a file, and are accurately assembled to be played
6) Beating a level causes the next level to begin
7) Levels must be beaten in one hundred seconds
8) The player must be able to save
9) The player must be able to create a save (if space allows)
10) The player must be able to delete an existing save (to make space)
11) The game must correctly load the relevant form (see 2.3)
12) Contain the above health system
13) The player can attack enemies to lower their health (and the inverse), with one attack animation doing one attack's worth of damage
14) Levels contain enemies which:

- Attack and move in diverse ways
- Have mutations which occur randomly between playthroughs, altering enemy stats.

15) The player can restore health by using healing items
16) The player can update their movement options using consumable items

The game should contain menus which:

1) Respond to mouse input – for example;
    - A button does a function when clicked
    - An image does not do a function when clicked
2) Connect to one another to separate information
3) Can be dynamically updated
    - A list of save files updates after a death/level clear
    - A list of custom levels can grow as levels are added

The game should also contain a level editor which:

1) Allows the player to create custom levels
2) Level should be created by "painting" tiles onto a grid
3) The player can save the levels they create
4) The player can edit the levels they have saved
5) The player can play the levels they have saved
6) Elements can be placed anywhere
7) The game should check to make sure the level is playable – and not running a "bad" level

As the program requirements are split into three sections, I will test the program in three sections. As a result, some tests may be redundant/tested within a test in a different section. Redundant tests will be included for completeness.

4.3 Testing the Game

In some tests below, I will check if the program is properly saving to the save file. For later reference, the save file will start as the following:

Testing_Name, 5, 10, 1
Old_Save, 3, 99, -1
Empty_Save, 5, 10, -1

I will first test with normal data (Video 1):

Req.	Description	Input Data	Expected Result	Actual Result
1	The player can move left and right	A key, left arrow, D key, right arrow	When pressing A or left the player moves left, when pressing D or right the player moves right. A combination of a left and a right results in no movement.	0:04
1	The player can jump	Up arrow, W key on normal ground	The player should jump up when the spacebar is pressed	0:16
1	The player falls when not grounded	None	The player should fall when not on a ground tile	0:16
2	The player accelerates horizontally	Hold the right arrow	The player should gradually accelerate to a top speed in the right direction	(Video 3) 0:36
2	The player experiences drag when turning	Hold the right arrow, release, then hold the left arrow	The player should gradually decelerate from top speed before turning left	(Video 3) 0:36
2	The player can vary their jump height	Tap the up arrow, Hold the up arrow	When tapping the spacebar, the player jumps up slightly. When holding the spacebar, the player jumps much higher	0:40
2	The player has coyote time	Press the up arrow after walking off a platform	After pressing the spacebar, the player should jump if they just left the platform	0:45
3	The screen scrolls	Left and right arrows	After getting close to the right side of the screen, the screen scrolls instead of the player moving. The same holds for when you move left from the right side of the screen	1:00

4	Floor tile	Left, up, and right arrows	The player cannot enter the tile from any direction	1:10
4	Spike tile	Left, up, and right arrows	The player cannot enter the tile from any direction, and making contact deals 1 damage to the player	1:10
4	One way tile	Left, up, and right arrows	The player can enter from any direction other than the indicated one	2:00
4	Honey tile	Left, up, and right arrows	The player can move horizontally through the honey, but not vertically	1:10
5	Levels load properly (White box test)	Load_Test.csv	Shortly after pressing play, the program loads the file into a 2d array	2:30
5	Objects are created properly (White box test)	Load_Test.csv	After loading the file, the program instantiates objects for each tile	2:30
6	Beating a level causes the next level to being	1.csv, 2.csv, save file A	Upon touching the flag in level 1, the screen should go black and level 2 should load	3:00
7	Levels must be beaten in one hundred seconds	2.csv, save file A	After waiting for 100 seconds, the player should die and the level should reload	4:50
8	The player can pause the game	P key	By pressing p, the game pauses, stopping the timer and opening the pause menu. Pressing "return" unpauses the game. Pressing "quit" closes the game and loads file select	3:10
8	The game saves good progress	1.csv, 2.csv, save file A	After beating the level, the game should save, updating the save file to reflect this: The first line becomes Testing_Name, 5, 10, 2	5:20
8	The game saves bad progress	2.csv, save file A	After dying (in this case falling off the screen), the game should save, with the first line becoming: Testing_Name, 4, 10, 2	5:30
8	The player can save their progress	2.csv, save file A	The player should take damage to the spike	5:45

			platform. After opening the pause menu, and pressing "quit", the game should save, with the first line becoming: Testing_Name, 4, 9, 2	
9	The player can create a save	Save file B, "Name A" entered into the textbox	Upon pressing the second save file button, the game should update the second line of the save file to: Name A, 5, 10, 1	5:55
10	The player can delete an existing save	Save file B	In the delete form, I will answer the captcha then click the second save. The program should delete the second save by updating the second line to: new, 5, 10, -1	6:05
10	The player needs to get the captcha right	"abcde" entered into the captcha	Upon clicking a save file, the program should just return to the file select form, as the captcha was incorrect	6:20
11	The game loads the correct form	None	The program loads game states according to the finite state machine in 2.3	6:30
12	The game contains a health system	Save file B, "Name B" entered into the textbox	Upon creating the new save, the player should start with 5 lives and 10 health.	6:50
13	The player can attack enemies (White box test)	Combat_Test.csv, spacebar	Upon pressing the spacebar, the enemy should take exactly 1 damage if it at least one tile in front of the player. Enemies outside this range take no damage	7:10
13	Enemies can attack the player	Combat_Test.csv	Upon touching the enemy, the player should take exactly 1 damage	7:40
13	Enemies die after losing all their health (White box test)	Combat_Test.csv, spacebar	Upon falling to 0 health, the enemy is deleted from memory	7:10
13	The player dies after losing all their health	Combat_Test.csv	Upon falling to 0 health, an animation plays and the player loses a life.	7:50

			The game then reloads the level	
13	The player gets a game over	Save file A, 2.csv	After losing their final life, the game should load the home screen, and the first line of the save file becomes: Testing_Name, 5, 10, 2	8:20
14	Levels contain enemies which attack and move in diverse ways - Walkers	Enemy_Test.csv	Walker enemies walk along a platform and attack like the player	9:00
14	Levels contain enemies which attack and move in diverse ways - Flyers	Enemy_Test.csv	Flyer enemies Pathfind the shortest path to the player and fly towards them. They attack by dealing damage on contact	8:40
14	Levels contain enemies which attack and move in diverse ways - Crawlers	Enemy_Test.csv	Crawler enemies crawler around a platform, and attack by dealing damage on contact	9:05
14	Enemies can mutate - Walkers (White box test)	Enemy_Test.csv	Walker enemies can have random health, damage, attack time, and speed	9:50
14	Enemies can mutate - Flyers (White box test)	Enemy_Test.csv	Flyer enemies can have random health, and damage	9:50
14	Enemies can mutate - Crawlers (White box test)	Enemy_Test.csv	Crawler enemies can have random health, damage, and attack time	9:50
15	Player can restore health using healing items	Heal_Test.csv	Upon touching a healing item, the heal disappears, and the player gains 1 health	10:20
16	The player can update their movement (White box test)	Upgrade_Test.csv	Upon touching the consumable, the player's upgrade becomes the relevant value	10:50
16	Run upgrade	Upgrade_Test.csv, right arrow	The player's max speed is doubled while active	11:15
16	Dash upgrade	Upgrade_Test.csv, right shift, left shift	The player can teleport 3 tiles forward by pressing left shift or right shift. This movement is blocked by walls	10:50

16	Double Strike upgrade	Upgrade_Test.csv, spacebar	The player attacks in both directions at once	11:05
16	Double Jump upgrade	Upgrade_Test.csv, up arrow	The player can perform a second jump midair	11:00
16	Wall Jump Upgrade	Upgrade_Test.csv, up arrow	The player can perform a jump whilst touching a wall – but not if that wall is honey	11:10

I will then test with erroneous data (Video 2). For some tests, there is no such data, such as requirement 1.

Req	Description	Input Data	Expected Result	Actual Result
5	The game tries to load a level which does not exist	Save file A (Save file A will be modified to Testing_Name, 5, 10, 101)	The file should start to load the file, then print the message "Congratulations, you have beaten Game Game :D", before loading level 100 instead	0:00
6	There is no next level	Save file A, 100.csv (Save file A will be modified to Testing_Name, 5, 10, 100)	Upon touching the flag, the same result as above should occur	0:45
9	The player tries to create a save without entering a name	Save file C, no value in the textbox	The form should not change, but the textbox should update to say "Input a name first". The textbox should also activate, accepting user input	0:55
9	Inputting a name in the textbox whilst selecting an existing save	Save file A, "New Name" in the textbox	The game should load save file A, thus loading level 100. Upon returning to the game, the save file should be unedited	1:10
9	The player inputs a name which is just whitespace	Save file C, " " in the textbox (3 spaces)	The form should not change, but the textbox should update to say "Input a name first". The textbox should also activate, accepting user input	1:20

Finally I will test with extreme data (Video 3). Again, for some tests there is no such extreme data, like requirement 1.

Req	Description	Input Data	Expected Result	Actual Result
2	Player tries to coyote jump out of honey	Up arrow	The player just falls	0:00
3	The screen scrolls to keep the player on screen regardless of speed	Hold the right arrow – max speed and acceleration will be modified to allow for 16 tiles per second of movement. Speed_Test.csv	The screen scrolls to the right, keeping the player a set distance from the edge of the screen	0:35
4	Tiles prevent movement regardless of speed	Hold the right arrow – max speed and acceleration will be modified to allow for 16 tiles per frame of movement. Speed_Test.csv	The tiles stop the movement	0:40
5	Player loads level 100 – the final level	Save file A, 100.csv	Level 100 loads just like any other level	1:10
7	The player beats the level in the final second	Save file B, 1.csv	Level progression should take precedence over timer death, so the second level loads	2:05
9	The player tries to input a 100 character name	Save file C, "abc…tuv" (a 100 character string) entered into the textbox	Only the first 16 characters are added to the textbox	2:20
11	The player repeatedly changes between two forms	None	The forms should load as indicated by the finite state machine in 2.3 irrespective of the speed	2:55
13	The player falls below 0 health (-1 health)	Enemy_Test.csv – This will require an enemy which deals 2 damage	The player should die as though they only fell to 0 health	4:40
15	The player heals way too much	Overheal_Test.csv	The player should continue to heal, overflowing the display after 99 health	5:05
16	The player runs into another upgrade at top speed	Up_Edge_Test.csv, right arrow	The player immediately overrides their upgrades, then decelerates to the standard top speed	5:50

4.4 Testing the UI

Alongside the testing the requirements above, I will also include further tests for each UI element – those being the Button, Entry, Listbox, Scrollbar, and Textbox

At this point, the save data file contains the following:

Testing Name, 5, 10, 100
Name B, 5, 10, 2
Empty_Save, 5, 10, -1

Starting with normal data (Video 4):

Req	Description	Input Data	Expected Result	Actual Result
1	UI elements respond to mouse input (White box test)	Left click	The UI element recognises the start and end of a click, printing the position of the mouse at both points	0:00
2	Menus are connected	None	The program loads game states according to the finite state machine in 2.3	0:10
3	Menus can update at runtime – modifying a save updates the file select screen	Save_data.txt, Save file A – will be used to update the save file	The top button of the files menu should go from (Testing Name, 5, 10, 100) to (Testing Name, 4, 10, 100)	0:25
3	Menus can update at runtime – creating a level extends the level select listbox	My_New_Level as the name of the level	Upon returning to the level select menu, the new level can now be selected in the Listbox	0:40
---	Images do nothing when clicked	Left click	Nothing happens	1:00
---	Buttons perform a function when clicked (White box test)	Left click	The stored function is called when you click the button	1:05
---	An entry acts as a container for a label and a button (White box test)	None	An entry stores a reference for an image and a button	1:15
---	The contained button is repositioned to be relative to the top left of the entry (White box test)	None	The position variable of the button updates	1:25
---	The position of a click is made relative to the top left corner of an entry (White box test)	Left click	The position passed into the entry is changed when it is passed to the button	1:55

---	Clicks on an entry are passed to its button (White box test)	Left click	A click is passed to the button	1:55
---	A Listbox acts as a container of entries	None	A Listbox stores a reference to each of its entries as a list	1:40
---	The contained entries are repositioned to be relative to the top left of the entry (White box test)	None	The position variable of the entry updates	1:40
---	The position of a click is made relative to the top left corner of the listbox (White box test)	Left click	The position passed into the listbox is changed when it is passed to the entries	2:10
---	Clicks on a Listbox are passed to each of its entries	Left click	A click is passed to the entries	2:10
---	Only 3 entries are visible at a time	None	A Listbox with >3 entries only displays the first 3. Scrolling results in the next 3 being displayed	2:35
---	Only 3 entries are active (clickable) at a time	Left click	Clicking below the Listbox with >3 entries does nothing	2:40
---	A scrollbar only generates with a Listbox of length 4 or more	None	A Listbox with 1 – 3 entries has no scrollbar. A Listbox with 4+ entries has a scrollbar	2:55
---	Clicking the bottom arrow of a scrollbar scrolls down one	Left click	The first entry is unloaded, and the fourth entry loads	3:15
---	Clicking the top arrow of a scrollbar scrolls up one	Left click	The fourth entry is unloaded, and the first entry loads	3:15
---	Clicking the middle of a scrollbar scrolls to the middle of the Listbox (White and black box test)	Left click	The Listbox scrolls to the middle of the list	3:15
---	Clicks on a Listbox are passed to its scrollbar	Left click	Clicking the Listbox passes it to the scrollbar	2:10
---	A scrollbar is repositioned to be relative to its Listbox (White box test)	None	The position variable of the scrollbar updates	3:30
---	A textbox starts empty (White box test)	None	Loading the Files screen loads the textbox with no stored value	3:40

---	A textbox loads inactivated, displaying a base message	None	Loading the Files screen loads the textbox in the inactive state	3:40
---	Clicking the textbox activates it	Left click	Clicking the textbox activates it – the textbox changes background colour to a lighter blue	3:45
---	Clicking elsewhere deactivates an active textbox	Left click	Clicking elsewhere deactivates the active textbox - the textbox changes background colour to a darker blue	3:50
---	Typing whilst a textbox is active adds those characters to the stored text (white box test)	"Test string" typed	The current message variable of the textbox gradually becomes "Test string"	3:55
---	A textbox displays its stored text when not empty	"Show Me" typed	The textbox gradually displays "Show Me" as it is typed	4:10
---	Pressing backspace removes a character from the textbox	"Show Me" typed followed by backspace	The textbox displays "Show M"	4:20
---	Holding backspace removes multiple characters from a textbox	"Show Me" typed followed by backspace being held	The message "Show Me" is gradually removed	4:25

We then test erroneous data (Video 5):

Req	Description	Input Data	Expected Result	Actual Result
1	Clicks must start and end in a UI element – Testing latching from 2.2.4	Left click which starts in bounds then ends out of bounds.	Nothing happens	0:00
1	Clicks must start and end in a UI element – Testing latching from 2.2.4	Left click which starts out of bounds then ends in bounds	Nothing happens	0:00
1	UI only responds to left clicks	Left click, right click, scroll wheel click	There is a response to the left click, but no response for the alternatives	0:10
3	Deleting a save file removes its information from the file select screen	Relevant information to delete a save file – depends on the captcha, Save file B	After reloading the File select screen, the middle button displays "NEW SAVE"	0:35
---	Pressing backspace while a textbox is empty	Backspace	Nothing happens	0:45

---	Only alphanumeric (+ space) characters can be added to a textbox	"a2#/* - +C" typed (a,2,#,/,*,space,-,tab,+,C)	The textbox displays "a2 C"	0:50

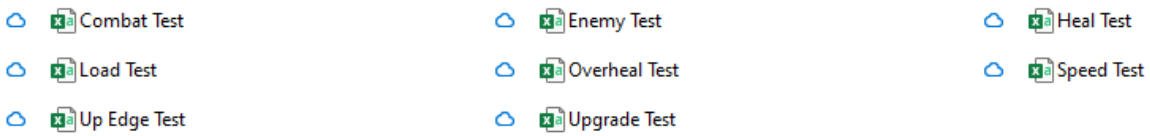
We then test extreme data (Video 6):

Req	Description	Input Data	Expected Result	Actual Result
---	The player tries to input a 100 character string into a textbox	Save file C, "abc...tuv" (a 100 character string) entered into the textbox	Only the first 16 characters are added to the textbox	0:00

4.5 Testing the Editor

The save data file is no longer relevant when testing the level editor. What matters is the contents of the Levels/Custom folder which stores the .csv files for custom levels.

The folder starts as the following:



The custom levels folder before these tests

Similar to testing the editor, I will also test the functionality of the controls in the editor – such as testing the different modes, or changing the tile – alongside testing the requirements above

We now start by testing with normal data (Video 7):

Req	Description	Input Data	Expected Result	Actual Result
1	The player can create custom levels	Editor_Test as the name of the new level	A new level is created in the editor, with the Editor_Test.csv file being added to the custom levels folder	0:15
2	The player paints tiles to a grid	Left click, Editor_Test.csv	The screen reflects the painted tiles	0:35
3	The player can save the levels they create	Editor_Test.csv	Editor_Test.csv updates to store the painted tiles	0:45
4	The player can load custom levels into the editor	Editor_Test.csv	Editor_Test from above is recreated in the editor	0:45
4	The player can edit custom levels	Left click, Editor_Test.csv	The player can paint over the recreated Editor_Test	1:00
5	The player can play custom levels	Load_Test.csv	Load_Test can be played as though it were a normal level. The	1:15

			player should start with 1 life and 10 health	
5	Beating a custom level	Load_Test.csv	Beating a custom level just reloads it	1:20
5	Dying in a custom level	Load_Test.csv	Dying in a custom level just reloads it	2:15
6	Elements can be placed anywhere on the grid	Left click	The player can place any element at any position in the grid	2:50
7	The program should check for the existence of a player object	No player object exists	The program stays in the editor and prints "You need both a flag and a player"	3:25
7	See above	Player object exists	The program loads the level into an instance of the game	3:20
7	The program should check for the existence of a flag object	No flag object exists	The program stays in the editor and prints "You need both a flag and a player"	3:35
7	See above	Flag object exists	The program loads the level into an instance of the game	3:20
---	Selecting a tile in the palette changes the active tile (White box test)	Editor_Test.csv	Clicking on a tile changes the active tile variable	3:45
---	Pressing the erase button sets the mode to "erase" (White box test)	Editor_Test.csv	Clicking on a tile sets the mode variable to "erase"	4:00
---	Pressing the draw button sets the mode to "draw" (White box test)	Editor_Test.csv	Clicking on a tile sets the mode variable to "draw"	4:00
---	The player can change the active palette	Editor_Test.csv	Clicking the purple arrows changes the active palette (palette order is circular)	4:15
---	The player can scroll through the level	Editor_Test.csv	Clicking a black arrow scrolls the screen in that direction	4:20
---	The player can hide the save and play buttons, as well as the palette for a clearer view	Editor_Test.csv, h key	Pressing h hides non-grid UI elements. The player can still draw in this state. Pressing h again shows non-grid UI elements	4:30
---	The player can use key binds to edit faster – hide shortcut	Editor_Test.csv, h key	Pressing h hides/shows non-grid UI elements	4:30

---	The player can use key binds to edit faster – mode alternate shortcut	Editor_Test.csv, m key	Pressing m alternates between the mode variable between "erase" and "draw"	4:45
---	The player can use key binds to edit faster – draw shortcut	Editor_Test.csv, d key	Pressing d sets the mode variable to "draw"	4:55
---	The player can use key binds to edit faster – erase shortcut	Editor_Test.csv, e key	Pressing d sets the mode variable to "erase"	5:00
---	The player can use key binds to edit faster – tile palette shortcut	Editor_Test.csv, 1 key	Pressing 1 sets the loads the tile palette, with the active tile becoming "Floor"	5:10
---	The player can use key binds to edit faster – entity palette shortcut	Editor_Test.csv, 2 key	Pressing 2 sets the loads the entities palette, with the active tile becoming "Walker"	5:10
---	The player can use key binds to edit faster – mod. palette shortcut	Editor_Test.csv, 3 key	Pressing 3 sets the loads the tile palette, with the active tile becoming "Small Heal"	5:10
---	The player can use key binds to edit faster – scroll left shortcut	Editor_Test.csv, left arrow	Pressing left scrolls the screen left 1 tile	5:25
---	The player can use key binds to edit faster – scroll right shortcut	Editor_Test.csv, right arrow	Pressing right scrolls the screen right 1 tile	5:25

We now test erroneous data (Video 8):

Req	Description	Input Data	Expected Result	Actual Result
1	Player creates a level with an existing name	Editor_Test used as a name	The editor doesn't load, and the name is rejected	0:00
1	Player creates a level with a whitespace name	" " (3 spaces) used as a name	The editor doesn't load, and the name is rejected	0:10
1	Player creates a level with no name	None	The editor doesn't load, and textbox becomes active to prompt an input	0:15
2	Player tries to paint off-screen*	Editor_Test.csv, Left click	The editor only considers clicks which occur on the screen – so these are rejected	0:25
7	Player tries to add a second player object	Editor_Test.csv, Left click	The editor deletes the old player object and creates a new one at the click position	0:50

7	Player tries to add a second flag object	Editor_Test.csv, Left click	Same as above but for the flag	1:00
7	Player places a crawler without a platform to crawl around	Editor_Test.csv, Left click	On play, the crawler does not spawn	1:10
7	Player places a crawler trapped inside 4 platforms	Editor_Test.csv, Left click	On play, the crawler does not spawn	1:25
---	The player tries to scroll out of bounds	Editor_Test.csv, left arrow	The editor won't scroll past the first column.	1:35

*The pygame mouse object tracks the mouse position relative to the window even if it leaves the window. This can lead to array indexing errors where we try to access the grid array in a row which doesn't exist.

Finally, we test extreme data (Video 9):

Req	Description	Input Data	Expected Result	Actual Result
1	Player tries to create a level called NEW LEVEL	NEW LEVEL used as a name	The editor loads and creates a file called NEW_LEVEL.csv. Returning to the file select screen will have two instances of NEW LEVEL	0:00
2, 7	Player draws over the player object, then tries to play	Editor_Test.csv, left click	The editor doesn't play the level, as there is no player object	0:30
2,7	Player draws over the flag object, then tries to play	Editor_Test.csv, left click	The editor doesn't play the level, as there is no flag object	0:35
2, 7	Player erases the player object with "player" as the active tile for drawing (White box test)	Editor_Test.csv, left click	The editor should recognise that the player wasn't drawing, so the player flag becomes empty	1:20
2, 7	Player erases the flag object with "flag" as the active tile for drawing (White box test)	Editor_Test.csv, left click	The editor should recognise that the player wasn't drawing, so the flag-object flag becomes empty	1:20
2, 7	Player erases another tile with "player" as the active tile for drawing (White box test)	Editor_Test.csv, left click	The editor should recognise that the player wasn't drawing, so the player flag stays empty	0:55
2, 7	Player erases another tile with "flag" as the active tile for drawing (White box test)	Editor_Test.csv, left click	The editor should recognise that the player wasn't drawing, so the flag-object flag stays empty	1:05

6	Elements can be placed in the corners of the level	Editor_Test.csv, left click, left arrow	The elements are drawn, as they are still in bounds	2:30

4.6 Test Videos

4.6.1 Video Links

Playlist: https://www.youtube.com/playlist?list=PLvlRDaHBGqXzaLUNgbPEz-aO85EU6Lo1t
Video 1: https://www.youtube.com/watch?v=PX4k7AjZ9ic
Video 2: https://www.youtube.com/watch?v=66k6vUhQ2_4
Video 3: https://www.youtube.com/watch?v=Ce1NBffZQrQ
Video 4: https://www.youtube.com/watch?v=FpkK5zH1YAs
Video 5: https://www.youtube.com/watch?v=IOKkboAyaGI
Video 6: https://www.youtube.com/watch?v=7VHhFJWoOqY
Video 7: https://www.youtube.com/watch?v=9JRZUnRvKcw
Video 8: https://www.youtube.com/watch?v=5ig2oAMLBAI
Video 9: https://www.youtube.com/watch?v=C29bGh1Y-mU

4.6.2 Video-Document Discrepancies, Errors and Notes

Repeated Discrepancies:
1) Underscores in level names are used to indicate spaces, as the text input system used in the project rejects underscores
2) Video numbers have been added to the document after recording
3) Timestamps for a test may not include the entire test, such as in video 1, I have timestamped a test at 4:50 so you don't need to watch 100 seconds of doing nothing, and can just see the aspect relevant to the test

Video 1 Discrepancies:
For the test starting at 2:30, I used 1.csv instead of Load_Test.csv.
For the tests starting from 5:20 – 5:55, I forgot to reset the lives value to 5. Everything works as intended, just relative to 4 lives instead of 5.

Video 1 Note:
I repeated the test at 4:40, as in the original test I died due to the shortened timer from 2:05.

Video 4 Error:
The test at 3:40 is not a white box test.

Video 5 Error:
The test at 0:35 is not an erroneous test – it is a normal test.

Video 9 Discrepancy:
For the tests at 0:55 and 1:05, I did not erase an existing tile, but rather, erased an empty space. The test is still successful, as it was a white-box test which recognized that I wasn't drawing a new player/flag.

Video 9 Note:
The tests at 1:20 were repeated, as in the original test I did not delete the player/flag tile.

5 Evaluation

5.1 Overview

The system meets all of the program requirements, with some extra quality of life extensions such as keyboard shortcuts in the level editor. The object oriented approach was definitely the right decision, allowing me to produce clearer structured code which can be by used dynamically at runtime – especially useful for the level editor, where a user could create anything.

To produce the system, I used a mix of waterfall and agile methodologies. I used the waterfall methodology to understand what the system would fully entail, and design a coherent solution accordingly. This also allowed me to perform research on existing systems and common algorithms, saving me time in development where I could reach a bottleneck due to a dispute between two components. I then used the agile methodology to develop my system, coding each element by itself and testing in dummy environments before integrating into the final system. This allowed me to test each component by itself before trying to fit it into the system. It also allowed me to produce milestones during development, such as "UI system works", "File handling works", etc, which helped improve productivity as I can see progress in action.

My system uses three types of file handling. The simplest is the save_data.txt file, treating each line as a save file – nothing complicated here. The second is literal file handling, creating .csv files for each new level and using the file names as entries in a listbox. Finally, I have used .csv files for my levels, which require the data to be tabulated – which I did using 2D arrays. As mentioned above, the csv files require the first entry to be used, otherwise it ignores rows until it finds one in use – not massively complicated to fix, but required trials to identify.

The use of the object-oriented paradigm has also helped my produce a complex system, with the game assets being built on inheritance of parent nodes in a class tree rooted at the Sprite class, allowing me to produce progressively more complicated classes without repeating a function across every asset. As the driver code is stored across 3 classes (MAIN, GAME, EDITOR), the system uses interfaces to interact with one another, allowing me to keep the game logic separate from the editor, and the editor separate from UI management. I have also used polymorphism in my solution, such as with the player class overriding the Dead() method of its superclass Combatant, or the platforms each having a polymorphic touched() method to implement their unique behaviours.

I have also used some complex algorithms to produce my solution, such as the A* algorithm for flyer pathfinding, and abstract data types like finite state machines and queues.

5.2 Review of Program Requirements

Req	Description	Review
The Game		
1	The player can move left and right using A/D respectively, or using the relative arrow key, as well as being able to jump by pressing W or the Up Arrow Key	As shown in many of the tests, the player can move their character using either of the control schemes
2	Include Mario's movement improvements (see 1.2.2)	Though slight, the player does experience acceleration and drag when they start/end movement, and the player can vary their jump height and perform coyote jumps
3	The game screen should scroll to keep the player on screen	The game will start to scroll once the player enters a margin at the edge of the screen
4	The game must contain the following tile types: Floor/Wall, One Way, Honey, Spike	All the tiles are present and have flawless collision. The special tiles also all function as described
5	Levels are loaded from a file, and are accurately assembled to be played	Levels are stored in a consistent format in a .csv file, which can be loaded and converted into a level at runtime
6	Beating a level causes the next level to begin	If there is a next level, the game does include level progression
7	Levels must be beaten in one hundred seconds	The game contains a 100 second timer which resets each level. When the timer reaches 0, the player dies, and the level resets
8	The player must be able to save	Beating a level or losing a life causes the game to automatically save
9	The player must be able to create a save (if space allows)	The game contains 3 save file slots, which can be assigned to if empty
10	The player must be able to delete an existing save	The game has a delete form to delete an existing file (done by setting the level value to -1)
11	The game must correctly load the relevant form (see 2.3)	All the forms are connected, as described by the finite state machine in 2.3
12	Contain the above health system	The player starts the game with 5 lives and 10 health
13	The player can attack enemies to lower their health (and the inverse)	If an enemy collides with the player's attack hitbox, that enemy loses 1 health. If the inverse happens, the player loses 1 or 2 health depending on the enemy's strength
14	Levels contain diverse enemies which can mutate between playthroughs	The game contains 3 different enemy types, which use completely different algorithms to move. Two enemy types are their own hitbox, whereas the other attacks like the player. Upon generation, they are randomly assigned stats from a given range
15	The player can restore health by using healing items	The game contains 3 tiers of healing items, which heal 1, 2, and 5 heal each. They are single use, and can only be used by the player

16	The player can update their movement options using consumable items	The consumable system for healing was reused for upgrades. The game contains 5 upgrades, 4 which alter movement, and 1 which alters attacks. The player can only have 1 upgrade at a time, and all the effects function correctly
	UI	
1	Menus respond to mouse input – buttons do a function when clicked, but images do not	As stated, only clickable elements respond to clicks, and each element responds to a click in the intended manner
2	Menus connect to one another	The player can move between one menu and another using only the buttons in the system itself
3	Menus can be dynamically updated at runtime	The menus generate their contents when they are loaded, and they are reloaded each time they are needed
	Editor	
1	The player can create custom levels	The player can create a custom level .csv file, so long as that file does not already exist
2	Levels are created by painting tiles onto a grid	By holding down click and dragging the mouse, the selected element is added to the screen – and to the grid structure in memory
3	The player can save the levels they create	At any time, the player can save their level
4	The player can edit the levels they have saved	In the LEVEL SELECT screen, the player can choose a custom level, and edit its contents
5	The player can play the levels they have saved	In the LEVEL SELECT screen, the player can choose a custom level, and play it
6	Elements can be placed anywhere	There are no restrictions on where an element can be placed, other than the element must be in the level
7	The game should check to make sure the level is playable	The level will not be loaded if the player does not add one flag and one player object

5.3 User Feedback

I sent the program to John to review on Friday 21st of March 2025. I then conducted an interview with him on Friday 28th of March 2025, which went as follows:

"Firstly, how do you feel about the gameplay?"
John - *"The gameplay is very fun, with a generally intuitive control scheme. I like the enemies that have been implemented, all being very distinct from one another, and the randomisation of their health and damage helps to make a replay more enjoyable. There are unfortunately too many tutorial type levels, which makes the start of the game way too easy, and massively slows down the difficulty curve. An alternative could be to add signs to these levels to explain what everything does, allowing you to create more complicated tutorial levels and get into proper gameplay sooner. Also, the lack of sound design is really apparent, and its inclusion could allow for more advanced feedback than just visual updates – for example, a sound effect to indicate that an attack has connected with an enemy"*

"What is your opinion on the file management I have made for the game?"
John - *"Your file select screen is perfect. Creating a save file is easy, and the ability to have multiple saves improves the replayability of the game. I will say, the captcha is a bit too extreme of a method to prevent accidental file deletion. I think a simple confirmation screen would have been better, as it can be quite annoying to fail the captcha and have to reload the delete screen."*

"Moving away from the game side of the system, how would you review the UI?"
John - *"The system provided functions perfectly and does what it needs to do. To turn this into an actual product, we would need to rework the graphics side of the system. The colour scheme makes buttons distinct from one another, but lacks a consistent theme. In this vein I really like the level select screen, which alternates the colours of the entries in the Listbox. The buttons are a bit too flat, and could benefit with some feedback while the mouse is down. The scrollbar really should have an indicator to show how far the user is into a listbox – though the 'jump to' feature is very handy. Overall, while the system functions logically, it is in desperate need of dynamic graphics."*

"Finally, what is your review on the editor?"
John - *"The editor is exactly what I wanted, and adds an extension to the system. The controls are easy to use, and levels are simple to create and test. I would comment that limiting scrolls to one tile per press is good for fine adjustments, but makes it really annoying to modify the other side of the level. It would be beneficial to have a system similar to holding backspace with the textbox, where you can hold the scroll button down to scroll multiple tiles."*

"So overall, how would you review the system?"
John - *"The system as a whole is great. The gameplay is fun, but let down by early level design. The UI works amazingly, but needs a graphical overhaul. The editor is excellent, only needing a slight modification to scrolling. So altogether, this is a system I will be happy with after I swap out the graphics."*

5.4 Improvements

5.4.1 Suggested Improvements

In this final interview, John mentioned a few extensions I could make to the system to improve it.

The major improvement would be an overhaul of the graphics system, and the graphics used. This was not considered much during development, and the project was just a demo. I would first update the sprite class to dynamically update the image each frame. This would allow buttons to provide visual feedback when pressed – namely by darkening the image. Next, I would determine a consistent colour palette for the game, which would probably centre around purple, brown and black to indicate the "magical monster-infested tower" theme to the game. Similarly, the images I use for the game assets would be updated to better fit into a world – as opposed to mono-chromatic squares. The final update (which John explicitly stated) would be to add an indicator to the scrollbar, which shows how deep into the scrollbar the user is. This would just require another class to act as the pointer, with some methods to determine the pointer size and position. With the dynamic system, the listbox could also automatically update as you drag the mouse on the scrollbar, making navigation much easier.

The next major improvement would be to include sound design into the project. Such a system would include background music and sound effects. This can be done using the pygame.mixer() object, which is a pre-built sound library. From there, I would just need to create the music files,

which can then be played when certain events take place in the game. This would require a global event listener, but that can easily be implemented into the MAIN class as a method. Some example sound effects I would add would be: button clicked, player hit, player jump, enemy hit, and enemy dead.

John has also selected some quality of life improvements. The first of which is a simpler captcha system, due to the tedious nature of a full-on 5 character captcha. He suggests using a confirmation screen, which gives the player time to review their decision. I agree that this is a better method, and can be implemented by just creating another screen – an easy task due to the modular nature of my system.
The second quality of life change would be to allow unlimited scrolling from one arrow press in the editor. John suggests a system similar to holding backspace with a textbox. To implement John's solution, I would need to lift the backspace code from the textbox and into the button.clicked() method. More specifically, I would need a subclass for these scroll buttons which overrides the clicked method to solve this problem.

The difficulty curve

We then have the adjustment of reducing the number of tutorial type levels, which John notices have drastically slowed the difficulty curve. He suggests adding signs which explain what everything does, which lets you move onto proper gameplay sooner. I think it could be beneficial to see what existing systems do solve this problem, as walls of text can distract from smooth gameplay.

Looking at the level design of the New Super Mario Bros games, Nintendo doesn't include tutorial levels. Instead, they use the first screen or two to silently explain the mechanic of the level, and then a screen or two to get the player comfortable with the mechanic. The rest of the level is then dedicated to small challenges which utilise that mechanic. A mechanic for a level could be as simple as introducing a new enemy, or it could be more complex, like an interaction between a power-up and a tile, or a completely gimmick level like a maze.

By using this methodology, I could design better levels by deciding the mechanic of each level first, then building a little playground around that mechanic for the player. A level curve I then easy to implement, as later levels can either be just harder implementations of an earlier mechanic (like more enemies on screen), or I could use more complicated mechanics for later levels – starting with just introducing the enemies one by one, then later on introducing mechanics such as sticking to the bottom of a honey tile to hang from a platform.

This design would also improve the level editor, as player's are more aware of the mechanics that they can create levels around.

5.4.2 Personal Improvements

One thing I would do if I were to revisit the project is to make an abstract latching function that can be called in Common_Fuctions, as latching ended up appearing a lot even outside of the UI, such as for shortcuts where you only want the action to take place once per shortcut. This would help clean up some areas of code, and in the process provide a more complex solution, due to an abstract latching function using reference parameters.

I would also review my implementation of the A* algorithm, which (while correct) is definitely a performance bottleneck when there is a large quantity of flyers on screen. Whilst this isn't a problem for most gameplay, it is an issue as it restricts the kinds of levels a user can create in the editor – even if a level of just flyers is evil.

If I were to implement sounds, I would also include an options menu to alter the volume. Such an addition could also include screen resolution options, as currently the program is restricted to 768 by 512, with no option to full screen. I would also include an option to modify the key binds for the game.

As well as the suggestions John provided for improving the UI, I would also add a search bar to let the user search for a custom level by name, saving a potentially long search if the device has many custom levels and improving user experience in the process.

A final improvement I would make, would be to include checkpoints. A checkpoint is simply a tile which – when reached – lets the player start there instead of all the way at the beginning. To implement this, I would need a new tile type for the checkpoint, and from there, I can just keep track of the current start-point in the GAME class.